BINARY TRADING PLATFORM

Platform Logic Reference

How the trading core, money, risk, market data, auth and the apps actually work — written from the code.

Generated 2026-10-14 · server-go, client, admin, mobile

Contents

1. How the platform works - logic reference
2. Trading core: business logic
3. Money: wallets, ledger, deposits, withdrawals, USDT
4. Platform logic: risk, market data, auth, realtime
5. Client-side logic: web (client/src) and mobile (mobile/src)

How the platform works - logic reference

These documents describe the rules the code actually implements. They were written by reading the code, not the brief: every statement cites the file it comes from, and where the code and the root `README.md` disagree, the documents say so.

#	Document	What it covers
1	Trading core	Trade lifecycle and statuses, the acceptance checks in the order they run (with error codes), how entry price and expiry are fixed, expiry detection and settlement, win/loss/draw and the draw policy, the single-settlement guarantee, early close pricing, assets and payout tables, DEMO vs LIVE, market sentiment, websocket events
2	Money, wallets and payments	Minor units and micro-USDT, LIVE/DEMO/FUNDING wallets, the lock model, the ledger and its idempotency keys, manual INR deposits and withdrawals, the USDT/BeeX logic, reconciliation, audit, which permission gates which money action
3	Risk, market data, auth, realtime	Exposure (net/gross), risk scoring and pre-trade limits, hedging, market data providers, feed health and candles, JWT sessions and refresh rotation, rate limits, RBAC, websocket protocol and events, workers and leader election, admin P&L
4	Client and mobile	HTTP and token handling, the realtime bridge, the stores, trade ticket rules, live result projection, chart logic and indicator formulas, formatting, the theme system, how the React Native app differs from the web client

Read 1 → 2 → 3 for the server, 4 for the apps.

The rules everything else follows

- **The server decides price, time and payout.** A trade request carries asset, direction, amount, expiry and account (plus an optional quoted price used only as a slippage guard). Nothing a client sends can change a result.
- **Money is an integer.** INR in paise, USDT in micro-USDT. No floating-point arithmetic on balances.
- **A trade settles once, a credit applies once.** Atomic status claims, unique indexes and ledger idempotency keys back each other up.
- **MongoDB is the record; Redis is a cache.** Exposure counters and the expiry queue are rebuilt from MongoDB at boot.
- **A stale feed stops new trades**, but open trades still settle on their published terms.
- **DEMO money never touches the house book** - no exposure, hedging, risk limits, P&L or platform reconciliation.

Found while writing these (not fixed)

Each item is explained, with file references, in the document named.

Things that do not work today

- **USDT is not connected** (doc 2, §5). The `usdt` and `beex` services are complete, but no route, webhook or worker uses them. The clients call `/wallet/usdt/*`, which the deployed API answers with `NOT_FOUND`

(checked against the live server on 2026-10-14).
- **Hedge realised P&L is always 0** (doc 3): nothing writes `realisedPnl`.

Crash and concurrency edge cases

- An early close that crashes between steps can leave the trade marked `DRAW`, settle a re-claimed trade before its expiry, or leave the ledger and the settlement row disagreeing (doc 1, §10).
- Early close prices off the last tick in Redis without the feed-health check (doc 1, §10).
- If no tick at or before expiry survives (ticks are kept 15 minutes), the expiry price falls back to a later tick (doc 1, §10).
- A manual deposit or withdrawal whose wallet step fails after APPROVED stays APPROVED with no way to retry or reject it; a withdrawal is saved and reserved in two separate steps; a retried admin adjustment is applied twice (doc 2).
- Every instance with workers enabled rebuilds exposure counters and the expiry queue at boot without a lock (doc 3, §8).
- Changing a payout closes the old row before inserting the new one; a failed insert leaves no active payout, and `minStake <= maxStake` is not checked (doc 1, §10).

Security

- The rate limiter fails open when Redis is down and trusts the first `X-Forwarded-For` value (doc 3, §5).
- Suspending a user revokes no tokens; access tokens and open sockets keep working until they expire (doc 3, §5).
- Any admin role can see platform exposure on the dashboard (doc 3, §9).
- The seed creates `risk@` and `finance@` accounts with passwords published in the root README; change them on any public deployment.

Where the root README is out of date

- Early close is not "one transaction": the Redis steps run after the MongoDB writes (docs 1 and 3).
- The tests it cites for concurrency and "hedged and unhedged settle the same" were Node suites that were never ported to Go (docs 1, 2, 3).
- `FEED_*_MS` only seed the first risk configuration; afterwards the admin console is the only control, and `feedStaleMaxMs` is never read (doc 3).
- Tick retention is a fixed 15-minute constant, not an env var; the server also accepts 3h and 1d chart frames (doc 3).
- Settlement runs on every instance, not only the leader (doc 3).
- It still says there is no payment provider and documents none of the `USDT_*` / `BEEX_*` variables (doc 2).
- "`utils/format.ts` is the only place that converts money" is not true: USDT has its own converters and a few screens divide by 100 by hand (doc 2).

Trading core: business logic

This document covers how the trading core decides things: how a binary trade is accepted, priced, timed, expired, settled and closed early, and how assets, payouts and the DEMO account fit in. Everything here comes from the Go backend in `server-go/`. Where a statement depends on a specific function, the file is cited. Where the code and the root `README.md` disagree, the code is treated as the reference and the disagreement is listed in section 11.

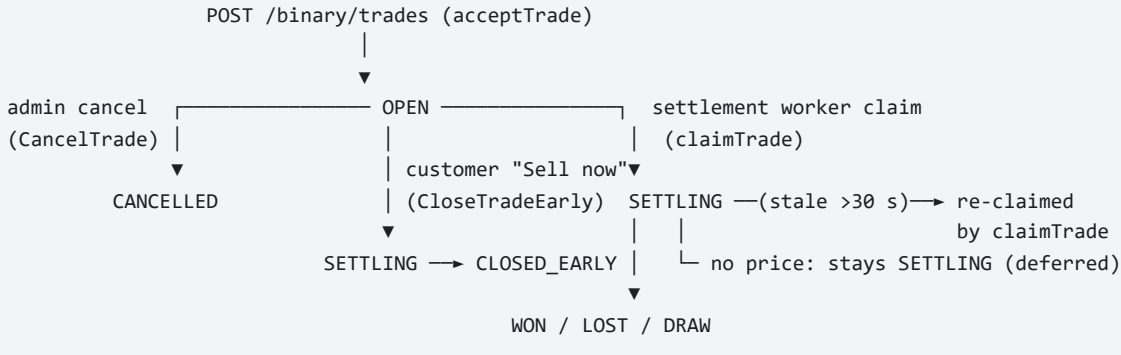
Conventions used throughout:

- **Money is always integer minor units** (paise). `10000` = ₹100.00. Prices are `float64`. Timestamps called `*Timestamp` are epoch milliseconds.
- `RoundHalfUp(x)` = `floor(x + 0.5)` (JS `Math.round`), from `server-go/internal/platform/util/util.go`.
- Error responses carry an error `code` and an HTTP status. Codes are defined in `server-go/internal/platform/apperr/errors.go`. `apperr.Validation` is **422**, `Unavailable` is **503**, `ConflictErr` is **409**, `NotFoundErr` is **404**, `Unauth` is **401** (`UNAUTHENTICATED`).
- Runtime-tunable rules live in the versioned **risk config** (`server-go/internal/models/risk_config.go`, defaults in `DefaultRiskConfig()`). It is cached for 2 s per process (`server-go/internal/service/riskconfig/riskconfig.go`), so a change can take up to about 2 s to reach every instance.

---

1. Trade lifecycle and status state machine

Statuses (`server-go/internal/models/domain.go`): `OPEN`, `SETTLING`, `WON`, `LOST`, `DRAW`, `CANCELLED`, `CLOSED_EARLY`. Results: `WIN`, `LOSS`, `DRAW`, `EARLY_CLOSE`.



Transition	Who	Guard	Code
(none) → `OPEN`	`binary.acceptTrade` from `POST /binary/trades`	full validation chain (section 2)	`server-go/internal/service/binary/binary.go`
`OPEN` → `SETTLING` (expiry)	settlement worker, `settlement.claimTrade`	conditional `FindOneAndUpdate` on `status: OPEN`	`server-go/internal/service/settlement/settlement.go`

Transition	Who	Guard	Code
`SETTLING` (stale) → `SETTLING`	settlement worker, `claimTrade`	`status: SETTLING` and `updatedAt < now − 30 000 ms` (`StaleSettlingMs`)	same
`SETTLING` → `WON` / `LOST` / `DRAW`	`settlement.applySettlement`	runs only after the claim, the wallet mutation and the settlement row insert	same
`OPEN` → `SETTLING` (early close)	`binary.CloseTradeEarly`	conditional update on `status: OPEN`	`server-go/internal/service/binary/earlyclose_service.go`
`SETTLING` → `CLOSED_EARLY`	`CloseTradeEarly`	follows its own claim	same
`OPEN` → `CANCELLED`	admin `POST /admin/binary/trades/{id}/cancel` (permission `PermTradesCancel`), `binary.CancelTrade`	conditional update on `status: OPEN` ; a `SETTLING` trade cannot be cancelled	`server-go/internal/service/binary/binary.go` , `server-go/internal/api/admin/trading.go`
repair: `OPEN` / `SETTLING` → final	`settlement.ensureTradeFinalised`	a settlement row already exists but the trade status was never updated	`server-go/internal/service/settlement/settlement.go`

"Live" statuses are `OPEN` and `SETTLING` (`binary.LiveStatuses`). Finished ones are `WON` , `LOST` , `DRAW` , `CANCELLED` , `CLOSED_EARLY` (`binary.FinishedStatuses`). List filters use `phase=OPEN|SETTLED` on top of these.

---

2. Trade acceptance

2.1 Route and request

`POST /binary/trades` (`server-go/internal/api/binary/routes.go`). Middleware runs in this order:

1. `httpx.Authenticate` .
2. `httpx.RequireActiveUser` : user missing → 401 `UNAUTHENTICATED` "Account no longer exists". Status not `ACTIVE` → 403 `ACCOUNT_DISABLED` . Role changed since the token was issued → 401 `TOKEN_INVALID` .
3. `RateLimit(RateLimits.Trade)` : 60 requests per minute per user, shared with `POST /trades/{id}/close` . Over the limit → 429 `RATE_LIMITED` .

Body (`parseCreateTrade`). A malformed JSON body → 400 `VALIDATION_ERROR` . A field failure → **422** `VALIDATION_ERROR` "Invalid request body" with per-field issues.

Field	Rule
`assetId`	required, 24-hex string
`direction`	required, `UP` | `DOWN`
`amount`	required, integer, > 0, ≤ 1 000 000 000 (minor units)
`expirySeconds`	required, integer, > 0, ≤ 86 400

Field	Rule
`quotedPrice`	optional, positive, finite. Used only by the slippage guard
`idempotencyKey`	optional, string of 8–128 chars
`accountType`	optional, `LIVE` | `DEMO`, default `LIVE`

The `Idempotency-Key` header is also accepted (first value, no length check). **The body key wins over the header** (`CreateTrade`). An empty key means no idempotency.

A request carries no entry price, no timestamps and no payout. The server decides all of them.

2.2 Order of operations

`binary.CreateTrade` → `acceptTrade` (`server-go/internal/service/binary/binary.go`). The whole flow runs on `context.WithoutCancel`, so it is not aborted if the client disconnects. Steps in order. The first failure stops the flow.

#	Step	Refusal	Status
0a	`accountType == FUNDING` (USDT wallet) is refused. The API enum already excludes it, so this is a defensive check	`VALIDATION_ERROR`	422
0b	**Idempotency resolution**, if a key is present (2.3). Runs **before** any business validation	`IDEMPOTENCY_PAYLOAD_MISMATCH` / `IDEMPOTENCY_IN_PROGRESS`	422 / 409
1	Global kill switch `riskConfig.tradingHalted` (`riskconfig.AssertTradingNotHalted`)	`TRADING_HALTED`	503
2	User still exists and is `ACTIVE`. This re-check reads `maxTradeAmount` and `maxUserExposure`	`UNAUTHENTICATED` / `ACCOUNT_DISABLED`	401 / 403
3a	Asset exists (`assets.RequireAssetDoc`, from the 2 s in-process cache)	`ASSET_NOT_FOUND`	404
3b	Asset `active` (`assets.AssertAssetTradable`)	`ASSET_INACTIVE`	409
3c	Inside trading hours (`util.IsWithinTradingHours`, section 6.2)	`MARKET_CLOSED`	409
4	`expirySeconds` ∈ `asset.supportedExpiries`	`EXPIRY_NOT_SUPPORTED`	400
5	**Feed gate** `marketdata.AssertFeedTradable` (2.4). Returns the tick that will price the trade	`FEED_UNHEALTHY` / `NO_PRICE_AVAILABLE`	503
5b	**Slippage guard**, only when `quotedPrice` was sent (2.5)	`PRICE_MOVED` (details: `serverPrice`, `quotedPrice`, `movePercent`, `tolerancePercent`)	409
6	Active payout row for (asset, expiry) (`payouts.RequireActivePayout`)	`PAYOUT_NOT_CONFIGURED`	409
7a	Per-expiry stake bounds from the payout row, when set: `stake < payout.minStake` / `stake > payout.maxStake`	`STAKE_BELOW_MIN` / `STAKE_ABOVE_MAX` ("…for this expiry is N")	400
7b	**Risk limits** `risk.AssertTradeWithinRiskLimits` (2.6)	first violation's code (`STAKE_BELOW_MIN`, `STAKE_ABOVE_MAX`, `USER_LIMIT_EXCEEDED`, `EXPOSURE_LIMIT_EXCEEDED`, `RISK_LIMIT_EXCEEDED`); `details.violations` lists all of them	400

#	Step	Refusal	Status
8	Server fixes `tradeId`, `entryTimestamp`, `expiryTimestamp` (section 3)	—	—
9a	DEMO only: `wallets.EnsureDemoWallet` creates and funds the DEMO wallet on first use	—	—
9b	**Reserve the stake**: `wallets.LockTradeFunds` (ledger key `trade:lock:<tradeId>`; guard `balance − lockedBalance ≥ stake`)	`INSUFFICIENT_BALANCE` (400); `WALLET_NOT_FOUND` (404) if the wallet row is missing; `SERVICE_UNAVAILABLE` (503) if the per-wallet Redis lock is not obtained within 15 s	
10	Insert the `binary_trades` document. On a duplicate key, release the stake (`ReleaseCancelled`) and, when the clash was on the idempotency key, return the winning trade as a replay	—	—
11	`ZADD expiry:queue <expiryTimestamp> <tradeId>`. For LIVE only, `exposure.ApplyExposureDelta(sign +1)`. Mark idempotency committed. Emit `trade.created`	—	—

Any refusal increments the `trades.rejected` metric and **deletes the Redis idempotency reservation**, so a retry with the same key is evaluated again from scratch. The handler writes an audit row: `TradeRejected` (failed, with the error message) or `TradeCreated` (with `replayed`). The response is **201** for a new trade and **200** with `meta.replayed = true` for a replay.

Steps 1–7 read caches: the risk config, the asset table and the payout table each have a 2 s TTL per process. Note that the stake is reserved (9b) *before* the trade is written (10). If the process crashes between the two, the customer's funds stay locked against a trade that does not exist. At worker boot, `wallets.ReleaseOrphanedTradeLocks` releases any `TRADE_LOCK` older than 60 s whose trade document does not exist, up to 500 per run (`server-go/internal/service/wallets/orphan_locks.go`, called from `server-go/internal/workers/workers.go`).

2.3 Idempotency

Two layers:

- **MongoDB**: a partial unique index on `{userId, idempotencyKey}` for string keys (`server-go/internal/models/binary_trade.go`). The trade stores `idempotencyFingerprint`.
- **Redis**: the key `idem:<userId>:<key>`, TTL 24 h. Its value is `in-progress:<fp>` while the request runs and `committed:<fp>:<tradeId>` after it succeeds.

The fingerprint (`TradeFingerprint`) is the SHA-256 of a canonical JSON of `{assetId, direction, amount, expirySeconds}`, plus `accountType` only when it is not LIVE (so fingerprints written before accounts existed still match). `quotedPrice` is **not** part of the fingerprint, so a retry with a fresh quote counts as the same request.

`resolveIdempotency`:

1. If a trade with (user, key) exists in Mongo: when its stored fingerprint is non-empty and differs → 422 `IDEMPOTENCY_PAYLOAD_MISMATCH`. Otherwise the stored trade is returned (`replayed: true`, no side effects).
2. Otherwise `SETNX` the Redis key with `in-progress:<fp>`. If the set succeeds, the request proceeds.

3. If the Redis key already holds `in-progress:` with another fingerprint → 422 `IDEMPOTENCY_PAYLOAD_MISMATCH`. With the same fingerprint → 409 `IDEMPOTENCY_IN_PROGRESS`. Any other value lets the request proceed, and the unique index catches a true duplicate at insert (step 10).

2.4 Feed gate (`assertFeedTradable`)

`server-go/internal/service/marketdata/marketdata.go`. Feed health comes from the last tick in `md:tick:<assetId>`:

```
no tick                      → OFFLINE
ageMs = now − tick.receivedAt
ageMs > feedTickMaxAgeMs     → OFFLINE                      (default 5000)
effective = max(tick.totalLatencyMs, ageMs)
effective ≤ feedHealthyMaxMs → HEALTHY                      (default 100)
effective ≤ feedDegradedMaxMs→ DEGRADED                     (default 300)
otherwise                    → STALE                        (feedStaleMaxMs default 1000; only used for config
validation)
```

A trade is refused with 503 `FEED_UNHEALTHY` when the status is `OFFLINE`, or `STALE` with `haltTradingOnStaleFeed` (default `true`), or `DEGRADED` with `haltTradingOnDegradedFeed` (default `false`). With the defaults, `HEALTHY` and `DEGRADED` feeds can trade. If the last tick disappears between the health check and the re-read → 503 `NO_PRICE_AVAILABLE`. On first boot the feed thresholds are copied from the env (`FEED_*_MS`) into risk config version 1.

2.5 Slippage guard (`quotedPrice`)

`server-go/internal/service/binary/slippage.go`. The quote is only ever used to decide whether to accept. It is never used as the entry price.

```
if quotedPrice is not finite or ≤ 0     → accept (no opinion)
movePercent = (serverPrice − quotedPrice) / quotedPrice × 100
adverse     = movePercent > 0 for UP,  movePercent < 0 for DOWN
if slippageTolerancePercent ≤ 0         → accept (guard disabled)
if !adverse                             → accept (a favourable move is never refused)
accept iff |movePercent| ≤ slippageTolerancePercent   (default 0.1; boundary accepted)
```

`serverPrice` is the tick returned by the feed gate. On refusal, `movePercent` is rounded to 4 decimals (JS `toFixed` semantics) in the error details.

2.6 Risk limits

`server-go/internal/service/risk/risk.go`, `CheckTradeLimits` (pure) fed by `AssertTradeWithinRiskLimits`. The function collects **every** violation, in this order:

Check	Formula	Code
min stake	`stake < max(asset.minStake, cfg.minTradeAmount)`	`STAKE_BELOW_MIN`
max stake	`stake > min(asset.maxStake, cfg.maxTradeAmount, user.maxTradeAmount ?? ∞)`	`STAKE_ABOVE_MAX`
open trades	count of the user's trades with `status: OPEN` **on the same account** ≥ `cfg.maxOpenTradesPerUser` (SETTLING not counted)	`USER_LIMIT_EXCEEDED`

Check	Formula	Code
DEMO stops here		
user exposure	`userTotalExposure + stake > (user.maxUserExposure ?? cfg.maxUserExposure)`	`USER_LIMIT_EXCEEDED`
asset gross	`asset.gross + stake > min(asset.exposureLimit, cfg.maxAssetExposure)`	`EXPOSURE_LIMIT_EXCEEDED`
asset net	`|asset.net ± stake| > asset.netExposureLimit` (+ for UP, − for DOWN)	`EXPOSURE_LIMIT_EXCEEDED`
platform gross	`portfolio.totalGross + stake > cfg.maxGrossExposure`	`RISK_LIMIT_EXCEEDED`
concentration	only if `projectedGross > userExposureLimit`: `(userAssetExposure + stake) / projectedGross × 100 > cfg.maxUserAssetConcentrationPercent`	`RISK_LIMIT_EXCEEDED`

Exposure figures come from the Redis counters (section 7) and are stake-based: `net = upStake − downStake`, `gross = upStake + downStake`.

---

3. What the server fixes on the trade

All values are set in `acceptTrade` and frozen on the document. Later changes to the payout table or the risk config never touch an existing trade.

Field	Value
`_id`	new ObjectID, created before the wallet lock (the lock's ledger key uses it)
`reference`	`"BT_" + base64url(9 random bytes)` (`util.Reference`), unique index
`entryPrice`	`tick.price` of the last stored tick returned by `AssertFeedTradable` (step 5)
`entryTickSequence`, `entryProviderTimestamp`, `entryFeedLatencyMs`	provenance of that tick (`entryFeedLatencyMs = tick.totalLatencyMs`)
`entryTimestamp`	`util.NowMs()` at step 8, i.e. server time *after* validation, not the tick's time
`expiryTimestamp`	`entryTimestamp + expirySeconds × 1000`
`payoutPercent`, `payoutConfigId`, `payoutConfigVersion`	from the active payout row at step 6
`profit`	`RoundHalfUp(stake × payoutPercent / 100)` (`util.CalculateProfit`)
`currency`	always `"INR"` (`BinaryTradeDefaultCurrency`)
`accountType`	`LIVE` or `DEMO` (missing on old documents = LIVE)
`status`	`OPEN`

Field	Value
`ip`	client IP from the request

`potentialReturn` is not stored. The view computes it as `stake + profit` (`models.ToTradeView`). Example: stake 10 000 at 82 % → profit 8 200, potential return 18 200.

---

4. Expiry detection and settlement

4.1 Queues

Redis key	Type	Producer	Consumer
`expiry:queue`	ZSET, score = `expiryTimestamp`, member = tradeId	`acceptTrade`; `RebuildExpiryQueue` at boot	expiry worker
`settlement:queue`	LIST (`LPUSH` / `BRPOP` + `RPOP` = FIFO)	expiry worker, sweeper, settlement worker (re-queue)	settlement worker

Redis is a cache here. At worker start (`server-go/internal/workers/workers.go`) the exposure counters (`RebuildExposureFromDatabase`) and the expiry queue (`binary.RebuildExpiryQueue`, all `OPEN` + `SETTLING` trades) are rebuilt from MongoDB before any loop starts.

4.2 Expiry worker (leader-elected)

`server-go/internal/service/expiry/worker.go`, `expiry.go`.

- Runs through `leader.New{Name: "expiry", Interval: EXPIRY_TICK_MS (default 100, min 10), LockTTL: 15 s}`. Leadership is a Redis `SETNX` lock `lock:worker:expiry` that is renewed on every tick (`server-go/internal/platform/leader/leader.go`). If the leader dies, another instance takes over within one TTL. Ticks never overlap. `WORKERS_ENABLED=false` disables the loop.
- Each tick, `ProcessExpiries(now, 200)`: 1. `FindDueTrades`: `ZRANGEBYSCORE expiry:queue -inf now LIMIT 0 200`, then loads those trades with status `OPEN` / `SETTLING`. Queue ids that don't match a live trade are `ZREM`ed as orphans. 2. For each due trade, with `elapsed = now − expiryTimestamp`: if `elapsed < 2000 ms` (`ExpiryTickGraceMs`) **and** no tick with provider timestamp ≥ `expiryTimestamp` exists (`HasTickAtOrAfter`), the trade waits for the next tick. This makes sure a tick still in flight is not missed. 3. Otherwise: emit `trade.expiring`, `LPUSH settlement:queue`, `ZREM expiry:queue`, record `expiry.detection` latency. **Nothing is settled here.**
- Every `round(30 000 / interval)` ticks (about 30 s), `SweepMissedExpiries(now, 500)` pushes back onto `settlement:queue` any trade that is `OPEN` with `expiryTimestamp ≤ now − 2000`, or `SETTLING` with `updatedAt < now − 30 s`.

4.3 Settlement worker (not leader-elected)

`server-go/internal/service/settlement/worker.go`. Every instance with workers enabled consumes the queue. It does a `BRPOP settlement:queue` with a 1 s timeout on a dedicated connection, then drains up to `SETTLEMENT_BATCH_SIZE` (default 200) ids with `RPOP`. Each id runs `SettleTrade` on a fresh background context

with a 60 s timeout. Errors are logged and the trade is left for the sweeper. A `SKIPPED_NO_PRICE` outcome is re-`LPUSH`ed. On a consumer error the loop backs off 250 ms.

4.4 `SettleTrade` step by step

`server-go/internal/service/settlement/settlement.go`:

1. If a `settlements` row exists for the trade, run `ensureTradeFinalised` (repair the trade status if needed) and return `ALREADY_SETTLED`.
2. `claimTrade`: atomic `FindOneAndUpdate` with `{_id, $or: [{status: OPEN}, {status: SETTLING, updatedAt < now-30s}]}` → `{status: SETTLING, updatedAt: now}`. No match → `CLAIMED_ELSEWHERE`.
3. **Expiry price**, `marketdata.GetPriceAtOrBefore(assetId, expiryTimestamp)`: - the newest tick in `md:hist:<assetId>` (ZSET scored by **provider timestamp**, i.e. market time) with score ≤ `expiryTimestamp` → source `TICK_AT_OR_BEFORE_EXPIRY`; - otherwise the current last tick → source `LAST_KNOWN_TICK`; - otherwise `nil` → return `SKIPPED_NO_PRICE` and leave the trade in `SETTLING` (**deferred, never guessed**).
4. `DetermineResult(direction, entryPrice, expiryPrice)`: exact float equality → `DRAW`. Otherwise UP wins if `expiry > entry`, DOWN wins if `expiry < entry`. The smallest move decides.
5. `ApplyDrawPolicy(result, cfg.drawPolicy)`: `REFUND` (default) keeps `DRAW`, `LOSS` turns a draw into `LOSS`, `WIN` turns it into `WIN`. The policy never changes a decided result. The policy-adjusted result is what gets stored; the raw draw is not recorded. `GET /assets` discloses the policy as `drawPolicy` on every asset.
6. `applySettlement`:
7. **Wallet first**, keyed `trade:settle:<tradeId>` (amounts below).
8. Insert a `settlements` row (unique on `tradeId`). A duplicate key returns `ALREADY_SETTLED`.
9. Update the trade: `status`, `result`, `expiryPrice`, `expiryPriceSource`, `settlementAmount`, `platformResult`, `settledAt`, `settlementLatencyMs = now − expiryTimestamp`.
10. `ZREM expiry:queue`. For LIVE only, `ApplyExposureDelta(sign −1)`.
11. Emit `trade.settled`. Create a `TRADE_SETTLED` notification (fire and forget, emits `notification.created`). Update metrics.

4.5 Settlement amounts

Result	Final status	Wallet op (type / movement)	Balance Δ	Locked Δ	`payoutAmount` (row)	`platformResult`	`settlementAmount` (trade)
WIN	`WON`	`TRADE_WIN` / CREDIT, amount = profit	+profit	−stake	stake + profit	−profit	+profit
LOSS	`LOST`	`TRADE_LOSS` / DEBIT, amount = stake	−stake	−stake	0	+stake	−stake
DRAW (REFUND)	`DRAW`	`TRADE_REFUND` / UNLOCK	0	−stake	stake	0	0
EARLY_CLOSE	`CLOSED_EARLY`	`TRADE_EARLY_CLOSE` / CREDIT or DEBIT, amount = |value − stake|	value − stake	−stake	value	stake − value	value − stake
(cancel)	`CANCELLED`	`TRADE_CANCEL` / UNLOCK	0	−stake	*no row*	0	0

At open, `TRADE_LOCK` / LOCK has balance Δ 0 and locked Δ +stake. Every wallet mutation goes through `applyMutation` (`server-go/internal/service/wallets/wallet.go`): a Redis lock per (user, account, currency) (TTL 10 s, wait 15 s), a replay check on `idempotencyKey` , a guarded `$inc` with `$expr` pre-conditions (locked ≥ stake for releases, otherwise `LOCKED_FUNDS_MISMATCH` 409) and the ledger row, all in one Mongo transaction (when `ENABLE_MONGO_TRANSACTIONS=true`).

4.6 Single-settlement guarantee

Three independent guards. Each alone stops a double payout:

1. **Claim**: only one caller can move `OPEN → SETTLING` . Early close uses the same conditional update.
2. **Wallet key** `trade:settle:<id>` : partial unique index on `wallet_transactions.idempotencyKey` . Win, loss, refund, cancel *and* early close all use this key, so money moves at most once per trade by any path. A replay returns the existing transaction id without moving money.
3. `settlements.tradeId` **unique index**: at most one ledger row per trade.

4.7 Recovery paths and deferral

- **Orphaned SETTLING** (a worker crashed after the claim): `claimTrade` itself re-claims a `SETTLING` trade whose `updatedAt` is more than 30 s old, and the sweeper re-queues such trades about every 30 s.
- **No price**: the trade stays `SETTLING` with a fresh `updatedAt` . The worker re-queues it right away, but that retry hits `claimTrade` within 30 s and returns `CLAIMED_ELSEWHERE` , which is not re-queued. **In practice the retry is the sweeper**, about 30–60 s later.
- **Row exists but trade not finalised**: `ensureTradeFinalised` sets the final status from the row (`settlementAmount = payoutAmount − stake` for WIN, `−stake` for LOSS, otherwise 0).
- **Queue lost**: `RebuildExpiryQueue` at boot, plus the DB sweep.

---

5. Early close ("Sell now")

5.1 Quote formula

`server-go/internal/service/binary/earlyclose.go` , `QuoteEarlyClose` (pure):

```
move        = currentPrice − entryPrice
signedMove  = move  (UP)   |  −move (DOWN)
inTheMoney  = signedMove > 0

P(win):
  secondsRemaining ≤ 0              → 1 if signedMove > 0 else 0
  expectedMove = currentPrice × σ_perSecond × √secondsRemaining
  expectedMove ≤ 0 / NaN / Inf      → 0.5 if signedMove == 0, 1 if > 0, 0 if < 0
  otherwise                         → Φ(signedMove / expectedMove)
P(win) is then clamped to [0.01, 0.99]   (always, including the cases above)

fullPayout = stake × (1 + payoutPercent/100)
fee        = clamp(earlyCloseFeePercent, 0, 100) / 100
value      = max(0, RoundHalfUp(fullPayout × P(win) × (1 − fee)))      minor units
customerResult = value − stake
winProbability = P(win) rounded to 4 dp
```

`Φ` is `0.5 × (1 + erf(z/√2))`, with erf from Abramowitz–Stegun 7.1.26.

σ per second (`volatilityPerSecond`, `earlyclose_service.go`): `σ_tick = stddev of log returns` over the ticks in the last 60 s (`GetRealisedVolatility`, sample stddev, needs ≥ 3 ticks, otherwise 0), then `σ_perSecond = σ_tick × √max(0.1, ticksInWindow / 60)`. When σ is 0 the "still market" branch applies.

Because P(win) ≤ 0.99 and fee ≥ 0, the quote is always below the full payout. Selling a winning position early pays less than holding it to expiry.

5.2 Availability

Risk-config fields: `earlyCloseEnabled` (default `true`), `earlyCloseFeePercent` (default 5, admin range 0–50), `minSecondsRemainingForEarlyClose` (default 10, admin range 0–3600).

`GET /binary/trades/{id}/quote` (`QuoteTrade`) **never mutates anything** and never fails for an un-closeable trade. It returns `{value, customerResult, winProbability, inTheMoney, tradeId, currentPrice, secondsRemaining (rounded), available, unavailableReason}`. `unavailableReason` is the first that applies:

1. `!earlyCloseEnabled` → "Closing early is currently disabled"
2. `status != OPEN` → "This trade is already _ → space>"
3. no last tick → "No live price is available for this asset" (the quote then uses `entryPrice` as the current price)
4. `secondsRemaining < minSecondsRemainingForEarlyClose` → "Too close to expiry to close early"

Someone else's trade, or an unknown id, gives 404 `TRADE_NOT_FOUND`. It is never 403, so the response does not reveal that the trade exists.

5.3 Close path

`POST /binary/trades/{id}/close` → `CloseTradeEarly` (requires an active user; same 60/min bucket as opening). The checks run in this order:

#	Check	Error
1	`earlyCloseEnabled`	503 `SERVICE_UNAVAILABLE`
2	trade exists and is the caller's	404 `TRADE_NOT_FOUND`
3	`status == OPEN`	409 `ALREADY_SETTLED` "This trade is no longer open"
4	`secondsRemaining ≥ minSecondsRemainingForEarlyClose`	409 `CONFLICT` "Too close to expiry…"
5	last tick exists	503 `NO_PRICE_AVAILABLE`
6	compute the quote from that tick (**price first, then claim**)	—
7	claim `OPEN → SETTLING`	lost race → 409 `ALREADY_SETTLED`
8	`wallets.SettleTradeEarlyClose(value)`: one guarded op, key `trade:settle:<id>`	wallet errors

#	Check	Error
9	insert a `settlements` row with `result: EARLY_CLOSE`, `expiryPrice = tick.price`, `payoutAmount = value`, `platformResult = stake − value`, `settledBy: "customer.early_close"` (a duplicate key is only logged)	—
10	trade → `CLOSED_EARLY`, `result: EARLY_CLOSE`, `expiryPriceSource: "EARLY_CLOSE_AT_MARKET"`, `settlementAmount = customerResult`, `platformResult`, `settledAt`	—
11	`ZREM expiry:queue`; for LIVE, `ApplyExposureDelta(sign −1)`; emit `trade.settled`; audit `trade.closed_early`	—

The close path does **not** check feed health, the global halt or trading hours. It uses whatever last tick is in Redis (section 10). It creates no settlement notification.

---

6. Assets and payouts

6.1 Asset fields and defaults

`server-go/internal/models/asset.go`. Defaults are applied by `assets.CreateAsset` (`server-go/internal/service/assets/assets.go`):

Field	Meaning	Default
`symbol`	upper-cased, trimmed, unique	—
`name`, `category` (`CRYPTO` / `FOREX` / `STOCK` / `COMMODITY` / `INDEX`)		—
`active`	trading enabled	`true`
`providerSymbol`, `provider`	feed mapping (provider symbol matched case-insensitively)	provider `simulated`
`pricePrecision`	display decimals	2
`minStake` / `maxStake`	asset stake bounds	1 000 / 10 000 000
`supportedExpiries`	seconds	`[30, 60, 300]`
`exposureLimit`	max gross exposure	500 000 000
`netExposureLimit`	max one-sided exposure	200 000 000
`hedgeEnabled`, `targetHedgeRatio`, `hedgeThresholdPercent` (null = global), `minHedgeNotional`, `maxHedgeNotional`	hedging only	`false`, 0.5, null, 100 000, 100 000 000
`tradingWindows`	`[{days: [ISO 1..7], startMinuteUtc, endMinuteUtc}]`	`[]` (always open)
`displayOrder`		0

The asset table is cached in-process for 2 s and invalidated on write by the writing instance. Other instances converge within the TTL.

6.2 Trading hours, `marketOpen`, `nextOpenAt`

`util.IsWithinTradingHours` (`server-go/internal/platform/util/util.go`):

- No windows means always open.
- A window matches when today's ISO weekday (UTC) is in `days` and the UTC minute is in `[start, end)`.
- If `start > end`, the window wraps **within the listed day**: it is open `00:00–end` and `start–24:00` on that same day. It does not spill into the next day.

`AssetView` (`ToAssetView`) adds `marketOpen = active && withinHours` and `nextOpenAt` (epoch ms). `nextOpenAt` is `null` when the market is open, the asset is inactive, or there are no windows. Otherwise it is the start of the next open span, searched over today plus 7 days (`util.NextTradingOpen`). It can also be null if the windows never open.

6.3 Payout tables and versioning

`server-go/internal/service/payouts/payouts.go`, `server-go/internal/models/payout_config.go`.

- Row fields: `assetId`, `expirySeconds`, `payoutPercent` (82 ⇒ 82 % profit), optional `minStake` / `maxStake`, `version`, `active`, `effectiveFrom`, `effectiveTo`, `createdBy`, `note`.
- Indexes: unique `{assetId, expirySeconds, version}`, and a partial unique `{assetId, expirySeconds}` where `active: true`, so at most one active row per (asset, expiry).
- **Upsert** (admin `POST /admin/assets/{id}/payouts`, `PermPayoutsWrite`; synchronous audit `PayoutUpdated`): the expiry must be in `asset.supportedExpiries` and the percent in 0–500 (otherwise 422). The current active row is closed (`active=false, effectiveTo=now`) *before* the new row is inserted with `version = latest + 1`. Rows are never edited in place.
- **Deactivate** (`DELETE /admin/assets/payouts/{id}`): sets `active=false`, `effectiveTo=now`. New trades on that expiry then fail with `PAYOUT_NOT_CONFIGURED`.
- Active rows are cached for 2 s per process. Trades record `payoutConfigId` / `payoutConfigVersion`, so a trade can always be traced back to the exact terms it was accepted under.

6.4 Stake bounds, effective resolution

```
effectiveMin = max(payout.minStake (if set), asset.minStake, cfg.minTradeAmount)
effectiveMax = min(payout.maxStake (if set), asset.maxStake, cfg.maxTradeAmount,
                   user.maxTradeAmount (if set), 1 000 000 000 API cap)
```

The payout bounds are checked first (step 7a, message "...for this expiry..."). The asset, config and user bounds come next, inside the risk check (step 7b, message "...for ..."). Both use `STAKE_BELOW_MIN` / `STAKE_ABOVE_MAX`. Defaults: `cfg.minTradeAmount = 1 000`, `cfg.maxTradeAmount = 10 000 000`.

---

7. DEMO vs LIVE

Each user has a LIVE wallet and a DEMO wallet. The DEMO wallet is opened lazily by `EnsureDemoWallet` and funded once with `DemoOpeningBalance = 1 000 000` (₹10 000), keyed `demo:open:<userId>` (`server-`

`go/internal/service/wallets/demo.go`). It can be reset only when no DEMO trade is `OPEN`/`SETTLING` and nothing is locked. The USDT `FUNDING` wallet cannot trade.

Aspect	LIVE	DEMO
Kill switch, user status, asset, hours, expiry, feed gate, slippage, payout	yes	yes, identical
Stake bounds, open-trade count	yes	yes (open-trade count is **per account**)
User/asset/net/platform exposure, concentration limits	yes	**skipped** (`CheckTradeLimits` returns early)
Exposure counters (`exp:asset:*`, `exp:user:*`), sentiment	`ApplyExposureDelta` ±1	never touched; excluded from the rebuild (`NotDemo()`)
Hedge evaluation	queued via `hedge:queue` on each exposure delta	none (no exposure delta)
Wallet, ledger keys, settlement path, draw policy, early close	same	same (DEMO wallet)
Metrics `trades.volume`, `platform.gross_result`, `customer.payout`	counted	not counted
Admin P&L / dashboards (`server-go/internal/service/admin/admin.go`)	included	excluded via `models.NotDemo()`
Reconciliation `platform_result_consistency`	included	excluded
Reconciliation `locked_vs_open_positions`, `settled_trades_vs_settlements`	included	included (per wallet / overall counts)
Idempotency fingerprint	no `accountType`	includes `accountType`

`accountType` is stored on the trade and on the settlement row. A missing value is read as LIVE (`models.NormAccountType`). The list filter `accountType=LIVE` also matches old documents without the field.

---

8. Market sentiment

`server-go/internal/service/exposure/sentiment.go`, served only on `GET /assets` (the list, not `GET /assets/{id}`) as `sentiment` per asset (`server-go/internal/api/assets/routes.go`).

```
DeriveSentiment(upStake, downStake):
  total = upStake + downStake
  total ≤ 0  → nil                      → upPercent = downPercent = null
  up   = RoundHalfUp(upStake / total × 100)
  down = 100 − up
```

Inputs are the LIVE exposure counters (stake-weighted, not trade-count weighted). They cover trades from acceptance until their exposure is released at settlement, cancellation or early close, so `SETTLING` trades still count. `openTrades` is also returned. **Amounts are never exposed**, only ratios, because the book size would tell a trader how close an exposure limit is.

The result is `null`, not 50/50, for an empty book. A made-up even split would read as real crowd information, so the client renders `--` instead.

---

9. Websocket events

Events are published through Redis pub/sub (`pubsub:broadcast`, fire and forget, `server-go/internal/platform/ws/events.go`). Names are in `server-go/internal/config/constants.go`. Channels: `user:<id>`, `admin:ops`, `admin:risk`.

Event	Channels	Payload	Emitted by
`wallet.updated`	`user:<id>`	`WalletView` (incl. `accountType`, `availableBalance`)	every applied wallet mutation (`performMutation`): stake lock at acceptance, settlement, refund, cancel, early close. Not sent when a replay is detected before the transaction
`trade.created`	`user:<id>`, `admin:ops`	`BinaryTradeView`	end of `acceptTrade` (not on replays)
`exposure.updated`	`admin:risk`	`ExposureSnapshot`	each LIVE `ApplyExposureDelta`
`trade.expiring`	`user:<id>`	`{tradeId, symbol, expiryTimestamp}`	expiry worker, when the trade is handed to the settlement queue (not by the sweeper)
`trade.settled`	`user:<id>`, `admin:ops`	`BinaryTradeView` (final)	`applySettlement` ; `CloseTradeEarly`
`trade.updated`	`user:<id>`, `admin:ops`	`BinaryTradeView`	admin `CancelTrade`
`notification.created`	`user:<id>`	notification view	`TRADE_SETTLED` notification after expiry settlement

Typical LIVE sequence: `wallet.updated` (lock) → `exposure.updated` → `trade.created` → … → `trade.expiring` → `wallet.updated` (settle) → `exposure.updated` → `trade.settled` → `notification.created`. Each event is published from its own goroutine, so the order clients receive them in is not guaranteed.

---

10. Edge cases worth knowing

- **Entry timestamp vs entry tick.** `entryPrice` comes from a tick read at step 5, while `entryTimestamp` is taken at step 8. The tick can be up to the accepted feed age old (≤ 300 ms with default thresholds).
- **Draw equality is exact `float64` equality** on stored prices.
- **The expiry price can come from after expiry.** The `LAST_KNOWN_TICK` fallback is used when no tick ≤ expiry is left in `md:hist`. Tick history and the last-tick hash are kept only 15 minutes (`TickHistoryRetentionMs`), and only in Redis. After a long outage or a Redis loss, a trade can settle on a tick published after its expiry.
- **The 2 s grace is not a freshness check.** If no post-expiry tick arrives, the trade settles on the newest tick ≤ expiry, however old it is.

- **Cancelling** first sets `CANCELLED`, then releases funds. If the release fails, the trade is cancelled but the stake stays locked (`locked_vs_open_positions` reconciliation flags it). Cancelled trades have no settlement row.
- **Early close and crashes.** A crash after the early-close claim leaves a `SETTLING` trade that the sweeper re-claims after 30 s. If its expiry has not passed yet, `GetPriceAtOrBefore` returns the newest tick, so it settles *before* expiry at the current price. A crash after the early-close wallet op but before the row makes expiry settlement replay the wallet key (no money moves) and write a WIN/LOSS/DRAW row whose amounts differ from what was paid. A crash after the row but before the trade update makes `ensureTradeFinalised` map `EARLY_CLOSE` to status **`DRAW`** (`finalStatusFor` has no `EARLY_CLOSE` case) with `settlementAmount 0`.
- **Early close uses any last tick.** It does not check feed health, so the tick can be up to 15 min old. With `minSecondsRemainingForEarlyClose = 0` (allowed by the admin API), a trade still `OPEN` during the 2 s post-expiry grace can be closed for `0.01 × fullPayout × (1−fee)` even when it has already lost.
- **Payout upsert is not atomic.** The old row is closed, then the new one is inserted. A failed insert leaves no active row. The risk config, by contrast, reinstates the previous version. There is no `minStake ≤ maxStake` check.
- **Removing an expiry** from `supportedExpiries` does not deactivate its payout rows. Acceptance refuses it at step 4 anyway.
- `ExpiryTickSequence` / `ExpiryProviderTimestamp` exist on the trade model but are never written. `RedisKeys.ExpiryClaim` is defined but unused.

---

11. Code vs README contradictions

1. **Early close "in the same transaction".** The README says exposure is released and the trade removed from the expiry queue "in the same transaction". The code (`earlyclose_service.go`) does these as separate Redis calls *after* the Mongo writes, with no transaction.
2. **"The request carries only asset, direction, amount and expiry."** The body also accepts `quotedPrice`, `idempotencyKey` and `accountType`. None of them can influence the price or the outcome, but they exist.
3. **"There is a test that proves the outcome is identical hedged and unhedged."** No such Go test exists. It was a Node suite that the README itself later lists as not ported. The Go tests cover only pure functions (`DetermineResult`, `ApplyDrawPolicy`, `QuoteEarlyClose`, `AssessSlippage`, summaries, risk limits).
4. **Payout `minStake` / `maxStake` "null falls back to the asset bounds"** (comment in `server-go/internal/models/payout_config.go`). The asset bounds always apply in the risk check, whether or not the payout bounds are set. This is a code-comment inaccuracy, not a README one.
5. **No-price retry** (code comment, not the README). The comment in `server-go/internal/service/settlement/worker.go` says the `SKIPPED_NO_PRICE` re-queue lets "the next pass try again once a price arrives". That re-queue always ends in `CLAIMED_ELSEWHERE`, because the claim is fresh. The real retry is the stale-SETTLING sweep (30–60 s), which the comment in `settlement.go` describes correctly. The README only says settlement "defers rather than guessing", which is accurate.

Points that do match the README: the wallet lock model (open: locked += stake; win: balance += profit; loss: balance −= stake; draw: release), the concentration rule only applying to a material book, settlement not being

leader-elected, DEMO excluded from exposure/hedging/P&L, the ₹10 000 DEMO balance, and the quote formula and clamp.

---

12. Where to look

Topic	Files
Trade API, body validation, audit	`server-go/internal/api/binary/routes.go`
Acceptance chain, idempotency, cancel, queue rebuild	`server-go/internal/service/binary/binary.go`
Slippage guard	`server-go/internal/service/binary/slippage.go`
Early-close pricing / service	`server-go/internal/service/binary/earlyclose.go` , `earlyclose_service.go`
Trade summary totals	`server-go/internal/service/binary/summary.go`
Expiry detection, sweeper, worker	`server-go/internal/service/expiry/expiry.go` , `worker.go`
Settlement, draw policy, consumer	`server-go/internal/service/settlement/settlement.go` , `worker.go`
Feed health, expiry price, volatility	`server-go/internal/service/marketdata/marketdata.go`
Risk limits	`server-go/internal/service/risk/risk.go`
Risk config and defaults	`server-go/internal/service/riskconfig/riskconfig.go` , `server-go/internal/models/risk_config.go` , `server-go/internal/api/admin/core.go`
Wallet mutations, ledger keys	`server-go/internal/service/wallets/wallet.go` , `demo.go` , `orphan_locks.go`
Exposure counters, sentiment	`server-go/internal/service/exposure/exposure.go` , `sentiment.go`
Assets, trading hours	`server-go/internal/service/assets/assets.go` , `server-go/internal/models/asset.go` , `server-go/internal/platform/util/util.go`
Payouts	`server-go/internal/service/payouts/payouts.go` , `server-go/internal/models/payout_config.go` , `server-go/internal/api/admin/assets.go`
Asset catalogue API (`drawPolicy` , `sentiment`)	`server-go/internal/api/assets/routes.go`
Admin trade routes (cancel)	`server-go/internal/api/admin/trading.go`
Models, statuses, indexes	`server-go/internal/models/binary_trade.go` , `settlement.go` , `domain.go` , `wallet.go`
Redis keys, WS events, channels	`server-go/internal/config/constants.go`
Error codes	`server-go/internal/platform/apperr/errors.go`
Worker boot, leader election	`server-go/internal/workers/workers.go` , `server-go/internal/platform/leader/leader.go`
Reconciliation	`server-go/internal/service/reconciliation/reconciliation.go`

02 — Money: wallets, ledger, deposits, withdrawals, USDT

How the platform stores and moves money: the wallet model, the append-only ledger, manual INR deposits and withdrawals, the USDT wallet through BeeX Payments, reconciliation and the audit trail.

Everything here comes from the code. Paths are relative to the repo root; Go paths are under `server-go/internal/` unless the path starts with `client/` or `admin/`.

> **Read this first: USDT is not wired up.** The USDT service (`service/usdt`, `service/beex`) is complete as a library, but **no package outside those two imports it**. `cmd/server/main.go` mounts no `/wallet/usdt/*`, `/admin/usdt/*` or webhook routes. `workers/workers.go` never calls `usdt.Start()`, and nothing calls `models.SeedUsdtNetworks`. The client (`client/src/services/api.ts`) still calls `/wallet/usdt/...`. Today those calls hit no route. Section 5 describes the service logic as written, so whoever wires it up knows the contract. Reconciliation (section 6) does already run the USDT checks, because it reads the collections directly.

---

1. Money representation

1.1 INR: integer minor units (paise)

The database, the API and websocket payloads store every INR amount as an `int64` count of **paise** (`₹100.00 → 10000`).

Where	Helper	Behaviour
`platform/util/util.go`	`MinorUnitsPerMajor = 100`	
	`ToMinor(major float64) int64`	`int64(floor(major*100 + 0.5))`, which is JS `Math.round`
	`FromMinor(minor) float64`	`minor / 100`
	`FormatMinor(minor) string`	en-IN grouping, `₹1,23,456.78`
	`CalculateProfit(stake, pct)`	`round_half_up(stake * pct / 100)` (pct is `82` for 82 %)
	`CalculateWinningCredit` / `CalculateLiability`	`stake + CalculateProfit(...)` (exposure and liability only, never credited as such)
`client/src/utils/format.ts` (same in `admin/src/utils/format.ts`)	`MINOR = 100`, `toMinor = Math.round(major*100)`, `fromMinor = minor/100`, `formatMoney`, `formatSignedMoney`, `formatMoneyCompact`	`Intl` en-IN currency formatting

The API rejects non-integer amounts. For example, `minorAmount` in `api/transfers/multipart.go` requires an integer, `> 0` and `<= 1_000_000_000_000`.

1.2 USDT: integer micro-USDT

`1 USDT = 1_000_000 micro`, 6 decimals (`models.USDTDecimals = 6`). No float ever touches a USDT amount. Parsing and printing are done with string and integer operations.

Where	Helper	Behaviour
`platform/micro/micro.go`	`Scale = 1_000_000`, `Decimals = 6`	
	`ParseStrict(s)`	Customer input. Unsigned decimal, at most 6 decimals, no sign, exponent or spaces. Errors: `ErrSyntax`, `ErrPrecision`, `ErrNegative`, `ErrRange` (more than 18 digits).
	`ParseLenient(s)`	Provider amounts. Trims spaces, accepts a leading `+` and an exponent, and **floors** anything past the 6th decimal. Returns `exact=false` when something was dropped. A negative amount is an error.
	`Format(v)` / `FormatFixed(v)`	Shortest exact decimal (`100500000 → "100.5"`) or exactly 6 decimals
	`PercentFloor(amount, "0.5")`	`floor(amount * percent / 100)` with `big.Int`
`service/beex/types.go`	`Amount.Micro()` → `ParseLenient`; `AmountOf(v)` → `Format`	BeeX amounts are decimal strings or JSON numbers
`client/src/utils/usdt.ts`	`USDT_DECIMALS = 6`, `MICRO_PER_USDT = 1_000_000`, `parseUsdt`, `formatUsdt`, `toUsdtInput`	Regex `^(?:(\d+)(?:\.(\d{0,6}))?|\.(\d{1,6}))$`. Refuses `1,000`, exponents and signs. Returns `null` past 16 digits or past `Number.isSafeInteger`.
`admin/src/utils/usdt.ts`	Its own integer-digit formatter (also takes `bigint` and strings)	Differs from the client file

---

2. Wallets

2.1 Accounts and documents

Collection `wallets` (`models/wallet.go`). Unique index `{userId, accountType, currency}`.

accountType	Currency	Purpose	Created by
`LIVE`	`INR`	Real trading money, manual deposits and withdrawals, the house book	`wallets.GetOrCreateWallet` on register and login (`service/auth/auth.go`), and on `GET /wallet`
`DEMO`	`INR`	Practice money, never in the house book	`wallets.EnsureDemoWallet`, lazily
`FUNDING`	`USDT`	USDT wallet (BeeX). Real money, but **not trading money**.	`usdt.fundingWallet`
`LIVE`	`USDT`	Target of "transfer to trading". Not in the trading account list.	`usdt.TransferToTrading`

- `AccountTypeValues = [LIVE, DEMO]`. FUNDING can never be requested where an API takes `accountType`.
- `NormAccountType`: a missing or unknown value means `LIVE`. The startup migration `models/migrate.go` back-fills `accountType: "LIVE"` and drops the legacy `{userId, currency}` unique index.

- `NotDemo()` is `{$nin: [DEMO, FUNDING]}`. Every house-book aggregate uses it, and it matches documents with no `accountType`. `INRCurrency()` is `{$in: ["INR", null]}`.

Fields: `balance` (total owned, **including** locked), `lockedBalance` (reserved by open trades or pending withdrawals), `version` (incremented on every mutation), timestamps. `availableBalance` is **always derived and never stored**:

```
availableBalance = balance - lockedBalance
```

`WalletView` (`models.ToWalletView`) returns all three.

2.2 The lock model

`models/wallet.go` and `service/wallets/wallet.go` define these movements. `P = CalculateProfit(stake, payoutPercent)` is fixed when the trade opens (`service/binary/binary.go`).

Event	balance Δ	locked Δ	Guard	Ledger type / movement	Idempotency key
Trade open (`LockTradeFunds`)	0	+stake	available ≥ stake	`TRADE_LOCK` / `LOCK`	`trade:lock:<tradeId>`
Lose (`SettleTradeLoss`)	−stake	−stake	locked ≥ stake, balance ≥ stake	`TRADE_LOSS` / `DEBIT`	`trade:settle:<tradeId>`
Win (`SettleTradeWin`)	**+P**	−stake	locked ≥ stake	`TRADE_WIN` / `CREDIT` (amount = P, metadata `{stake, profit}`)	`trade:settle:<tradeId>`
Draw (`ReleaseTradeFunds(DRAW)`)	0	−stake	locked ≥ stake	`TRADE_REFUND` / `UNLOCK`	`trade:settle:<tradeId>`
Cancel or orphan (`ReleaseTradeFunds(CANCELLED)`)	0	−stake	locked ≥ stake	`TRADE_CANCEL` / `UNLOCK`	`trade:settle:<tradeId>`
Early close at value V (`SettleTradeEarlyClose`)	V − stake	−stake	locked ≥ stake	`TRADE_EARLY_CLOSE` / `CREDIT` if V ≥ stake else `DEBIT`, amount `abs(V − stake)`	`trade:settle:<tradeId>`

Every settlement path shares the key `trade:settle:<id>`, so a trade settles **at most once**, whether through expiry, early close, cancel or the orphan sweep.

On a win the stake was never removed from `balance`, so the platform credits only the profit. The root README's "Spec deviations" section explains this.

Non-trade movements (`AdjustmentInput`):

Function	balance Δ	locked Δ	Guard	Movement
`CreditWallet`	+a	0	`a > 0`, otherwise 400 `INVALID_AMOUNT`	`CREDIT`
`DebitWallet`	−a	0	`a > 0`; available ≥ a	`DEBIT`

Function	balance Δ	locked Δ	Guard	Movement
`LockWithdrawalFunds`	0	+a	available ≥ a	`LOCK`
`UnlockWithdrawalFunds`	0	−a	locked ≥ a	`UNLOCK`
`SettleWithdrawal`	−a	−a	locked ≥ a	`DEBIT`
`TransferBetweenAccounts`	−a from, +a to	0	from: available ≥ a	`DEBIT` + `CREDIT`

2.3 DEMO account

`service/wallets/demo.go`:

- **Opening balance:** `models.DemoOpeningBalance = 1_000_000` paise, which is ₹10,000.00.
- **Lazy creation:** `EnsureDemoWallet` upserts the empty DEMO wallet, then looks for the ledger key `demo:open:<userId>`. If the key is missing, it credits `ADJUSTMENT_CREDIT` with reference `DemoAccount/demo-open`. Concurrent first requests collide on the key, so the wallet is funded once. The following call it: `GET /wallet?accountType=DEMO`, `GET /wallet/accounts` (returns `[LIVE, DEMO]`), a trader's first DEMO trade, and an admin adjustment of the DEMO account.
- **Reset** (`POST /wallet/demo/reset`, `ResetDemoWallet`):
- It is refused with **409** `CONFLICT` **"Close your open demo trades first"** when the user has DEMO trades in `OPEN` or `SETTLING`, or when the DEMO `lockedBalance > 0`.
- `delta = DemoOpeningBalance − balance`. A delta of `0` is a no-op. A positive delta posts `ADJUSTMENT_CREDIT`, a negative delta posts `ADJUSTMENT_DEBIT`. The reference is `DemoAccount/demo-reset`.
- The key is `demo:reset:<userId>:<unixMs>:<uuid>`. Every reset is a new operation, so a replay can never undo a later reset.
- The route has two rate limits: `wallet` (120/min) and `wallet:transfer` (10/min per user). It is audited as `wallet.demo_reset`.
- DEMO rows use the same guarded, ledgered path as real money. They are excluded from house-book totals, P&L and `platform_result_consistency`.

2.4 The ledger: `wallet_transactions`

`models/wallet_transaction.go`. There is **one row per balance change**, and rows are never updated or deleted.

Field	Notes
`walletId`, `userId`, `accountType`	A missing `accountType` means LIVE
`type`	`DEPOSIT`, `WITHDRAWAL`, `TRADE_LOCK`, `TRADE_LOSS`, `TRADE_WIN`, `TRADE_REFUND`, `TRADE_EARLY_CLOSE`, `TRADE_CANCEL`, `ADJUSTMENT_CREDIT`, `ADJUSTMENT_DEBIT`, `FEE`, `TRANSFER` (`models/domain.go`). `FEE` is declared but never written.
`movement`	`CREDIT`, `DEBIT`, `LOCK`, `UNLOCK`
`amount`	Always positive. `movement` carries the direction.
`balanceBefore/After`, `lockedBefore/After`	Snapshot taken from the guarded update's pre-image (`ReturnDocument: Before`)

Field	Notes
`referenceType` / `referenceId`	`BinaryTrade`, `Transfer`, `AdminAdjustment`, `DemoAccount`, `UsdtDeposit`, `UsdtWithdrawal`, `UsdtTransfer`
`idempotencyKey`	**Partial unique index** where `idempotencyKey` is a string. Rows without a key are unconstrained. A row with a key can only ever be written once.
`network`, `fee`	Only on USDT rows (fee is in micro-USDT)
`metadata`	A free-form object, `{}` by default

Other indexes: `type`, `referenceId`, `{userId, createdAt}`, `{walletId, createdAt}`, `createdAt`.

A withdrawal therefore posts **two or three rows with the same type** `WITHDRAWAL`. They differ by movement: `LOCK` when requested, then either `DEBIT` when settled or `UNLOCK` when released.

Idempotency keys by operation:

Key	Written by
`trade:lock:<tradeId>` / `trade:settle:<tradeId>`	Trade engine, settlement, early close, orphan sweep
`demo:open:<userId>` / `demo:reset:<userId>:<ms>:<uuid>`	DEMO account
`adjust:ADJ_<random>`	Admin manual adjustment
`deposit:credit:<transferId>`	Manual deposit approval
`withdrawal:lock:` / `withdrawal:settle:` / `withdrawal:unlock:<transferId>`	Manual withdrawal
`usdt:deposit:<depositId>`	USDT deposit credit (automatic or operator)
`usdt:withdrawal:lock:` / `:settle:` / `:unlock:<withdrawalId>`	USDT withdrawal
`usdt:transfer:<userId>:<sha256(clientKey)[:32]>:out` and `:in`	USDT transfer to trading

`GET /wallet/transactions` (`api/wallet/routes.go`):

- `accountType` defaults to LIVE, and LIVE excludes FUNDING. The currency is INR.
- The `type` filter enum leaves out `TRADE_EARLY_CLOSE` and `TRANSFER`.

2.5 Concurrency guarantees

`applyMutation` in `service/wallets/wallet.go` is **the only code that writes** `wallets`. It makes every mutation:

1. **Guarded.** The preconditions sit in the `$expr` of the `findOneAndUpdate` filter, together with the `$inc`. The preconditions are: `balance − locked ≥ RequireAvailable`, `locked ≥ RequireLocked`, and non-negative results for negative deltas. Two concurrent requests cannot both pass. If the filter misses: - no wallet gives 404 `WALLET_NOT_FOUND`; - a locked guard gives 409 `LOCKED_FUNDS_MISMATCH`; - anything else gives 400 `INSUFFICIENT_BALANCE`.
2. **Serialised.** A Redis lock `lock:wallet:<uid>:<currency>` (LIVE) or `lock:wallet:<uid>:<ACCOUNT>:<currency>` is taken with TTL 10 s and a 15 s wait. A timeout returns 503 `SERVICE_UNAVAILABLE` "Your account is busy…". Callers that already hold the lock pass `ExternallyLocked`. `TransferBetweenAccounts` takes both wallet locks in sorted key order, so two opposite transfers cannot deadlock.

3. **Idempotent.** With a key present, the ledger is checked first and a hit returns `Replayed: true` without touching the balance. If two racing callers both miss, the loser gets a duplicate-key error on insert, reads the winner's row and returns it as a replay.
4. **Not cancellable.** It runs with `context.WithoutCancel`, so a client disconnect cannot abandon a half-done movement.
5. **Announced.** On success it emits websocket `wallet.updated` on `user:<id>`.

Transactions. `ENABLE_MONGO_TRANSACTIONS` (default `true`, `config/env.go`) and a startup probe (`platform/mongox/mongo.go`) together decide `transactionsSupported`.

- **With transactions:** the guarded update, the ledger insert and the caller's `InTx` hook commit together. The hook is how the USDT code flips `credited` or the withdrawal status. Read concern is `local`, write concern is `majority`, and transient errors are retried 5 times with backoff `10ms·2^n + jitter`.
- **With `ENABLE_MONGO_TRANSACTIONS=false`** (or when the probe fails): `WithTransaction` runs the function as plain sequential writes. The guard and the unique key still prevent double spends and double credits, but:
- a failure *after* the balance update (ledger insert, `InTx` hook) leaves the balance changed with no ledger row or record;
- `reconciliation` then reports `wallet_ledger_vs_balance` drift;
- `usdt.Start` logs a warning in this mode.

Orphaned stake locks (`service/wallets/orphan_locks.go`). The trade engine locks the stake before it inserts the trade. If the process crashes in between, the money stays locked.

- At worker start (`workers/workers.go`), `ReleaseOrphanedTradeLocks` finds `TRADE_LOCK` rows that meet both conditions:
- older than `orphanGrace = 60s`;
- no `binary_trades` document with that id.
- It processes at most `500` per run and skips any lock that already has a `trade:settle:<id>` row.
- It releases each orphan with `ReleaseTradeFunds(CANCELLED)`, which shares the settle key with settlement.

---

3. Manual INR deposits

`service/transfers/transfer.go`, `api/transfers/routes.go`, `models/transfer.go`. Collection `transfers` with `kind: DEPOSIT`.

3.1 Payment methods

`payment_methods` (`models/payment_method.go`, `service/transfers/payment_method.go`).

- Fields: `key` (trimmed, lower-cased, unique), `label`, `type`, the type's details, `instructions`, `minAmount` and `maxAmount`, `active`, `displayOrder`.
- `type` is `UPI` (`upiId`, `qrFileId`) or `BANK` (`accountName`, `accountNumber`, `ifsc`, `bankName`).
- Defaults: `minAmount = 100_00` (₹100), `maxAmount = 500_000_00` (₹5,00,000), `active = true`.
- Validation on upsert:
- UPI must match `^[a-zA-Z0-9.\-_]{2,64}@[a-zA-Z]{2,32}$`;

- an account number is 6–20 digits;
- IFSC must match `^[A-Z]{4}0[A-Z0-9]{6}$` after upper-casing;
- `min ≤ max`.
- Customers read active methods with `GET /transfers/payment-methods`.
- Admins:
- `GET /admin/payment-methods` needs `deposits:read`;
- `PUT /admin/payment-methods` and `POST /admin/payment-methods/{key}/active` need `settings:write`. Both write `payment_method.updated` to the audit log synchronously, with before and after.

3.2 Submitting a claim: `POST /transfers/deposits`

The route needs an authenticated, active user, with rate limit `wallet:transfer` (10/min). It accepts JSON or `multipart/form-data`. The multipart form has fields `amount`, `paymentMethodKey`, `utr`, `customerNote` and the file field `proof`.

Route validation:

- `amount`: integer paise, `> 0`, `≤ 1e12`;
- `paymentMethodKey`: 2–40 characters;
- `utr`: 6–40 characters;
- `customerNote`: up to 500 characters.

Multipart limits are 1 file, 12 fields, and 4096 bytes per field (longer values are truncated). The body cap is `MAX_UPLOAD_BYTES + 64 KiB`. Breaking a limit returns 413 `PAYLOAD_TOO_LARGE`.

`RequestDeposit` then runs these rules:

1. `NormaliseUtr`: trim, then **upper-case**, then 6–40 characters matching `^[A-Z0-9-]+$`. Otherwise 422 `VALIDATION_ERROR`.
2. **A UTR or a proof file is required.** With neither, the request fails 422 "Provide the UTR … or attach a screenshot".
3. If `paymentMethodKey` is given, it must name an *active* method, and the amount must lie in `[minAmount, maxAmount]`. Otherwise 422. *Edge case: when the key is omitted, no method limits apply. Only the route's `1..1e12` bound does.*
4. **UTR uniqueness.** A friendly pre-check returns 409 `CONFLICT` "That reference number has already been submitted…". The real guarantee is the partial unique index `{utr: 1}` filtered on `kind: "DEPOSIT"` and `utr` of type string. Withdrawals and rows without a UTR are unconstrained. Because the UTR is upper-cased first, changing its case does not get around the index. If the insert hits the index, the stored proof file is deleted and the request fails 409.
5. The proof is stored (section 3.4). The row is inserted with these fields: - `reference = "DEP_" + base64url(9 random bytes)` (unique index); - `status = PENDING`, `currency = INR`; - `method` = the method's label, plus `paymentMethodId`.
6. **Nothing is credited.** The route writes `deposit.created` to the audit log asynchronously, with reference, amount, method, utr and hasProof.

3.3 State machine and approval

```
PENDING --approve--> APPROVED --credit ok--> COMPLETED
PENDING --reject---> REJECTED
```

`TransferStatus` also declares `FAILED` (used by withdrawals).

- **Approve** (`POST /admin/deposits/{id}/approve` , `deposits:approve` , optional `note` up to 500 characters): 1. `claimForReview` atomically moves `{_id, kind, status: PENDING}` to `APPROVED` . A second operator gets 409 `CONFLICT` "Transfer is already APPROVED|COMPLETED|...", and an unknown id gets 404 `NOT_FOUND` . 2. `CreditWallet` on LIVE/INR posts `DEPOSIT` / `CREDIT` with key **`deposit:credit:<transferId>`** . The description is `Deposit <ref> (UTR <utr>)` , and the metadata is `{utr, approvedBy}` . 3. The transfer becomes `COMPLETED` with `reviewedBy` , `reviewedAt` , `reviewNote` and `walletTransactionId` . The audit log gets `deposit.approved` , written synchronously.
- **Reject** (`POST /admin/transfers/{id}/reject` , required `note` of 3–500 characters): `PENDING → REJECTED` atomically. Nothing to undo. Audit action `deposit.rejected` .

Edge case: if step 2 fails, for example because the wallet lock timed out, the transfer **stays** **`APPROVED`** . Approve and reject both require `PENDING` , so no API path retries it. An operator has to fix it in the database.

3.4 Proof uploads

`service/transfers/attachment.go` :

- **Type comes from magic bytes only.** The declared content type and the file extension are ignored. Recognised types:

Type	Signature
JPEG	`FF D8 FF`
PNG	`89 50 4E 47`
WebP	`RIFF....WEBP`
PDF	`%PDF-`

A file under 12 bytes, or of any other type, fails 422 "Upload a JPG, PNG, WebP image or a PDF". An empty file fails 422. - **Size:** at most `MAX_UPLOAD_BYTES` (default `5_242_880` , which is 5 MiB). Larger files fail 413. - **Stored name:** `<uuid>.<ext-from-sniffed-type>` , written with `O_EXCL` under `UPLOAD_DIR` (default `./uploads`). The uploaded name is only displayed, after `SanitiseFilename` : - the basename is taken; - characters outside `[\w.\-]` are removed; - the result is capped at 120 characters; - the fallback is `"attachment"` . - **Path safety:** `ResolveAttachmentPath` refuses any path that escapes the upload root. - **Serving:** `GET /transfers/{id}/proof` streams the file from disk. It is never a static mount. - Allowed readers: the owner, or a holder of `deposits:read` or `withdrawals:read` . Anyone else gets **404 "Transfer not found"**, not 403. - Headers: `Content-Type` from the stored mime type, `Content-Disposition: inline; filename="<originalName>"` and `Cache-Control: private, max-age=60` .

---

4. Manual INR withdrawals

4.1 Request: `POST /transfers/withdrawals`

The route needs an active user, with rate limit `wallet:transfer`. Body: `amount` (integer paise, `1..1e12`), optional `customerNote` (up to 500 characters) and `destination`.

Destination is validated at the route, then again by `ValidateDestination`. Only the fields that belong to the chosen type are kept; the rest are stored as null.

Type	Field	Route rule	Service rule
`UPI`	`upiId`	3–80 characters	Trimmed, must match `^[a-zA-Z0-9.\-_]{2,64}@[a-zA-Z]{2,32}$`
`BANK`	`accountName`	2–120 characters	At least 2 characters
	`accountNumber`	6–20 characters	Must match `^[0-9]{6,20}$`
	`ifsc`	Exactly 11 characters	Upper-cased, must match `^[A-Z]{4}0[A-Z0-9]{6}$`
	`bankName`	Optional	

Any other `type` fails with "Expected 'UPI' | 'BANK'".

`RequestWithdrawal` then:

1. Reads the LIVE wallet. It fails 404 `WALLET_NOT_FOUND` if there is none, and 400 `INSUFFICIENT_BALANCE` if `available < amount`. This check is advisory; the guarded lock in step 3 is the real one.
2. Inserts the transfer with these fields: - `reference = "WDR_…"`, `status = PENDING`; - `method = "UPI"` or `"Bank transfer"`; - `lockedAmount = amount`.
3. **Reserves the amount immediately**: `LockWithdrawalFunds`, posting `WITHDRAWAL` / `LOCK` with key **`withdrawal:lock:<transferId>`**. If the lock fails, for example because a concurrent spend won, the transfer is set to `FAILED` with `reviewNote: "Could not reserve funds"` and `lockedAmount: null`, and the error is returned. - The insert and the lock are **not** one transaction. A crash between them leaves a `PENDING` row with no lock. `locked_vs_open_positions` would then flag it.
4. The audit log gets `withdrawal.created` asynchronously, with reference, amount and destinationType.

4.2 Approval and rejection

```
PENDING --approve(utr)--> APPROVED --settle ok--> COMPLETED
PENDING --reject(note)--> REJECTED  (+ reservation released)
PENDING ...lock failed.... FAILED
```

- **Approve** (`POST /admin/withdrawals/{id}/approve`, `withdrawals:approve`):
- **A payout UTR is required**: `utr` of 6–40 characters, normalised by `NormaliseUtr` (upper-cased). A missing UTR fails 422 "Record the UTR / reference of the payment you made before approving".
- `claimForReview` moves the transfer `PENDING → APPROVED`. Then `SettleWithdrawal` runs: balance −a, locked −a, with key **`withdrawal:settle:<transferId>`**. The metadata is `{utr, approvedBy}`.
- The transfer becomes `COMPLETED` with `utr`, `lockedAmount: null` and `walletTransactionId`. Audit action `withdrawal.approved`, written synchronously.

- Payout UTRs are not unique-indexed: the partial index covers deposits only.
- **Reject** (`POST /admin/transfers/{id}/reject` , `note` of 3–500 characters):
- The transfer moves `PENDING → REJECTED` . When `lockedAmount ≠ 0` , `UnlockWithdrawalFunds` posts `WITHDRAWAL` / `UNLOCK` with key **`withdrawal:unlock:<transferId>`** , then `lockedAmount` is set to null.
- Audit action `withdrawal.rejected` with `reason = note` .
- *Edge case, same as deposits:* if the settle fails after the claim, the withdrawal stays `APPROVED` with its funds still locked, and no API path recovers it.

---

5. USDT wallet via BeeX (service logic, currently unrouted)

The package overview is in `service/usdt/settings.go` . The money is the customer's `FUNDING` / `USDT` wallet, and it moves only through `service/wallets` .

The provider-side records are:

- `usdt_deposits` (one BeeX invoice per deposit);
- `usdt_withdrawals` (one payout per withdrawal);
- `usdt_transfers` ;
- `beex_webhook_events` ;
- `usdt_networks` .

5.1 Configuration and production guards

`config/env.go` :

Variable	Default	Meaning
`USDT_ENABLED`	`false`	Master switch. When off, service calls fail **403** **`USDT_DISABLED`** .
`BEEX_PROVIDER`	`beex`	`beex` or `mock`
`BEEX_API_BASE_URL`	`https://beexpayments.com/api/v1`	Must be http(s), https in production
`BEEX_API_KEY`	—	Must start `ck_live_` or `ck_test_` ; the prefix sets `Mode()`
`BEEX_WEBHOOK_SECRET`	—	HMAC secret
`BEEX_ACCEPT_TEST_MODE`	`false`	Credit test-mode payments outside production
`BEEX_PAY_URL_BASE`	`https://beexpayments.com/pay/`	`paymentUrl = base + invoiceId`
`USDT_SUCCESS_URL` / `USDT_CANCEL_URL`	`CLIENT_URL + /wallet/usdt/success` / `/wallet/usdt`	`?deposit=<id>` is appended to the success URL
`USDT_DEPOSIT_CREDIT_BASIS`	`received`	`received` or `net` (after BeeX's fee)
`USDT_WITHDRAWAL_REVIEW_ABOVE`	`0`	micro-USDT. **0 means every withdrawal waits for an operator.**
`USDT_TRANSFER_TO_TRADING_ENABLED`	`false`	Gates transfer to trading

Variable	Default	Meaning
`USDT_USER_DAILY_WITHDRAWAL_LIMIT`	`0`	micro-USDT, rolling 24 h. 0 means use BeeX `dailyWithdrawalCap`.

`validateUsdt` stops the server from booting in production when any of these hold:

- `BEEX_PROVIDER=mock`. As a second line of defence, `beex.New` also refuses the mock in production.
- `BEEX_ACCEPT_TEST_MODE=true`.
- `USDT_ENABLED=true` together with any of:
- a missing `BEEX_API_KEY`;
- a key that is not `ck_live_`;
- a `BEEX_WEBHOOK_SECRET` shorter than 16 characters;
- a non-https `BEEX_API_BASE_URL`.

Credit-time mode gate (`ModeAccepted`):

- `live` is always accepted.
- `test` is accepted only when `NODE_ENV != production` **and** `BEEX_ACCEPT_TEST_MODE=true`.
- An empty or unknown mode is refused.

None of these variables appear in `.env.example` or `server-go/README.md`.

5.2 Networks

Seeds are in `models.DefaultUsdtNetworks`. Rows are only ever inserted with `$setOnInsert`, so an operator's edits survive restarts.

code	label	chain	minDeposit	Address rule (`service/usdt/chainaddr`)
`USDTTRC20`	TRC20	TRON	`10_000_000` (10 USDT)	34 characters, starts `T`, base58 decodes to 25 bytes: prefix `0x41`, then 20-byte id, then a 4-byte double-SHA256 checksum. The zero id is refused.
`USDTBSC`	BSC	BSC	`10_000_000`	`0x` + 40 hex digits. All-lower or all-upper is accepted. Mixed case must match EIP-55. The zero address is refused.

- `depositEnabled` and `withdrawEnabled` can be toggled per network (`UpdateNetwork`). A paused network fails 400 `INVALID_NETWORK`.
- An unknown code fails 400 `INVALID_NETWORK`.
- A bad address fails 400 `INVALID_ADDRESS`.
- The client's `addressHint` (`client/src/features/usdt/model.ts`) is only a hint; the server's check is authoritative.

5.3 Idempotency per attempt

Every money call (deposit, withdraw, transfer) carries a client `Idempotency-Key`. `createAttemptKey` in `client/src/features/usdt/model.ts`:

- keeps the same key for the same JSON body;
- generates a new key when the body changes;

- calls `reset()` after a success, so the next identical request counts as a new attempt.

On the server:

- `usdt_deposits`, `usdt_withdrawals` and `usdt_transfers` each have a unique index `{userId, idempotencyKey}`.
- Each record stores `requestHash = sha256(join("|", parts))`:
- `deposit|network|amount`;
- `withdrawal|network|address|amount`;
- `transfer|amount`.
- Replaying a key with a different hash fails **422** `IDEMPOTENCY_PAYLOAD_MISMATCH`. Replaying with the same hash returns the existing record.
- When two concurrent inserts race, the duplicate-key loser reads back the winner.

5.4 Deposits (BeeX invoices)

`service/usdt/deposits.go`, `service/usdt/webhook.go`.

Creation (`CreateDeposit`):

1. The service checks that USDT is enabled, that the network exists with deposits enabled, that `amount > 0`, and that `amount ≥ network.minDeposit` (400 `INVALID_AMOUNT`).
2. It inserts a row with `status: created` and `orderId = "dep-<_id>"`. `orderId` is unique, and BeeX returns the same invoice for the same orderId.
3. `ensureInvoice` calls `CreateInvoice`. The amount is sent as an exact `json.Number` built by `micro.Format`.
4. On success, `adoptInvoice` stores `invoiceId` (or the BeeX reference as a fallback), `paymentUrl`, `reference`, `mode`, `payAddress` and `expiresAt`. If BeeX's `payAmount` differs from the request, `amountExpected` **is overwritten** to match the invoice.
5. On a **definite** BeeX refusal (a 4xx that is not a credentials or config problem): `status: failed` with a customer-safe `failureReason`.
6. On an **unknown** outcome: the row stays `created`. A replay with the same key calls BeeX again with the same orderId. If the invoice is still missing more than 1 h later and BeeX answers 404, `RefreshDeposit` marks the row `failed`.

Statuses (`models/usdt.go`) and the rank that prevents moving backwards:

Status	Rank
`created`	0
`awaiting_payment`	1
`partially_paid`	2
`confirming`	3
`underpaid`, `expired`, `cancelled`, `failed`	4
`paid`	5
`overpaid`	6

- Open statuses: `created`, `awaiting_payment`, `partially_paid`, `confirming`.
- **Only `paid` and `overpaid` can be credited** (`InvoiceCreditable`).

`applyInvoiceUpdate` is the single path for webhook, refresh, sweep, admin, user and create updates:

- These updates are **ignored** (recorded as ignored, never retried):
- an unknown status;
- a `reference` mismatch;
- a `mode` that differs from the stored mode;
- a malformed amount.
- **Already credited:** money never changes again. The update only records `providerStatus`, a higher `amountReceived`, and an upgrade to a higher creditable status (`paid → overpaid`). When the basis is `received`, `received > amountCredited` logs "received beyond the credited amount needs manual review". Nothing is auto-credited.
- **Stale:** `received < recorded` is ignored. A lower rank is ignored unless more money arrived. Closed-unpaid statuses share rank 4, so a late payment can move one of them on to `paid`.
- **Creditable but missing `amountReceived`:** returns `errNeedAmounts`, and the webhook path then refreshes the full invoice from BeeX.
- **Mode not accepted:** `providerStatus` is recorded, there is no credit, and the event is ignored.

Crediting (`creditDeposit`):

```
basis = USDT_DEPOSIT_CREDIT_BASIS
amount = received                      if basis = received   (overpaid → full received is credited)
amount = net ?? (received − fee)       if basis = net         (neither reported → wait)
amount ≤ 0 → error, retried later
```

- The credit posts `CreditWallet` on FUNDING/USDT as `DEPOSIT` / `CREDIT`, with key `usdt:deposit:<depositId>`, `fee` and `network` on the row, and metadata `{orderId, invoiceReference, status, basis, amountReceived, source}`.
- Inside the same transaction, `credited: false → true` is a conditional update that sets `amountCredited`, `creditedAt`, `creditBasis` and `walletTransactionId`. If it matches nothing, the credit rolls back with "already credited".
- Precision: amounts with more than 6 decimals are floored, and the raw string is kept in `amountReceivedRaw`.

Underpaid and partially paid deposits are never auto-credited. `AdminCreditDeposit` (operator, with a note):

- Allowed for `underpaid`, or for `partially_paid` once `expiresAt` has passed. Otherwise 409 `CONFLICT`.
- Credits `amountReceived`, or the net amount under the `net` basis, using the same key `usdt:deposit:<id>`. It records `manualCredit {by, note, amount, at}`.
- If the deposit was already credited: 409 `TRANSACTION_ALREADY_PROCESSED`.

Refresh and polling:

- `RefreshDeposit` calls `GetInvoice(orderId)`.
- `UserRefreshDeposit` lets a customer refresh **at most once per 30 s**. This is claimed through `lastUserRefreshAt`. Otherwise the stored record is returned.

- The client keeps polling while the deposit is open, or while it is `paid` / `overpaid` but not yet credited (`depositNeedsPolling`).
- `GetWallet` reports `pendingDeposits` : the sum of `amountReceived` on uncredited `partially_paid` or `confirming` deposits. It is not spendable.

5.5 Webhooks

`ReceiveWebhook` :

1. **Verify before parsing.** The header is `X-Signature: t=<unix>,v1=<hex>` and `v1 = hex(HMAC-SHA256(secret, t + "." + rawBody))` (`service/beex/signature.go`). - The comparison is constant-time, and any `v1` in the header may match (key rotation). - The timestamp tolerance is **±5 min**. - Errors: no secret configured gives 503 `SERVICE_UNAVAILABLE` ; a bad, stale or future signature gives 401. The body cap is `MaxWebhookBody = 64 KiB` .
2. **Store under the event identity** (unique `eventKey`): - invoice events: `event|reference|status|amountReceived` ; - payout events: `event|reference|status|txHash` ; - otherwise `other|event|sha256(body)` , or `malformed|sha256(body)` (stored as failed).

A redelivery increments `deliveries` and does nothing else. 3. **Process**, matching the event to a record: - invoice: `orderId` , falling back to `reference` ; - payout: `clientReference` (our reference), falling back to a `providerReference` we already stored.

Results:

Result	When
`processed`	Something changed
`duplicate`	No change
`ignored`	No matching record, or a mismatch
`failed`	An error

BeeX still receives 200 once the event is stored. 4. **Retry** (`RetryWebhookEvents` , a leader loop every 30 s, batch 50): - The initial `nextAttemptAt` is `now + 2 min` , which keeps the loop off an event that is still being processed. - Backoff is `1min << attempts` , capped at 4 h. - Retrying stops after **10 attempts**. - When USDT is disabled, events stay `pending` and are retried in 10 min. - The operator's "process again" (`ReprocessEvent`) forces a rerun and is idempotent.

5.6 Withdrawal quote, limits and fees

`service/usdt/limits.go` , `service/usdt/withdrawals.go` .

BeeX payout limits (`GetPayoutLimits`):

Item	Value
Fields	`minWithdrawal` , `maxWithdrawal` , `dailyWithdrawalCap` , `reviewedAbove` , `fee.fixed` , `fee.percent`
Fresh for	10 min

Item	Value
Shared cache	Redis key `usdt:payout-limits`, kept 24 h
Fetch timeout	10 s, one fetch per process at a time
BeeX unreachable	The stale copy is used and flagged `limitsStale: true`. With no copy at all: 503 `PROVIDER_UNAVAILABLE`.

Formulas, with all amounts in micro-USDT:

```
fee   = feeFixed + floor(amount * feePercent / 100)        (estimate; BeeX's payout fee is authoritative)
net   = amount − fee
userDailyCap = USDT_USER_DAILY_WITHDRAWAL_LIMIT > 0 ? that : dailyWithdrawalCap
used(24h)    = Σ amount of withdrawals created in the last 24h with status ∉ {failed, rejected}
needsReview  = USDT_WITHDRAWAL_REVIEW_ABOVE ≤ 0  ||  amount > USDT_WITHDRAWAL_REVIEW_ABOVE
```

`GetQuote` returns these fields: `amount`, `feeAmount`, `netAmount`, `feeIsEstimate: true`, `min` / `maxWithdrawal`, `dailyCap`, `dailyUsed`, `reviewedAbove`, `needsReview`, `feePercent`, `feeFixed`, `availableBalance`, `providerReviewedAbove` and `limitsStale`.

Request checks, in order (`RequestWithdrawal`). A limit of 0 means no limit.

Check	Error
USDT enabled	403 `USDT_DISABLED`
Network exists and has withdrawals enabled	400 `INVALID_NETWORK`
Address valid for the chain (trimmed)	400 `INVALID_ADDRESS`
`amount > 0`	400 `INVALID_AMOUNT`
Same key replayed	Existing record, or 422 on a hash mismatch
BeeX lists the currency	400 `INVALID_NETWORK`
`amount ≥ minWithdrawal`	400 `MIN_WITHDRAWAL_NOT_MET`
`amount ≤ maxWithdrawal`	400 `MAX_WITHDRAWAL_EXCEEDED`
`fee < amount`	400 `INVALID_AMOUNT`
Under the per-user Redis lock `lock:usdt:withdraw:<uid>` (TTL 20 s, wait 10 s; timeout gives 503): `used + amount ≤ userDailyCap`	400 `DAILY_CAP_EXCEEDED`
Under the same lock: platform-wide `used + amount ≤ dailyWithdrawalCap`	400 `DAILY_CAP_EXCEEDED`
Available balance	400 `INSUFFICIENT_BALANCE` (from the guarded lock)

Lock and record commit together.

- `LockWithdrawalFunds` on FUNDING/USDT posts `WITHDRAWAL` / `LOCK` with key **`usdt:withdrawal:lock:<id>`**, and the `usdt_withdrawals` insert happens inside `InTx`.
- `reference = "withdrawal-<id>"` (unique). This is our payout reference, and it is resent on every attempt.
- The new record starts in one of two states:
- `pending_review`;

- `requested` with `submissionPending: true`, followed by an immediate `SubmitPayout`.

5.7 Withdrawal state machine

```
pending_review --operator approve--> requested --> [awaiting_approval] --> sending --> completed  (settle)
pending_review --operator reject---> rejected                                          (release)
requested/awaiting_approval/sending --BeeX--> failed | rejected                       (release)
requested (never acknowledged, BeeX confirms no payout) --operator reject--> rejected (release)
```

- Open statuses, which hold the gross locked: `pending_review`, `requested`, `awaiting_approval`, `sending`.
- Final statuses: `completed`, `failed`, `rejected`. Nothing changes a final status.
- **Submission** (`SubmitPayout`):
- Redis lock `lock:usdt:payout:<id>`, TTL 90 s.
- The next attempt is scheduled *before* calling BeeX. Backoff is 30 s, 1 m, 2 m, 5 m, 10 m, then 30 m.

- Outcomes:

BeeX result	Action
Answered (a repeat comes back as `created:false`)	Adopt the payout
Definite refusal on attempt > 1	Check `GetPayout(reference)` first, in case an earlier unknown attempt succeeded
Definite refusal, no payout exists	`failed`, funds released
Unknown outcome or config error	Funds stay locked; `RetryPendingSubmissions` (every 30 s, batch 20) resends with the same reference

- **`applyPayoutUpdate`** ignores an update when any of these mismatch:

- `clientReference`;
- `providerReference`;
- **amount**;
- address (EVM addresses compare case-insensitively);
- currency.

It also ignores updates for a withdrawal still in `pending_review`. When submission hits a mismatch, resubmission stops and an operator has to look. - **completed**: `SettleWithdrawal` posts `DEBIT` of the **gross** (balance −gross, locked −gross) with key **`usdt:withdrawal:settle:<id>`**. It records the final `fee`, `net` and `txHash`, and the status flips in the same transaction. - **failed or rejected**: `UnlockWithdrawalFunds` posts the **full gross** back with key **`usdt:withdrawal:unlock:<id>`**. - **BeeX reports `completed` for a withdrawal we already released**: this is logged as `CONFLICT` and ignored. Money left without a debit, so an operator must act. - Intermediate statuses only move forward. A reported fee sets `feeIsEstimate: false`. - **Operator actions:** - `ApproveWithdrawal` takes `pending_review → requested` and then submits. - `RejectWithdrawal` works as shown in the diagram above. - Otherwise the result is 409 `WITHDRAWAL_ALREADY_PROCESSED`. - Approving while USDT is disabled gives 409 `USDT_DISABLED`.

5.8 Background work and sweep

`service/usdt/worker.go`. These loops are leader-elected, but **nothing calls `usdt.Start` today** (see the top of this page).

Loop	Interval	Work
`usdt-webhook-retry`	30 s	`RetryWebhookEvents(50)`
`usdt-payout-submit`	30 s	`RetryPendingSubmissions(20)`
`usdt-reconciliation`	10 min	`Sweep`

`Sweep` refreshes at most 50 records per run, within a 5 min budget:

- uncredited deposits in open statuses that are older than 30 min, with no webhook in the last 30 min and no refresh in the last 10 min;
- withdrawals in `requested`, `awaiting_approval` or `sending` with `submissionPending: false` and `updatedAt` older than 30 min.

5.9 Transfer to trading

`service/usdt/transfer.go`:

- **Gated** twice: `USDT_ENABLED` (otherwise 403 `USDT_DISABLED`), and `USDT_TRANSFER_TO_TRADING_ENABLED` (otherwise **409** `USDT_TRANSFER_DISABLED`).
- Moves `amount` from **FUNDING/USDT to LIVE/USDT**. There is no conversion to INR, and the INR trading engine never sees this wallet.
- `TransferBetweenAccounts` writes two `TRANSFER` rows (`DEBIT` on FUNDING, `CREDIT` on LIVE) in one transaction. Both rows have `referenceType: UsdtTransfer`. The keys are `usdt:transfer:<uid>:<sha256(clientKey)[:32]>` with `:out` and `:in`.
- The `usdt_transfers` row is inserted in `InTx`. Events: `usdt.updated` `{kind, id, status}` and `wallet.updated`.

5.10 Error codes the client shows verbatim

`KNOWN_ERROR_CODES` in `client/src/features/usdt/model.ts`:

`INVALID_NETWORK`, `INVALID_ADDRESS`, `INVALID_AMOUNT`, `INSUFFICIENT_BALANCE`, `MIN_WITHDRAWAL_NOT_MET`, `MAX_WITHDRAWAL_EXCEEDED`, `DAILY_CAP_EXCEEDED`, `USDT_DISABLED`, `USDT_TRANSFER_DISABLED`, `PROVIDER_ERROR`, `PROVIDER_UNAVAILABLE`, `RATE_LIMITED`, `IDEMPOTENCY_PAYLOAD_MISMATCH`.

Any other code is shown as a generic message.

`beex.ToAppError` maps BeeX error codes to these:

BeeX code(s)	API error
`currency_not_enabled`	`INVALID_NETWORK` 400
`below_network_minimum`, `amount_below_minimum`, `fiat_pricing_not_supported`	`INVALID_AMOUNT` 400

BeeX code(s)	API error
`below_minimum`	`MIN_WITHDRAWAL_NOT_MET` 400
`above_maximum`	`MAX_WITHDRAWAL_EXCEEDED` 400
`invalid_address`	`INVALID_ADDRESS` 400
`daily_cap_exceeded`	`DAILY_CAP_EXCEEDED` 400
`rate_limited`	`RATE_LIMITED` 429
Credential codes, `insufficient_balance`, temporary errors	`PROVIDER_UNAVAILABLE` 503
Anything else	`PROVIDER_ERROR` 502

Raw provider text is stored only for admins (`providerError`).

---

6. Reconciliation and audit

6.1 Reconciliation

`service/reconciliation/{reconciliation,usdt,worker}.go` . A leader loop runs every `RECONCILIATION_INTERVAL_MS` (default `300_000` , minimum `10_000`).

- Each run writes a `reconciliation_runs` document with status `OK` , `ALERT` or `FAILED` . A check that errors aborts the run, which becomes `FAILED` .
- On `ALERT` the run writes an audit row `reconciliation.alert` (actor `SYSTEM` , `success: false`) and logs `RECONCILIATION_ALERT` .
- **Nothing here corrects a balance.**

Check	Rule
`wallet_ledger_vs_balance`	For every INR wallet: `Σ(balanceAfter − balanceBefore) == balance` **and** `Σ(lockedAfter − lockedBefore) == lockedBalance`
`locked_vs_open_positions`	For every non-FUNDING INR wallet with `locked > 0` : `locked == Σ stake` of its account's `OPEN` + `SETTLING` trades, plus (LIVE only) `Σ amount` of `PENDING` manual withdrawals. Expected locks with no wallet lock are flagged too.
`settled_trades_vs_settlements`	Count of WON, LOST, DRAW and CLOSED_EARLY trades == count of settlements
`settlements_vs_wallet_transactions`	Count of settlements == count of `TRADE_WIN` , `TRADE_LOSS` , `TRADE_REFUND` and `TRADE_EARLY_CLOSE` rows
`platform_result_consistency`	`Σ platformResult` of non-DEMO trades settled in the window (24 h) == the same sum over settlements
`hedge_order_integrity`	Filled hedge orders have a fill price, a venue id and a notional > 0
`usdt_wallet_ledger_vs_balance`	The ledger-vs-balance check for every USDT wallet (FUNDING and LIVE/USDT)

Check	Rule
`usdt_locked_vs_open_withdrawals`	FUNDING `locked == Σ amount` of open USDT withdrawals
`usdt_paid_deposits_credited`	No `paid` or `overpaid` deposit has `credited ≠ true` (reports up to 20 ids)
`usdt_credited_deposits_vs_ledger`	Each credited deposit has exactly one `DEPOSIT` row whose amount is `amountCredited`, and no uncredited deposit has a row
`usdt_completed_withdrawals_settled`	Each `completed` withdrawal has its `usdt:withdrawal:settle:<id>` row for the gross amount

INR and micro-USDT figures are never summed together.

`locked_vs_open_positions` counts only `PENDING` withdrawals. The stuck `APPROVED` state from section 4.2 therefore shows up as a mismatch.

Admin routes: `GET /admin/reconciliation` and `POST /admin/reconciliation/run` need `reconciliation:read`. `POST /admin/reconciliation/{id}/resolve` needs `reconciliation:resolve`; it records a resolution note and audits `reconciliation.resolved`.

6.2 Audit log

`service/audit/audit.go`, `models/audit_log.go`. The collection is `audit_logs`.

- **Fields:** `actorId`, `actorRole`, `actorEmail`, `action`, `resource`, `resourceId`, `before`, `after`, `ip`, `userAgent`, `requestId`, `success`, `reason`, `createdAt`.
- **Append-only by convention.** The application only ever calls `InsertOne`; there is no update or delete code. There is **no database-level enforcement**: no validator and no role restriction.
- **Write modes:** `Write` is synchronous and used for admin money and config actions. `Log` is fire-and-forget, with a 10 s timeout. A failed write logs `AUDIT WRITE FAILED` at error level.

Money-related actions that are actually written:

Action	When	Mode	Payload
`deposit.created`	Customer submits a claim	async	reference, amount, method, utr, hasProof
`withdrawal.created`	Customer requests a withdrawal	async	reference, amount, destinationType
`deposit.approved` / `withdrawal.approved`	Operator approves	sync	The full transfer view
`deposit.rejected` / `withdrawal.rejected`	Operator rejects	sync	Transfer view, `reason = note`
`wallet.adjusted`	Admin manual credit or debit	sync, before the response	reference, direction, accountType, amount, reason, balanceAfter
`wallet.demo_reset`	Customer resets DEMO	async	New balance
`payment_method.updated`	Admin upsert or toggle	sync	before / after

Action	When	Mode	Payload
`reconciliation.alert` / `reconciliation.resolved`	Reconciliation	async	Alerts / note

`wallet.credit`, `wallet.debit`, `wallet.lock` and `wallet.unlock` are declared but never written. Per-movement history lives in the ledger. No USDT action is audited, because no USDT route exists.

Manual adjustment (`POST /admin/users/{id}/wallet/adjust`, `wallets:adjust`, `api/admin/users.go`):

- Body: `amount` (integer paise, `1..1e12`), `direction` (`CREDIT` or `DEBIT`), `reason` (3–500 characters), optional `accountType` (default LIVE).
- Posts `ADJUSTMENT_CREDIT` or `ADJUSTMENT_DEBIT` with reference `AdminAdjustment/ADJ_…` and key `adjust:ADJ_…`.
- That key is random per request, so **a retried HTTP request creates a second adjustment**.

---

7. Permissions on money actions

`config/permissions.go`. `HasPermission` grants a permission when any of these holds:

- the role is `SUPER_ADMIN`;
- the role's own set contains it;
- the user's explicit extra permissions contain it.

`httpx.RequirePermission(a, b)` requires **all** of the listed permissions.

Action	Route	Permission
List wallets with LIVE INR totals	`GET /admin/wallets`	`wallets:read`
A user's ledger	`GET /admin/users/{id}/transactions`	`wallets:read`
Manual credit or debit (LIVE or DEMO)	`POST /admin/users/{id}/wallet/adjust`	`wallets:adjust`
List deposits	`GET /admin/deposits`	`deposits:read`
Approve a deposit (credits)	`POST /admin/deposits/{id}/approve`	`deposits:approve`
List withdrawals	`GET /admin/withdrawals`	`withdrawals:read`
Approve a withdrawal (settles)	`POST /admin/withdrawals/{id}/approve`	`withdrawals:approve`
Reject a deposit **or** a withdrawal	`POST /admin/transfers/{id}/reject`	**both** `deposits:approve` **and** `withdrawals:approve`
View a payment proof	`GET /transfers/{id}/proof`	Owner, or `deposits:read`, or `withdrawals:read`
List payment methods (admin)	`GET /admin/payment-methods`	`deposits:read`
Change payment methods	`PUT /admin/payment-methods`, `POST …/{key}/active`	`settings:write`
View or run reconciliation	`GET /admin/reconciliation`, `POST …/run`	`reconciliation:read`

Action	Route	Permission
Resolve a reconciliation run	`POST /admin/reconciliation/{id}/resolve`	`reconciliation:resolve`
Read the audit log	`GET /admin/audit-logs`	`audit:read`
Customer deposit, withdrawal, DEMO reset	`/transfers/*`, `/wallet/demo/reset`	Authenticated. Transfer writes also need `RequireActiveUser`.

Default role grants:

- `FINANCE` holds every money permission above except `settings:write`, `reconciliation:resolve` and `audit:read`.
- `SUPPORT` holds only `wallets:read`, `deposits:read` and `withdrawals:read`.
- `ADMIN` holds the read permissions, `audit:read` and `settings:read`. It has no approve, adjust or `settings:write`.
- Only `SUPER_ADMIN` holds `settings:write` and `reconciliation:resolve` by default.

USDT operator actions (approve or reject a withdrawal, manual credit, webhook reprocess, network patch) have **no route and no permission** defined yet.

Where to look

Topic	File(s)
Money helpers (INR)	`server-go/internal/platform/util/util.go`, `client/src/utils/format.ts`, `admin/src/utils/format.ts`
Money helpers (USDT)	`server-go/internal/platform/micro/micro.go`, `client/src/utils/usdt.ts`, `admin/src/utils/usdt.ts`
Wallet and ledger schemas	`server-go/internal/models/wallet.go`, `models/wallet_transaction.go`, `models/domain.go`, `models/migrate.go`
Guarded mutations, lock model, keys	`server-go/internal/service/wallets/wallet.go`
DEMO account	`service/wallets/demo.go`, `api/wallet/routes.go`
Account-to-account transfer	`service/wallets/transfer.go`
Orphaned stake locks	`service/wallets/orphan_locks.go`, `workers/workers.go`
Transactions and the probe	`platform/mongox/mongo.go`, `config/env.go` (`ENABLE_MONGO_TRANSACTIONS`)
Redis locks	`platform/redisx/lock.go`, `config/constants.go`
Manual deposits and withdrawals	`service/transfers/transfer.go`, `models/transfer.go`, `api/transfers/routes.go`, `api/transfers/multipart.go`
Proof storage	`service/transfers/attachment.go`
Payment methods	`service/transfers/payment_method.go`, `models/payment_method.go`, `api/admin/finance.go`
Admin finance and adjustments	`api/admin/finance.go`, `api/admin/users.go`, `service/wallets/admin.go`

Topic	File(s)
USDT config and guards	`config/env.go` (`validateUsdt`), `service/usdt/settings.go`, `service/beex/provider.go`
USDT deposits and webhooks	`service/usdt/deposits.go`, `service/usdt/webhook.go`, `service/usdt/apply.go`, `service/beex/signature.go`
USDT withdrawals and limits	`service/usdt/withdrawals.go`, `service/usdt/limits.go`, `service/usdt/chainaddr/chainaddr.go`
USDT transfer and workers	`service/usdt/transfer.go`, `service/usdt/worker.go`
USDT models	`models/usdt.go`
BeeX client, errors, mock	`service/beex/{client,errors,types,mock}.go`
Client USDT rules	`client/src/features/usdt/model.ts`, `client/src/services/api.ts`
Reconciliation	`service/reconciliation/{reconciliation,usdt,worker}.go`
Audit	`service/audit/audit.go`, `models/audit_log.go`, `api/admin/audit.go`
RBAC	`config/permissions.go`, `platform/httpx/middleware.go`
Error codes	`platform/apperr/errors.go`

Code vs README disagreements

1. **USDT is unreachable.** The memory notes and the client UI describe a working USDT wallet. In `server-go`, no route, worker or seed calls into `service/usdt` or `service/beex`. The root README says nothing about USDT at all. The `/wallet/usdt*` calls in `client/src/services/api.ts` have no server routes.
2. **"The only places that convert."** The root README names `internal/platform/util` and `utils/format.ts` as the only conversion points. Three things contradict this: - USDT has its own converters: `platform/micro`, `client/src/utils/usdt.ts` and `admin/src/utils/usdt.ts`. - Code outside those files converts by hand:
    - `client/src/features/trade/ticketModel.ts` (`amountText`: `minor / 100`);
    - `client/src/pages/Trades.tsx` (CSV export: `/ 100`);
    - `admin/src/pages/Dashboard.tsx` (`Math.round(value * 100)`).
    - The server's `util.ToMinor` and `util.FromMinor` have no callers.
3. **"Mutation blocked at the schema level" for `audit_logs`** (root README). The Go backend has no schema or database guard. Append-only holds only because the code never updates or deletes.
4. **"An operator with `deposits:approve` … approves"** (root README). That is true for approval. Rejection, though, needs **both** `deposits:approve` and `withdrawals:approve`, whatever the transfer's kind. The default `ADMIN` role cannot approve or reject at all.
5. **Test table.** The root README cites Node tests (`wallet.test.ts`, `transfers.test.ts`, …), including the "25 concurrent locks" and "two operators approving" concurrency cases. The Go tests (`service/wallets/*_test.go`, `service/transfers/transfer_test.go`) are unit tests of mutation construction, guards, UTR, destination and file sniffing. The concurrency claims have no Go test behind them. There are no tests for `service/usdt`, `service/beex` or `service/reconciliation`.
6. **Environment docs.** None of the `USDT_*` and `BEEX_*` variables are in `.env.example`, `server-go/README.md` or the root README's environment table.

7. **Client ledger labels.** `LEDGER_LABELS` in `client/src/features/usdt/model.ts` maps `WITHDRAWAL_LOCK`, `WITHDRAWAL_RELEASE`, `WITHDRAWAL_REFUND`, `TRANSFER_TO_TRADING` and `TRANSFER_OUT`. The server never writes those types. It writes `WITHDRAWAL` with movement `LOCK`, `UNLOCK` or `DEBIT`, and `TRANSFER`. Labels fall back to `humanise(type)`, so `TRANSFER` displays as "Transfer".
8. **"No payment provider"** (root README, "What is deliberately not built"). This is accurate for what is reachable, but the BeeX integration code exists in the tree.

03 - Platform logic: risk, market data, auth, realtime

Audience: engineers joining the project. This document describes how the Go backend (`server-go/`) manages the house book, prices, sessions and live updates. Everything here comes from the code. File paths are relative to the repo root. Where the code and the READMEs disagree, the code wins, and the disagreement is listed in section 11.

Conventions used throughout:

- **Money is integer minor units** (paise/cents) everywhere: Mongo, API and websocket payloads. "₹100.00" is `10000` .
- **Times are epoch milliseconds**, and the server stamps them itself (`util.NowMs()`). Client clocks and client prices are never trusted.
- **MongoDB is the record and Redis is a cache.** Exposure counters, the expiry queue, ticks and rate-limit counters live in Redis. Anything financial can be rebuilt from Mongo.
- **DEMO trades never enter the house book.** That means no exposure, no hedging, no house limits and no admin P&L. Only LIVE trades are covered by sections 1 to 3.

---

1. Exposure engine

Code: `server-go/internal/service/exposure/exposure.go` , `sentiment.go` , `server-go/internal/models/exposure.go` .

1.1 Storage

Store	Key / collection	Contents
Redis hash	`exp:asset:<assetId>`	`upStake` , `downStake` , `upProfit` , `downProfit` , `openTrades` , `symbol` , `updatedAt`
Redis set	`exp:assets`	asset ids that have counters
Redis hash	`exp:user:<userId>`	one field per `<assetId>` (open stake on that asset) plus `total`
Mongo	`exposures` (unique `assetId`)	durable snapshot of the derived figures plus `hedgedNotional`

1.2 Derived figures (`DeriveExposure` , pure)

Each raw counter is clamped with `max(0, ·)` first. Then:

```
netExposure          = upStake - downStake        (signed: + means the book leans UP)
grossExposure        = upStake + downStake
upPotentialPayout    = upStake + upProfit         (paid out if every UP trade wins)
downPotentialPayout  = downStake + downProfit
potentialLiability   = max(0, upProfit - downStake, downProfit - upStake)
openTrades           = max(0, openTrades)
```

`potentialLiability` is the platform's worst-case **net** loss. The winning side's profits cost the house only the part that the losing side's forfeited stakes do not cover.

Worked example (the spec example, every trade on an 85% payout):

	UP	DOWN
stake	10,000	4,000
profit (85%)	8,500	3,400

- net = 10,000 - 4,000 = **6,000** (leans UP)
- gross = 10,000 + 4,000 = **14,000**
- upPotentialPayout = 18,500 and downPotentialPayout = 7,400
- potentialLiability = max(0, 8,500 - 4,000, 3,400 - 10,000) = **4,500**. If UP wins, the house pays 8,500 in profit and keeps 4,000 of DOWN stakes.

Net drives hedging, which offsets the direction the book leans. Gross drives the asset and platform ceilings, because it is the total stake at risk.

1.3 Updates (`ApplyExposureDelta`)

One Redis pipeline (not MULTI) does the following: `SADD exp:assets`, `HINCRBY` stake/profit/openTrades by `sign`, `HSET symbol, updatedAt`, and `HINCRBY` the user hash for the asset field and `total`. After the pipeline it re-reads the snapshot and fire-and-forgets `exposure.updated` to `admin:risk`. It also `LPUSH`es the asset id onto `hedge:queue` (in a goroutine), so the hedge worker re-evaluates that asset.

Event	Caller	sign
Trade accepted (LIVE only)	`service/binary/binary.go` `acceptTrade` step 11, after the Mongo insert and `ZADD expiry:queue`	`+1`
Expiry settlement	`service/settlement/settlement.go` after the trade's final `FindOneAndUpdate`	`-1`
Early close ("Sell now")	`service/binary/earlyclose_service.go` after the `CLOSED_EARLY` update	`-1`
Admin cancel	`service/binary/binary.go` (cancel path) after funds are released	`-1`

The delta always uses the **frozen** `stake` and `profit` stored on the trade, so the `+1` and `-1` cancel exactly. The Redis write happens *after* the Mongo write and outside any Mongo transaction. If the process crashes between the two, the counters drift until the next rebuild.

1.4 Rebuild at boot (`RebuildExposureFromDatabase`)

`internal/workers/workers.go` runs the rebuild before any worker starts, on every instance with `WORKERS_ENABLED=true`:

1. It aggregates `binary_trades` with `status ∈ {OPEN, SETTLING}` and a non-DEMO account, grouped by `(assetId, direction)`: Σstake, Σprofit and a count.
2. For **every** asset in the registry it runs `DEL` and then `HSET`, writing zeros for assets with no open trades.
3. For users it deletes every `exp:user:*` key (found with `KEYS`) and rewrites them from a `(userId, assetId)` aggregation.

Edge case: the rebuild is not leader-gated or atomic. Suppose a second worker instance boots while the first is accepting trades. Any increments that land between the aggregation and the rewrite are lost or doubled until the next boot.

The durable `exposures` snapshot is rewritten every risk-worker tick by `PersistAllExposures`. That is an upsert per asset that adds `hedgedNotional`. Failures are logged, never returned.

1.5 Sentiment (customer-facing)

`GetSentiment` exposes only the UP/DOWN **ratio**: `up = round(upStake / (up+down) × 100)` and `down = 100 - up`. When the book is empty both are `null`, never an invented 50/50. The endpoint never returns stake amounts. It is served from `server-go/internal/api/assets/routes.go`.

---

2. Risk engine

Code: `server-go/internal/service/risk/risk.go`, `worker.go`, `server-go/internal/service/riskconfig/riskconfig.go`, `server-go/internal/models/risk_config.go`.

2.1 Risk configuration: versioned and append-only

The `risk_configs` collection has `scope = "GLOBAL"`. Its indexes are unique `(scope, version desc)` and a **partial unique** `scope` where `active: true`, so only one row can be active.

- **Read:** `GetRiskConfig` keeps an in-process cache for **2 s**, uses `singleflight` so concurrent cold reads share one query, and returns a private clone. The row is decoded over `DefaultRiskConfig()`, so a field added to the schema after a row was stored gets its default rather than the Go zero value. Config changes therefore reach other instances within about 2 s, and that includes the kill switch.
- **Bootstrap:** on an empty collection, version 1 is inserted from the defaults below. Only the four feed thresholds are copied from env. If two processes race to insert, the loser reads the winner's row. `cmd/server/main.go` forces this at boot and fails fast on error.
- **Update** (`UpdateRiskConfig`), called from `PUT /admin/settings/risk`, which needs `risk:write` and the AdminWrite rate limit: 1. Clone the active row, apply the patch and run `ValidateConfig`. 2. Set the active row to `active=false`, then insert `version+1` with `updatedBy` and `note`. If the insert fails, the old row is re-activated. 3. Invalidate the cache. The handler then writes an audit row synchronously (`audit.RiskConfigUpdated`, holding the full before/after) before it responds.
- **History:** `ListRiskConfigVersions` returns the newest versions first, 50 by default.
- `ValidateConfig` checks:
- `feedHealthy < feedDegraded < feedStale`
- `minTradeAmount ≤ maxTradeAmount`
- `0 ≤ hedgeThresholdPercent ≤ 100`
- hedge levels sorted by `fromPercent` must not be empty (`from < to`) and must not overlap.

Field defaults (`DefaultRiskConfig`) and the bounds that `server-go/internal/api/admin/core.go` `parseRiskPatch` accepts:

Field	Default	Admin bounds	Used by
`maxAssetExposure`	5,000,000_00	≥0	gross limit (with asset `exposureLimit`)
`maxUserExposure`	500,000_00	≥0	per-user open stake
`maxGrossExposure`	20,000,000_00	≥0	platform-wide gross
`minTradeAmount` / `maxTradeAmount`	10_00 / 100,000_00	≥0	stake bounds
`maxOpenTradesPerUser`	50	≥1	per-account open count
`maxUserAssetConcentrationPercent`	40	1..100	concentration rule
`hedgeThresholdPercent`	30	0..100	MEDIUM band and hedge trigger
`criticalExposurePercent`	90	1..100	CRITICAL band
`hedgeLevels`	see 3.1	≥1 item; name 2..40 chars, from/to ≥0, ratio 0..1	hedge sizing
`minSecondsToExpiryForHedge`	10	≥0	hedge suppression
`volatilityHedgeBoost`	0.1	0..1	hedge ratio add-on
`slippageTolerancePercent`	0.1	0..100 (0 disables)	trade price-moved guard
`earlyCloseEnabled` / `earlyCloseFeePercent` / `minSecondsRemainingForEarlyClose`	true / 5 / 10	- / 0..50 / 0..3600	early close
`drawPolicy`	`REFUND`	REFUND, LOSS or WIN	settlement
`feedHealthyMaxMs` / `feedDegradedMaxMs` / `feedStaleMaxMs` / `feedTickMaxAgeMs`	100 / 300 / 1000 / 5000 (bootstrapped from env)	≥1	feed health (4.4)
`haltTradingOnStaleFeed`	true	bool	feed gate
`haltTradingOnDegradedFeed`	false	bool	feed gate
`tradingHalted`	false	bool	global kill switch
`note`	-	≤500 chars	stored on the version

2.2 Risk scoring (`AssessRisk`, pure)

Inputs: the exposure snapshot, the asset limits, the config, global gross, realised volatility, feed status and the seconds to the nearest open LIVE expiry.

```
assetGrossLimit    = min(asset.exposureLimit, cfg.maxAssetExposure)
grossUtilisation   = gross / assetGrossLimit × 100
netUtilisation     = |net| / asset.netExposureLimit × 100
globalUtilisation  = platformGross / cfg.maxGrossExposure × 100
utilisation        = max(gross, net, global)          (the binding constraint)

score = utilisation
      + feedPenalty  {HEALTHY 0, DEGRADED 8, STALE 20, OFFLINE 35}
      + (vol > 0.0015 ? min(15, vol / 0.0015 × 5) : 0)
      + (secondsToNearestExpiry ≤ 10 and openTrades > 0 ? 5 : 0)
```

A zero denominator gives 0% (`util.SafePercent`). Volatility is the sample standard deviation of per-tick log returns over the last 60 s of tick history. It returns 0 when there are fewer than 3 ticks. `ElevatedVolatility = 0.0015` . Each component adds a human-readable string to `reasons[]` . `score` and `utilisation` are rounded to 2 dp and the hedge ratio to 4 dp, using JS `toFixed` semantics.

Levels (`ClassifyRiskLevel`) come from the config, not from constants:

```
critical = cfg.criticalExposurePercent        (default 90)
high     = critical × 0.7                     (63)
medium   = min(cfg.hedgeThresholdPercent, high)  (30)
score ≥ critical → CRITICAL;  ≥ high → HIGH;  ≥ medium → MEDIUM;  else LOW
```

The result is cached in the Redis hash `risk:asset:<id>` . `GetCachedAssessment` re-runs the assessment whenever that key exists, so it is not really a cache read. `AssessAllAssets` broadcasts the full list as `risk.updated` on `admin:risk` .

Risk worker (leader-only, `RISK_INTERVAL_MS` , default 1000): each tick it runs `AssessAllAssets` , then `PersistAllExposures(hedgedByAsset)` , and logs a warning listing every CRITICAL asset.

2.3 Pre-trade limits (`CheckTradeLimits` , pure; `AssertTradeWithinRiskLimits`)

The checks run inside `acceptTrade` , after the halt, account, asset, expiry, feed, slippage and per-expiry payout stake checks. Every violation is collected. The **first** one becomes the error (HTTP 400) and all of them go in `details.violations` .

#	Rule	Error code	Applies to DEMO
1	`stake < max(asset.minStake, cfg.minTradeAmount)`	`STAKE_BELOW_MIN`	yes
2	`stake > min(asset.maxStake, cfg.maxTradeAmount, user.maxTradeAmount?)`	`STAKE_ABOVE_MAX`	yes
3	open trades on this account ≥ `cfg.maxOpenTradesPerUser` (counted per LIVE/DEMO account)	`USER_LIMIT_EXCEEDED`	yes
4	`userTotal + stake > (user.maxUserExposure ?? cfg.maxUserExposure)`	`USER_LIMIT_EXCEEDED`	no
5	`gross + stake > min(asset.exposureLimit, cfg.maxAssetExposure)`	`EXPOSURE_LIMIT_EXCEEDED`	no
6	`|net ± stake| > asset.netExposureLimit` (+ for UP, - for DOWN)	`EXPOSURE_LIMIT_EXCEEDED`	no
7	`platformGross + stake > cfg.maxGrossExposure`	`RISK_LIMIT_EXCEEDED`	no
8	concentration (below)	`RISK_LIMIT_EXCEEDED`	no

The concentration rule only engages on a material book. It is evaluated only when `projectedGross (gross + stake) > userExposureLimit` . In that case it fails when `(userAssetStake + stake) / projectedGross × 100 > maxUserAssetConcentrationPercent` . Below that size the per-user limit (rule 4) already binds. The guard exists because the first customer on a newly listed asset is unavoidably 100% of the book.

Per-user overrides `maxTradeAmount` and `maxUserExposure` live on the user document (`null` means use the config value). They are set by `PATCH /admin/users/{id}` , which needs `users:write` .

Related gates in the same chain: `TRADING_HALTED` (503, from `cfg.tradingHalted`), `FEED_UNHEALTHY` and `NO_PRICE_AVAILABLE` (503, see 4.5), and `PRICE_MOVED` (409, slippage beyond `slippageTolerancePercent`; the details carry `serverPrice`, `quotedPrice`, `movePercent` and `tolerancePercent`).

---

3. Hedging

Code: `server-go/internal/service/hedging/hedging.go`, `worker.go`, `repository.go`, `providers/{provider,paper,rest}.go`.

3.1 Hedge target (`risk.ComputeHedgeTarget`)

Default `hedgeLevels` (bands are percentages of utilisation):

Name	from	to	hedgeRatio
NO_HEDGE	0	30	0
PARTIAL_HEDGE	30	60	0.4
HIGH_HEDGE	60	80	0.7
MAX_ALLOWED_HEDGE	80	100,000	0.95

`SelectHedgeLevel` picks the band with `from ≤ utilisation < to`, falling back to the highest band. The decision is `NO_HEDGE` in each of these cases:

- `!asset.hedgeEnabled`
- no band was found, or the band's ratio is ≤ 0
- `utilisation < (asset.hedgeThresholdPercent ?? cfg.hedgeThresholdPercent)`
- `secondsToNearestExpiry < cfg.minSecondsToExpiryForHedge`

Otherwise:

```
ratio    = clamp01(band.hedgeRatio × asset.targetHedgeRatio × 2 + (vol > 0.0015 ? cfg.volatilityHedgeBoost : 0))
notional = min(round(|net| × ratio), asset.maxHedgeNotional)
notional < asset.minHedgeNotional → NO_HEDGE
```

The asset default `targetHedgeRatio` is 0.5, so `× 2` makes the default neutral (`models/asset.go`). Other asset defaults are `minHedgeNotional = 1,000_00`, `maxHedgeNotional = 1,000,000_00` and `netExposureLimit = 2,000,000_00`.

Worked example. Utilisation is 60% and is binding, net is +120,000,000, volatility is normal, and the asset uses the defaults. The band is HIGH_HEDGE, so ratio = 0.7 × 0.5 × 2 = 0.7 and target = 84,000,000. The **risk level** is still MEDIUM (60 < 63). The hedge band and the risk level are independent scales.

3.2 Order sizing and placement (`EvaluateAndHedge`)

1. `currentSigned` is the sum of `filledNotional` over orders in `{PENDING, PARTIALLY_FILLED, FILLED}` with `closedAt = null`, counting UP-side orders as + and DOWN-side orders as -.

2. `ComputeHedgeAdjustment`: `targetSigned = sign(net) × target` and `delta = targetSigned - currentSigned`. If `|delta| < asset.minHedgeNotional` nothing is placed. Otherwise the side is BUY when delta > 0 and SELL when it is not. A book leaning UP means the house is short the market, so it hedges long. When net goes back to 0 the target is 0 and the existing hedge is unwound with an opposite order.
3. If there is no last tick, the evaluation is skipped with "no reference price available".
4. The idempotency key is `sha256(assetId:side:notional:targetSigned:currentSigned)`. If an order already has that key, it is returned instead of placing a duplicate.
5. A `PENDING` order is persisted **before** the venue call, so a lost response is still reconcilable. It records a decision snapshot: net/gross at decision, utilisation, risk level, hedge level, target, existing hedge and `riskConfigVersion`. `hedge.created` is emitted. A BUY is stored with `side = UP` and a SELL with `side = DOWN`.
6. `provider.PlaceOrder` runs. On error the order becomes `FAILED` with the error message. On success the status, rounded `filledNotional`, `averageFillPrice` and latencies are stored. `hedge.updated` is emitted either way.

Worker (leader-only, `HEDGE_INTERVAL_MS`, default 1000):

- Each tick drains up to 200 entries of `hedge:queue`, deduplicated, and evaluates those assets.
- Every `round(15000 / interval)` ticks (about 15 s) it does a full pass over active `hedgeEnabled` assets that were not just evaluated. On the same pass it runs `ReconcilePendingHedgeOrders`, which polls the venue for up to 200 PENDING or PARTIALLY_FILLED orders, and `MarkHedgePositions`: `unrealisedPnl = round(filled × (tick - avgFill) / avgFill × (UP ? +1 : -1))`, applied only to FILLED orders.
- An evaluation error is logged and never stops the loop.

Admin cancel: `POST /admin/hedging/orders/{id}/cancel` needs `hedge:write`. FILLED and CANCELLED orders are refused with `CONFLICT`. Otherwise the order is cancelled at the venue if it has a provider id, then set to `CANCELLED` and `closedAt` is stamped.

3.3 Providers (`HEDGE_PROVIDER`)

Provider	`Live()`	Behaviour
`paper` (default)	false	In-process ledger. Fill = `notional × partialFillRatio` (default 1). Fill price = `ref × (1 ± 2 bps)` (+ for BUY). Optional `failureRate` and `latencyMs`. Idempotent on its key. Nothing leaves the process.
`broker`	true	REST: `POST /orders`, `DELETE /orders/{id}` and `GET /orders/{id}`. Statuses FILLED and COMPLETE count as filled.
`lp`	true	REST: `POST /quotes` (key `<idem>:quote`), then `POST /executions` for `min(notional, quote.maxNotional)`. An expired quote is FAILED. Statuses FILLED and EXECUTED count as filled.

Live execution fails closed. `restBase.assertExecutable` runs before every request and returns:

- `LIVE_EXECUTION_DISABLED` (503) unless `HEDGE_LIVE_EXECUTION_ENABLED=true`
- `HEDGE_NOT_CONFIGURED` (503) unless `HEDGE_API_URL`, `HEDGE_API_KEY` and `HEDGE_API_SECRET` are all set.

Requests send `x-api-key` and `idempotency-key` with a 30 s timeout. At first use, `GetHedgeProvider` warns when a live provider is selected but disabled, and when production is running on paper. `GET /ready` and the

boot log report `liveHedging = provider.Live() && HEDGE_LIVE_EXECUTION_ENABLED`.

3.4 Settlement never reads hedges

The hedging engine takes only aggregate per-asset exposure as input. The service import DAG (`server-go/CONVENTIONS.md`) is `settlement -> exposure, marketdata, riskconfig, metrics, wallets, notifications`. It contains no `hedging`, and `server-go/internal/service/settlement/settlement.go` contains no reference to hedge orders. A customer's result therefore depends only on entry price, expiry price, payout terms and draw policy. The guarantee holds by construction, through package dependencies. The Go suite has no test that proves it (see section 11).

---

4. Market data

Code: `server-go/internal/service/marketdata/{marketdata,worker,candles,change24h}.go`, `providers/{provider,factory,simulated,binance,genericws,wsclient}.go`, `server-go/internal/api/marketdata/routes.go`.

4.1 Providers (`MARKET_DATA_PROVIDER`)

Provider	Authoritative	Notes
`simulated` (default)	**false**	Ticks every 50 ms. Mean-reverting log random walk (OU, half-life 4 h) with momentum 0.6. Per-symbol minute volatility and seed prices for btcusdt, ethusdt, xauusd, eurusd, gbpusd and usdjpy.
`binance`	true	`wss://stream.binance.com:9443/stream?streams=<sym>@trade/...`. No key needed. Timestamp is `T`, falling back to `E` and then now. Sequence is the trade id.
`generic-ws`	true	Requires `MARKET_DATA_WS_URL` (validated at boot). Optional `Authorization: Bearer MARKET_DATA_API_KEY`. Sends `{action:"subscribe"/"unsubscribe", symbols}`. Accepts objects or arrays with `symbol|s`, `price|p`, `bid|b`, `ask|a`, `timestamp|t` and `sequence|seq`.

An unknown name falls back to simulated. Websocket providers reconnect with backoff `1000 × 2^min(attempt,5)` ms, capped at 30 s, so 2 s, 4 s, 8 s, 16 s, 30 s. Handshake timeout is 15 s. Connection states are CONNECTING, CONNECTED, RECONNECTING, DISCONNECTED and FAILED. Each one is recorded in memory, mirrored to the `md:provider:status` hash, and emitted as `market.status` on `admin:market`.

Production refuses a non-authoritative feed at two layers:

1. `config.Load` fails when `NODE_ENV=production` and `MARKET_DATA_PROVIDER=simulated`, unless `ALLOW_SIMULATED_FEED_IN_PRODUCTION=true`.
2. `providers.GetMarketDataProvider` refuses any `!Authoritative()` provider in production unless that flag is set, and logs a warning for a non-authoritative feed in any environment.

`MARKET_DATA_API_SECRET` is loaded but not used by any provider.

4.2 Ingestion and normalisation

The market-feed worker is a leader loop: it runs every 5 s with a default lock TTL of 25 s. It owns the provider connection, and on each tick it brings subscriptions into line with the lower-cased `providerSymbol` of the active assets. It reconnects if the provider drops.

Ticks go into an in-process channel with a buffer of 8192. When the buffer is full, ticks are dropped with a warning. `receivedAt` is stamped on arrival. A single goroutine drains the channel in batches of up to 512, which keeps arrival order.

`normaliseTick`:

- Matches the provider symbol to an asset (`FindAssetByProviderSymbol`). Unknown symbols are ignored.
- Discards NaN, ±Inf and prices ≤ 0.
- Sets `timestamp = providerTimestamp` (or `receivedAt` when the provider gives none).
- Sets `feedLatencyMs = max(0, receivedAt - providerTimestamp)`, `processingLatencyMs = processedAt - receivedAt` and `totalLatencyMs = max(0, processedAt - providerTimestamp)`.

`writeTicks` makes **one Redis pipeline per batch**:

- `HSET md:tick:<id>` with the last tick plus the provider name, and a 15 min `PEXPIRE`.
- `ZADD md:hist:<id>` scored by the **provider timestamp**, then `ZREMRANGEBYSCORE` for anything older than `receivedAt - 15 min`, and a 15 min `PEXPIRE`.
- `PUBLISH` of a `market.tick` envelope to `market:<id>` and `admin:market`.
- Latency samples.

Tick history retention is the constant `config.TickHistoryRetentionMs = 15 min`. It is not an env var.

4.3 Expiry price

`GetPriceAtOrBefore` returns the newest history entry with `score ≤ expiry` (source `TICK_AT_OR_BEFORE_EXPIRY`), falling back to the last tick (`LAST_KNOWN_TICK`). The expiry worker waits until `HasTickAtOrAfter(expiry)` is true, or for `ExpiryTickGraceMs = 2000`, before it queues a trade for settlement (`service/expiry/expiry.go`).

4.4 Feed health

`ClassifyFeed` (pure) combines freshness and transport latency, and the worse of the two wins:

```
ageMs = now - lastTick.receivedAt
if no tick                → OFFLINE
if ageMs > tickMaxAgeMs   → OFFLINE              (default 5000)
effective = max(totalLatencyMs, ageMs)
effective ≤ healthyMaxMs  → HEALTHY              (default 100)
effective ≤ degradedMaxMs → DEGRADED             (default 300)
otherwise                 → STALE                (stale band 300..5000 by default)
```

The thresholds are read from the **active risk configuration**, not from env. `FEED_HEALTHY_MAX_MS`, `FEED_DEGRADED_MAX_MS`, `FEED_STALE_MAX_MS` and `FEED_TICK_MAX_AGE_MS` are validated at boot (`healthy < degraded < stale`), but they only seed version 1 of the config. After that, the runtime override is `PUT`

`/admin/settings/risk`. `feedStaleMaxMs` is validated and stored, but `ClassifyFeed` never reads it. Anything above degraded is STALE until the tick max age is exceeded.

The simulated feed ticks every 50 ms because a slower generator would read as DEGRADED against the 100 ms healthy band. The `md:health:<id>` key is declared but unused: health is computed on demand.

4.5 Trading gate (`AssertFeedTradable`)

Feed status	New trades
OFFLINE	always refused
STALE	refused if `haltTradingOnStaleFeed` (default true)
DEGRADED	refused if `haltTradingOnDegradedFeed` (default false)
no last tick	refused

A refused feed status returns `FEED_UNHEALTHY` (503). A missing last tick returns `NO_PRICE_AVAILABLE` (503). When the gate passes, it returns the tick that becomes the trade's entry price. Open trades still settle.

4.6 Candles

- **Persisted:** only 60 s candles go to the Mongo `candles` collection, which is unique on `(assetId, interval, openTime)`. Each tick folds into an in-memory bucket keyed by `floor(timestamp / 60000)`. When the next bucket opens, the previous one is upserted asynchronously. Open buckets are flushed on leadership release and shutdown. Only the leader builds candles.
- **Supported intervals** (s): 5, 15, 30, 60, 300, 900, 1800, 3600, 10800 and 86400.
- **Over 60 s:** reads `limit × (interval/60)` stored minutes (capped at 5000), appends the open in-memory bucket, and folds with `foldCandles`. Open is the first open, close the last close, and high/low the extremes.
- **Under 60 s:** rebuilt from tick history only, so these frames reach back at most 15 minutes.
- `to` **parameter:** limits the series to candles with `openTime ≤ to`.
- **24h change** (`change24h.go`): `(last - firstOpenInLast24h) / firstOpen × 100`, rounded to 2 dp. It is cached for 60 s with a 3 s compute timeout, and any error degrades to `null`.

4.7 Endpoints (`/market-data`, authenticated, `Read` rate limit 600/min)

Route	Returns
`GET /market-data/health`	feed health for every asset
`GET /market-data/{id}/price`	`{tick, feed}`
`GET /market-data/{id}/candles?limit=10..1000 (300)&interval=<supported> (60)&to=`	`[{time,open,high,low,close}]`
`GET /market-data/{id}/ticks?windowMs=1000..900000 (300000)&from=&to=`	raw ticks. `from < to` and `to - from ≤ 15 min`. A window older than retention returns an empty list, not an error.

Admin: `GET /admin/market-data` needs `market-data:read` and returns the provider status, feeds and latency report.

---

5. Authentication and sessions

Code: `server-go/internal/service/auth/auth.go`, `server-go/internal/platform/authtoken/token.go`, `server-go/internal/api/auth/routes.go`, `server-go/internal/platform/httpx/middleware.go`, `server-go/internal/models/refresh_token.go`.

5.1 Endpoints

Route	Rate limit	Notes
`POST /auth/register`	5/min per IP	Validation below. Creates the LIVE wallet and issues a session. Returns 201.
`POST /auth/login`	10/min per IP	
`POST /auth/refresh`	60/min per IP	body `{refreshToken}` (min 10 chars)
`POST /auth/logout`	auth	`{refreshToken?, allSessions?}`
`GET /auth/me`	auth	user plus LIVE wallet
`POST /auth/change-password`	auth	
`POST /admin/login`	10/min per IP	Ordinary login, then a role gate. A non-admin's new refresh token is revoked immediately and the call returns `FORBIDDEN`.

Validation:

- email: valid, ≤254 chars, lower-cased and trimmed.
- username: 3..24 chars, `[A-Za-z0-9_]`.
- password: 8..128 chars with at least one lowercase letter, one uppercase letter and one digit.
- fullName: 2..120 chars.
- country: exactly 2 letters, upper-cased.

Duplicates return `EMAIL_TAKEN` or `USERNAME_TAKEN` (409).

5.2 Passwords and login

- bcrypt with cost `BCRYPT_ROUNDS` (default 12, allowed 8..15). Input is truncated to 72 bytes so hashes stay compatible with bcryptjs.
- An unknown email is compared against a dummy cost-12 hash, so timing does not reveal whether the account exists.
- **Lockout:** each wrong password increments `failedLoginAttempts`. When it reaches **8**, `lockedUntil = now + 15 min` is set and the counter resets.
- The order of checks matters. Credentials are verified *before* the lock is checked, so during a lock:
- a wrong password still returns `INVALID_CREDENTIALS` (401) and keeps counting;
- a correct password returns `ACCOUNT_DISABLED` (403), "Too many failed attempts. Try again after ".

- A non-ACTIVE status returns `ACCOUNT_DISABLED` (403, "Account is suspended/closed").
- A successful login resets the counter and stamps `lastLoginAt` and `lastLoginIp`.

Every login, failed login, register, refresh, logout and password change writes an audit row.

5.3 Tokens

- HS256, issuer `binary-trading`.
- **Access token:** `{sub, role, permissions, sid, typ:"access"}`, signed with `JWT_SECRET`, TTL `JWT_ACCESS_TTL` (default `15m`).
- **Refresh token:** `{sub, sid, fid, typ:"refresh"}`, signed with `JWT_REFRESH_SECRET`, TTL `JWT_REFRESH_TTL` (default `30d`).
- The duration format is `^\d+(ms|s|m|h|d)$`. Expiry is computed in whole seconds, so a sub-second TTL produces an already-expired token.
- The `permissions` claim holds the **effective** set at issue time (role permissions plus the user's extras). `HasPermission` also re-adds the role's own permissions on every check.
- Verify errors: an expired token returns `TOKEN_EXPIRED`. A bad token, wrong issuer or algorithm, or wrong `typ` returns `TOKEN_INVALID`. All are 401.
- Response body: `{accessToken, refreshToken, accessTokenExpiresIn, refreshTokenExpiresIn (seconds), tokenType:"Bearer"}`, plus `sessionId`.

5.4 Refresh rotation and replay detection

Refresh tokens are stored in the `refresh_tokens` collection. Each row holds `tokenHash` (sha256 hex, unique), `familyId`, `sessionId`, `expiresAt` (TTL index, so Mongo deletes expired rows), `revokedAt` and `replacedBy`.

State machine for one refresh row:

```
issued (revokedAt=null) --refresh--> rotated (revokedAt=now, replacedBy=<new sid>)
issued --logout / change-password / admin-gate--> revoked
issued --expiresAt passed--> expired (TTL-deleted later)
```

`Refresh(token)`:

1. Verify the JWT signature.
2. Look the row up by hash. If no row exists although the signature is valid, treat it as **replay**: revoke every row with `familyId = claims.fid` and return `TOKEN_INVALID`.
3. If the row has `revokedAt` set, the token was already rotated or revoked. Revoke the whole family and return `TOKEN_INVALID`, "Refresh token has been revoked". This is the main replay defence: a stolen token used after the real client rotated it kills every session in the family.
4. If `expiresAt` has passed, return `TOKEN_EXPIRED`.
5. If the user is missing, return `UNAUTHENTICATED`. If the user is not ACTIVE, return `ACCOUNT_DISABLED`.
6. Issue a new pair in the **same family**, then mark the presented row rotated.

Logout:

- `allSessions: true` revokes every row for the user.
- Otherwise it revokes the row matching the presented token.

- With neither it does nothing, and the access token stays valid until it expires.

Change password: verifies the current password (`INVALID_CREDENTIALS` , "Current password is incorrect"), rehashes, then **revokes all of the user's refresh tokens**, the current session included.

5.5 Account status enforcement

- `httpx.Authenticate` only verifies the JWT. It does not read the database.
- `httpx.RequireActiveUser` re-reads the user. It returns `ACCOUNT_DISABLED` (403) when the status is not ACTIVE, and `TOKEN_INVALID` , "Session is stale", when the stored role differs from the token's role. It is attached only to `POST /binary/trades` , `POST /binary/trades/{id}/close` and transfer writes.
- `acceptTrade` re-checks the status as well.
- Suspending a user (`PATCH /admin/users/{id}` with `status`) revokes no tokens. Read endpoints and existing websockets keep working until the access token expires (at most 15 min by default). The next refresh then fails with `ACCOUNT_DISABLED` .

5.6 Rate limiting

A fixed-window counter in Redis (`rl:<bucket>:<identity>:<windowIndex>` , `INCR` plus `PEXPIRE`), shared across instances. Responses carry `X-Ratelimit-Limit` , `X-Ratelimit-Remaining` and `X-Ratelimit-Reset` . Over the limit, the response is 429 `RATE_LIMITED` with `Retry-After` and `details.retryAfterMs` . **The limiter fails open** when Redis is unreachable and logs an error.

Bucket	Limit/min	Identity
auth:login	10	IP
auth:register	5	IP
auth:refresh	60	IP
binary:trade	60	user
wallet	120	user
wallet:transfer	10	user
support:chat	60	user
read	600	user, else IP
admin	600	user
admin:write	120	user

The client IP is the first `X-Forwarded-For` entry when the header is present. It is trusted unconditionally, so without a proxy that overwrites the header, the IP-keyed limits can be bypassed.

5.7 Production secret checks (`config.Load`)

Every problem is collected and the process exits:

- `JWT_SECRET` and `JWT_REFRESH_SECRET` must each be ≥32 chars (in every environment).
- In production:

- the two secrets must differ;
- `JWT_SECRET` must not start with `change-me`;
- the simulated feed is refused (see 4.1).
- `BCRYPT_ROUNDS` must be between 8 and 15.

Note that `JWT_REFRESH_SECRET` is not checked for the `change-me` prefix.

Other HTTP hardening:

- helmet-style security headers, with a CSP in production only;
- a CORS allow-list of `CLIENT_URL` and `ADMIN_URL`, plus any loopback origin outside production;
- a JSON body cap of `MAX_REQUEST_BODY_BYTES` (default 262,144), which returns `PAYLOAD_TOO_LARGE`.

---

6. RBAC

Code: `server-go/internal/config/permissions.go`. Roles are labels. The server checks **permissions**. `SUPER_ADMIN` passes every check. `HasPermission(role, granted, p)` is true when `p` is in the role's built-in set **or** in the user's explicitly granted extras. Admin roles are SUPPORT, FINANCE, RISK_MANAGER, ADMIN and SUPER_ADMIN. USER has no permissions.

Role	Permissions
USER	none
SUPPORT	users:read, trades:read, wallets:read, deposits:read, withdrawals:read, assets:read, market-data:read, support:read, support:write
FINANCE	users:read, wallets:read, wallets:adjust, deposits:read, deposits:approve, withdrawals:read, withdrawals:approve, trades:read, reports:read, reconciliation:read
RISK_MANAGER	exposure:read, risk:read, risk:write, hedge:read, hedge:write, assets:read, payouts:read, trades:read, market-data:read, reports:read
ADMIN	SUPPORT ∪ RISK_MANAGER ∪ {users:write, users:suspend, trades:cancel, assets:write, payouts:write, market-data:write, reports:read, audit:read, settings:read, reconciliation:read}
SUPER_ADMIN	all 31 permissions

What each permission gates (routes under `/admin` also need an admin role and the Admin rate limit):

Permission	Gates
users:read	`GET /admin/users`, `GET /admin/users/{id}`
users:write	`PATCH /admin/users/{id}` (status, overrides, emailVerified). Role and permission changes are additionally limited to SUPER_ADMIN.
trades:read / trades:cancel	`GET /admin/binary/trades[/{id}]` / `POST /admin/binary/trades/{id}/cancel`
assets:read / assets:write	`GET /admin/assets[/{id}]` / `POST`, `PATCH /admin/assets`
payouts:read / payouts:write	`GET /admin/assets/{id}/payouts` / `POST /admin/assets/{id}/payouts`, `DELETE /admin/assets/payouts/{id}`

Permission	Gates
risk:read	`GET /admin/settings/risk`, `GET /admin/risk`, `GET /risk/assessments`, `GET /risk/config`, WS `admin:risk`
risk:write	`PUT /admin/settings/risk`
exposure:read	`GET /admin/exposure`, `GET /risk/exposure[/{id}]`, WS `admin:risk`
hedge:read / hedge:write	`GET /admin/hedging`, `GET /admin/hedging/orders` / `POST /admin/hedging/orders/{id}/cancel`
wallets:read	`GET /admin/wallets`, `GET /admin/users/{id}/transactions`
wallets:adjust	`POST /admin/users/{id}/wallet/adjust`
deposits:read / withdrawals:read	`GET /admin/deposits` / `GET /admin/withdrawals`. Either one makes the caller a proof "reviewer" in `/transfers`. deposits:read also covers `GET /admin/payment-methods`.
deposits:approve / withdrawals:approve	the matching `/approve` route. `POST /admin/transfers/{id}/reject` needs **both**.
market-data:read	`GET /admin/market-data`, WS `admin:market`
reports:read	`GET /admin/reports/pnl`, `GET /admin/reports/counters`
audit:read	`GET /admin/audit-logs`
reconciliation:read	`GET /admin/reconciliation`, `POST /admin/reconciliation/run`
reconciliation:resolve	`POST /admin/reconciliation/{id}/resolve`
settings:write	`PUT /admin/payment-methods`, `POST /admin/payment-methods/{key}/active`
support:read / support:write	admin support desk routes, ws `admin:support`
users:suspend, market-data:write, settings:read, admin:manage	**declared but gate nothing**

Any admin role, with no permission check, can reach `GET /admin/dashboard`, `/dashboard/series`, `/dashboard/assets` and `/metrics/latency`. The dashboard includes platform exposure and liability totals, so a SUPPORT user sees figures that `/risk/exposure` would refuse them. Websocket `admin:ops` is also open to any admin role.

---

7. Realtime (websocket)

Code: `server-go/internal/platform/ws/{hub,events}.go`, `server-go/internal/config/constants.go`, `cmd/server/main.go`.

7.1 Connection

- `GET /ws?token=<access token>` (or `Authorization: Bearer`) is handled beside the router, and only when the request is an upgrade.
- The handshake checks, in order: 1. total connections on this instance ≥ `WS_MAX_CONNECTIONS_TOTAL` (10,000): HTTP 503; 2. missing token: 401; 3. invalid or expired access token: 401; 4. this user's sockets on this instance ≥ `WS_MAX_CONNECTIONS_PER_USER` (5): 429.
- `CheckOrigin` always passes, because the token is the authentication.

- Both limits count **per instance** (in memory). The `ws:connections` Redis key is declared but unused.
- The token is checked only at handshake. The socket outlives token expiry and account suspension.
- Read limit 64 KiB. The server pings every 30 s and closes a socket that has not ponged or sent a frame since the last ping. Write timeout 10 s. Send buffer 256 frames: a client that falls behind is disconnected.
- On connect, the socket is auto-subscribed to `user:<ownId>` and receives `connection.ready` `{socketId, userId, role, serverTime, channels}`.

7.2 Client protocol

Frames are JSON `{type, channels?}`. `channels` is an array of strings, each ≤128 chars, at most 100 per frame.

type	Server reply
`ping`	`connection.pong {serverTime}`
`subscribe`	`subscription.updated {channels, accepted, rejected}`
`unsubscribe`	`subscription.updated {channels}`. The own `user:` channel cannot be removed.
other	`connection.error {code:"UNKNOWN_TYPE"}`
non-JSON	`connection.error {code:"BAD_FRAME"}`

Channel authorisation (`canSubscribe`):

Channel	Who may subscribe
`market:<assetId>`	anyone authenticated. The asset id is not validated.
`user:<id>`	only the socket's own user
`admin:risk`	risk:read or exposure:read
`admin:market`	market-data:read
`admin:support`	support:read
`admin:ops`	any admin role

Server frames: `{event, payload, emittedAt, serverTime}`. Direct replies have no `emittedAt`. Producers call `ws.Emit` (fire-and-forget) or publish a pipelined envelope `{channels, event, payload, emittedAt}` to the Redis channel `pubsub:broadcast`. Every instance consumes it and delivers to its own sockets, so events cross instances.

7.3 Events

Event	Channels	Payload	Producer
`market.tick`	`market:<id>`, `admin:market`	NormalisedTick: assetId, asset, symbol, price, bid?, ask?, timestamp, providerTimestamp, receivedAt, sequence?, processedAt, feedLatencyMs, processingLatencyMs, totalLatencyMs	marketdata `writeTicks`
`market.status`	`admin:market`	`{provider, state, at}`	provider status change

Event	Channels	Payload	Producer
`trade.created`	`user:<id>`, `admin:ops`	BinaryTradeView	`acceptTrade`
`trade.updated`	`user:<id>`, `admin:ops`	BinaryTradeView	admin cancel
`trade.expiring`	`user:<id>`	`{tradeId, symbol, expiryTimestamp}`	expiry worker, when queued for settlement
`trade.settled`	`user:<id>`, `admin:ops`	BinaryTradeView	settlement, early close
`exposure.updated`	`admin:risk`	ExposureSnapshot (fields of 1.2 plus assetId, symbol, updatedAt)	`ApplyExposureDelta`
`risk.updated`	`admin:risk`	RiskAssessment[] (assetId, symbol, level, score, utilisationPercent, net, gross, potentialLiability, reasons, hedgeDecision, targetHedgeRatio, targetHedgeNotional, evaluatedAt)	`AssessAllAssets` (risk worker every tick, and `GET /risk/assessments`)
`hedge.created` / `hedge.updated`	`admin:risk`	HedgeOrderView (id, reference, assetId, symbol, provider, side, requested, filled, remaining notional, averageFillPrice, requestPrice, status, providerOrderId, riskLevel, hedgeLevel, targetHedgeNotional, realised/unrealised PnL, error, requestLatencyMs, timestamps)	hedging
`wallet.updated`	`user:<id>`	WalletView	wallets (mutations, transfers)
`notification.created`	`user:<id>`	`{id, type, title, body, data, read, createdAt}`	notifications
`usdt.updated`	`user:<id>`	`{kind, id, status}`	usdt (not wired, see section 8)
`support.message`	`user:<id>`, `admin:support`	message view	support
`support.conversation`	`user:<id>` or `admin:support`	conversation view (separate user and desk shapes)	support
`support.typing`	`admin:support` or `user:<id>`	`{conversationId, side, name?}`	support
`connection.ready` / `connection.pong` / `connection.error` / `subscription.updated`	the socket only	see 7.1 and 7.2	hub

---

8. Workers and leader election

Code: `server-go/internal/platform/leader/leader.go`, `server-go/internal/platform/redisx/lock.go`, `internal/workers/workers.go`.

Leader election. `leader.Loop` tries `SET lock:worker:<name> <uuid> NX PX ttl` on every tick, and the leader renews it with a compare-and-`PEXPIRE` Lua script. The default TTL is `max(5 s, 5 × interval)`. A failed renew triggers `OnRelease` and leadership is lost. If the leader dies, another instance takes over within one TTL. Ticks never overlap. A panic is recovered and logged. Stopping runs `OnRelease` and releases the lock with a compare-and-delete. If Redis errors during a renew, the tick is skipped without releasing, so a partitioned former leader can briefly overlap a new one.

Boot sequence (`cmd/server/main.go`):

1. Load config.
2. Connect to Mongo, run `models.Migrate`, ensure indexes.
3. Connect to Redis.
4. Load (or bootstrap) the risk config.
5. Resolve the market provider. This fails in production with a non-authoritative feed.
6. Start HTTP, then the ws hub.
7. `workers.Start`: rebuild exposure, rebuild the expiry queue from OPEN/SETTLING trades, release orphaned trade locks, then start the workers.

Shutdown (SIGINT/SIGTERM, 15 s hard limit): workers stop in reverse order and concurrently, then the ws hub, HTTP, Redis and Mongo.

Worker	Mode	Interval / lock TTL	Env
market-feed	leader	5 s / 25 s	-
expiry	leader	`EXPIRY_TICK_MS` (100, min 10) / 15 s. A DB sweep runs about every 30 s.	`EXPIRY_TICK_MS`
settlement	**all instances** (BRPOP on `settlement:queue`, dedicated connection, then drains up to `SETTLEMENT_BATCH_SIZE`)	blocking, 1 s timeout	`SETTLEMENT_BATCH_SIZE` (200)
risk	leader	`RISK_INTERVAL_MS` (1000, min 100) / max(5 s, 5×)	`RISK_INTERVAL_MS`
hedge	leader	`HEDGE_INTERVAL_MS` (1000, min 100), full pass about every 15 s	`HEDGE_INTERVAL_MS`
reconciliation	leader	`RECONCILIATION_INTERVAL_MS` (300,000, min 10,000) / max(60 s, interval)	`RECONCILIATION_INTERVAL_MS`

Settlement can run on every instance because it is idempotent: an atomic `OPEN→SETTLING` claim, a unique `settlements.tradeId` and a keyed wallet ledger. A trade skipped for lack of a price is re-pushed onto the queue.

`WORKERS_ENABLED=false` (default true) makes an instance API-only. It skips the boot rebuilds and starts no workers. The websocket hub still runs, because event delivery goes through Redis pub/sub.

`server-go/internal/service/usdt/worker.go` defines three more leader loops (`usdt-webhook-retry` and `usdt-payout-submit` every 30 s, `usdt-reconciliation` every 10 min). Nothing in `workers.Start` or `cmd/server` calls them, and no `/usdt` HTTP routes are mounted.

`GET /ready` reports Mongo (including transaction support), Redis, `marketFeed {provider, state, healthyAssets, totalAssets}`, `workers {enabled, started, marketFeedLeader}` and ws stats. It returns 503 only when Mongo or Redis is down. The provider state comes from in-process memory, so on an instance that is not the feed leader it reads `UNKNOWN`. The `md:provider:status` hash is written but never read back.

---

9. Admin analytics and platform P&L

Code: `server-go/internal/service/admin/admin.go`. Every figure excludes DEMO.

- **Dashboard** (`GET /admin/dashboard`):
- users: total and active today (distinct traders since UTC midnight);
- trades: open, today's count, volume and Σ `platformResult`;
- exposure: current gross, net and potential liability;
- hedging: hedged exposure (Σ active filled hedge notional) and unhedged exposure (`max(0, |net| - hedged)`);
- assets: active count;
- latency: feed p50/p95/p99 and trade-acceptance p95, from the Redis samples `metrics:latency:*`, capped at 2000 per metric;
- balances: INR LIVE wallet balance and locked totals;
- transfers: pending deposits and withdrawals.
- **Series** (`/dashboard/series`, default 14 UTC days, every day emitted): trades, volume, platformResult and wins/losses/draws by trade `createdAt`, plus customerPayout from `settlements`. **Asset volume** (`/dashboard/assets`, default 7 days) sorts by volume.
- **P&L** (`GET /admin/reports/pnl`, needs reports:read). Each component comes from its own ledger, so they can be cross-checked:
- `customerPayout`, `customerStakesCollected` and `platformGrossResult` come from `settlements` in the window;
- `customerLiability` = Σ(stake + profit) of open trades now;
- `hedgeRealisedPnl` and `hedgeUnrealisedPnl` come from `hedge_orders` created since `from`;
- `fees` = Σ `FEE` wallet transactions;
- `adjustments` = `ADJUSTMENT_DEBIT - ADJUSTMENT_CREDIT`.
- `netResult = platformGrossResult + hedgeRealisedPnl + fees + adjustments`. Unrealised hedge P&L is reported but not included.

`realisedPnl` is initialised to 0 and nothing in the code ever updates it, so `hedgeRealisedPnl` is currently always 0.

---

10. Where to look

Topic	Files
Env, defaults, boot validation	`server-go/internal/config/env.go`, `.env.example`
Redis keys, ws events, channels, latency metrics	`server-go/internal/config/constants.go`
Roles and permissions	`server-go/internal/config/permissions.go`
Exposure counters, rebuild, sentiment	`server-go/internal/service/exposure/exposure.go`, `sentiment.go`
Risk scoring, pre-trade limits, risk worker	`server-go/internal/service/risk/risk.go`, `worker.go`
Risk config versioning, kill switch	`server-go/internal/service/riskconfig/riskconfig.go`, `server-go/internal/models/risk_config.go`
Risk settings admin API (bounds, audit)	`server-go/internal/api/admin/core.go`
Read-only risk API	`server-go/internal/api/risk/routes.go`
Hedging decision, worker, repository	`server-go/internal/service/hedging/{hedging,worker,repository}.go`
Hedge venues	`server-go/internal/service/hedging/providers/{provider,paper,rest}.go`
Trade acceptance chain	`server-go/internal/service/binary/binary.go` (`acceptTrade`)
Feed providers	`server-go/internal/service/marketdata/providers/*.go`
Tick pipeline, feed health, expiry price, volatility	`server-go/internal/service/marketdata/marketdata.go`
Feed worker	`server-go/internal/service/marketdata/worker.go`
Candles, 24h change	`server-go/internal/service/marketdata/candles.go`, `change24h.go`
Market data API	`server-go/internal/api/marketdata/routes.go`
Auth service, JWT	`server-go/internal/service/auth/auth.go`, `server-go/internal/platform/authtoken/token.go`
Auth API and validation	`server-go/internal/api/auth/routes.go`
Middleware: auth, RBAC, rate limit, CORS	`server-go/internal/platform/httpx/middleware.go`
Error codes	`server-go/internal/platform/apperr/errors.go`
Websocket hub and bus	`server-go/internal/platform/ws/hub.go`, `events.go`
Leader election, locks	`server-go/internal/platform/leader/leader.go`, `server-go/internal/platform/redisx/lock.go`
Worker lifecycle	`server-go/internal/workers/workers.go`, `server-go/cmd/server/main.go`
Expiry and settlement workers	`server-go/internal/service/expiry/`, `server-go/internal/service/settlement/`
Admin analytics and P&L	`server-go/internal/service/admin/admin.go`
Readiness	`server-go/internal/api/health/routes.go`

---

11. Code vs documentation disagreements

1. **Feed thresholds are not live env settings.** The root README says the `FEED_*_MS` variables are "also editable at runtime". In code they only seed version 1 of the risk config. Once that row exists, changing the env has no effect, and the admin console is the only control. `feedStaleMaxMs` is never read by `ClassifyFeed`.
2. **Early close is not a single transaction.** The root README says exposure is released and the trade is dropped from the expiry queue "in the same transaction". The code runs sequential steps: claim, wallet, settlement row, trade update, then the Redis `ZREM` and exposure delta. Safety comes from idempotency keys, not from a transaction.
3. **"There is a test that proves" hedging cannot affect settlement.** That test (`hedging.test.ts`) was a Node suite and was not ported. The Go guarantee is structural (the import DAG), and `hedging_test.go` tests only sizing.
4. **Tick retention name.** The README calls it `TICK_HISTORY_RETENTION_MS`. It is the Go constant `config.TickHistoryRetentionMs` (15 min), not an env var.
5. **Chart timeframes.** The README says "5s to 1h". The server also accepts 3h (10800) and 1d (86400).
6. **"Workers are leader-elected either way"** (`server-go/README.md`). Settlement is not leader-elected, and every worker-enabled instance runs the exposure and expiry-queue rebuild at boot without a lock.
7. **Hedge decisions are not fully replayable.** The package doc says every order records its decision inputs, but `placeHedgeOrder` always stores `volatility: 0` and `secondsToNearestExpiry: null`. `RiskInputs.ExistingHedgeNotional` is gathered but never used by `AssessRisk`.
8. **Hedge realised P&L.** P&L reports include `hedgeRealisedPnl`, but nothing ever writes `realisedPnl`. Cancelling a PARTIALLY_FILLED order stamps `closedAt`, which drops its filled part from the signed hedge position.
9. **Unused declarations.** - Permissions `users:suspend`, `market-data:write`, `settings:read` and `admin:manage` gate nothing. Suspension is controlled by `users:write`. - Redis keys `md:health:*`, `md:subscribed`, `ws:connections`, `cfg:risk` (risk config is cached in process instead) and `expiry:claim:*` are never used. - `MARKET_DATA_API_SECRET` is unused.
10. **Candle model comment.** `models/candle.go` says candles are what "the volatility estimate reads from", with intervals like "1 or 60". Volatility actually reads Redis tick history, and only 60 s candles are persisted.
11. **USDT is not wired.** The env, the `usdt.updated` event and the service exist, but `cmd/server` mounts no USDT routes and `workers.Start` does not start the USDT loops.

04 - Client-side logic: web (`client/src`) and mobile (`mobile/src`)

This document describes the logic that runs in the two clients of the platform:

- **Web**: `client/src`, React + Vite + Tailwind, chart on `lightweight-charts`.
- **Mobile**: `mobile/src`, Expo SDK 54 / React Native 0.81 / expo-router 6 / NativeWind 4, chart on Skia + Reanimated. It is a feature-for-feature port of the web client; the port contract lives in `mobile/PORTING.md` and the run/build notes in `mobile/README.md`.

Both clients talk to the same Go server (`server-go/`). Most non-UI modules are **byte-identical copies** in both trees (see the table in section 10). When this document cites `client/src/...` without saying otherwise, the same file exists at the same relative path under `mobile/src/`.

> **The governing rule.** The client never decides a price, a time or a payout. Prices come from the server's ticks and candles, expiries from `entryTimestamp` / `expiryTimestamp` on the trade row, payouts from `asset.payouts` and the trade's own `profit` / `potentialReturn` / `settlementAmount`. Everything the client "computes" is either a validation that mirrors a server rule (so the trader gets a message before a round trip), a projection that is explicitly labelled as one ("If it ends now"), or a drawing aid (indicators, drawings) that is never sent anywhere. The only price the client ever sends is `quotedPrice`, and it is a slippage guard, not a fill price.

---

1. Data layer

1.1 HTTP client - `services/http.ts`

An axios instance with `timeout: 15_000` and JSON content type.

- **Base URL**: web uses `import.meta.env.VITE_API_URL ?? '/api'` (Vite proxy); mobile uses `API_URL` from `mobile/src/config.ts` (absolute, see 1.3).
- **Request interceptor**: attaches `authorization: Bearer <accessToken>` from `loadTokens()`.
- **Success envelope**: the server answers `{ success: true, data }`. The helpers `apiGet`, `apiPost(url, body, headers?)`, `apiPatch`, `apiPut`, `postForm` all return `response.data.data`, so callers deal in plain data.
- **Error envelope -> `ApiRequestError`**: `{ success: false, error: { code, message, details } }` becomes `new ApiRequestError(code, message, status, details)`. Anything without that envelope becomes `ApiRequestError('NETWORK_ERROR', ...)`; axios' `"Network Error"` is reworded to `"Cannot reach the trading server"`. `status` is `0` when there was no HTTP response.

Single-flight token refresh on 401. The response interceptor treats a response as an auth failure when `status === 401 && error.code !== 'INVALID_CREDENTIALS'` (a wrong password must not trigger a refresh). If the original request has not been retried (`_retried` flag) and is not itself `/auth/refresh`:

```
original._retried = true
refreshing = refreshing ?? refreshAccessToken().finally(() => refreshing = null)
token = await refreshing          // every concurrent 401 awaits the same promise
token ? retry original with new Bearer : onUnauthorised()
```

`refreshAccessToken()` posts `{ refreshToken }` to `baseURL + '/auth/refresh'` with a **bare axios** call (so it cannot recurse into the interceptor), saves the new pair on success and calls `clearTokens()` on failure. `setUnauthorisedHandler()` is wired at app start (web `App.tsx`, mobile `app/_layout.tsx`) to drop the session: `useAuthStore.setState({ status: 'anonymous', user: null, wallet: null })`, which the route guards turn into a redirect.

Multipart (`postForm`): web deliberately sets `content-type: undefined` so the browser writes the boundary; mobile sets `multipart/form-data` and `transformRequest: (d) => d`, because React Native's networking layer writes the boundary for a `FormData` body. Mobile additionally exports `authenticatedSource(path)` -> `{ uri, headers: { Authorization } }` for authenticated images (deposit screenshots in `expo-image`).

1.2 Token storage

	Web	Mobile
Store	`localStorage['bt.tokens']` (JSON `{accessToken, refreshToken}`)	`expo-secure-store` key `bt.tokens` (device keychain)
Read	synchronous, every call	from an in-memory mirror `cached`
Write	synchronous; ignored on failure (private browsing)	mirror updated synchronously, `setItemAsync` fire-and-forget
Init	none	`initTokens()` must run once before `hydrate()`

On mobile, the request interceptor and the websocket need tokens synchronously, so `initTokens()` reads SecureStore once into `cached` (guarded by an `initialised` flag) and `loadTokens()` returns the mirror. `mobile/src/app/_layout.tsx` runs `initTokens().then(hydrate).finally(setBooted)` and keeps the splash screen up (`SplashScreen.preventAutoHideAsync`) until `booted`, so a signed-in trader never sees the welcome screen flash.

1.3 Mobile API URL and Render cold start

`mobile/src/config.ts`:

- `API_URL = trimSlash(EXPO_PUBLIC_API_URL || 'https://binary-trading-api-ih4v.onrender.com')`.
- `WS_URL = trimSlash(EXPO_PUBLIC_WS_URL || API_URL.replace(/^http/, 'ws') + '/ws')`.
- `APP_SCHEME = 'binarytrading'` (deep links, matches `app.json`).

The free Render instance sleeps when idle (README: first request 30-60 s). Two mitigations:

1. **Wake-up ping**: at module load of `app/_layout.tsx`, `fetch(API_URL + '/health').catch(() => undefined)`. Nothing awaits it; it just makes the server likely awake by the time the user has typed a password.

2. **Hydrate retry that keeps the session** (`mobile/src/stores/authStore.ts`): `hydrate()` tries `api.auth.me()` up to **2 times**, waiting **1500 ms** after the first failure. A `401` / `403` clears the tokens (the session was refused). Any other failure (network, 5xx, timeout) leaves the tokens in SecureStore - "unreachable is not refused" - so the next launch can restore the session; the in-memory status still ends `anonymous` for this launch. The web `hydrate()` has no retry and clears tokens on **any** failure.

1.4 Auth store and route gating

`stores/authStore.ts` - status machine `idle -> loading -> authenticated | anonymous` . `login` / `register` save tokens then set `user` + `wallet` ; on error they set `error` and rethrow. `logout` posts the refresh token, ignores the server's answer, then always clears locally.

- Web: `ProtectedRoute` shows a spinner for `idle|loading` , redirects to `/login` with `state.from = pathname + search` otherwise; `PublicOnlyRoute` sends an authenticated session to `/home` . Legacy paths redirect (`/dashboard -> /home` , `/assets -> /markets` , `/profile -> /account` , `/support -> /account/support`).
- Mobile: `Stack.Protected guard={authenticated}` for `(app)` and `onboarding` , `guard={!authenticated}` for `welcome|login|register` ; the router moves itself when the status flips. `app/index.tsx` redirects to `/home` or `/welcome` . After registration `continueAfterRegistration()` (`features/auth/afterRegister.ts`) replaces to `/onboarding` on the next tick unless the device's onboarding flag is set.

1.5 API map and React Query

`services/api.ts` is the single list of endpoints (`api.auth` , `profile` , `favourites` , `assets` , `market` , `trades` , `wallet` , `transfers` , `usdt` , `notifications` , `support`). Notable shapes:

- `trades.create(input, idempotencyKey)` sends header `idempotency-key` ; `input.quotedPrice` is the slippage guard; `accountType` defaults to LIVE server-side.
- `market.candles(assetId, limit = 300, interval = 60, to?)` ; `market.ticks(assetId, {from, to, windowMs})` - the server keeps only 15 minutes of raw ticks.
- USDT money calls take an `Idempotency-Key` ; amounts are integer micro-USDT.
- Mobile `transfers.deposit` takes `proof: UploadFile { uri, name, type }` instead of a `File` .

QueryClient defaults (identical in `client/src/main.tsx` and `mobile/src/app/_layout.tsx`): `refetchOnWindowFocus: false` (live data comes over the socket), `staleTime: 5_000` , and `retry: (n, err) => 4xx ? false : n < 2` .

Query keys in use (the invalidation targets matter, because the realtime bridge hits them):

Key	Used for
`['assets']`	asset list (also read by `useDrawPolicy` , `staleTime 60_000`)
`['wallet', 'accounts']` (`ACCOUNTS_KEY`)	both INR balances; any `['wallet']` invalidation reaches it
`['wallet', 'transactions', {...}]`	ledger pages
`['trades', 'open']`	open trades (nav count + trade screen)

Key	Used for
`['trades', filters, page]`, `['trades', 'summary', filters]`, `['trades', 'recent-settled', account]`	history
`['trade', id]`, `['trade', id, 'quote']`	trade detail, early-close quote
`['trade-window', id]`	trade detail chart data
`['candles', assetId, timeframe]`	main chart history
`['market', 'health']`, `['notifications']`, `['transfers']`, `['payment-methods']`, `['profile']`	misc
`['usdt', ...]` (`USDT_KEY`, `features/usdt/queries.ts`)	every USDT query under one root

---

2. Realtime bridge - `hooks/useRealtime.ts` + `websocket/client.ts`

2.1 The socket (`websocket/client.ts`)

A singleton `TradingSocket`. URL: web `VITE_WS_URL ?? (wss|ws)://<host>/ws`, mobile `WS_URL`; the access token goes in the query string (`?token=`). `connect()` is a no-op without a token or when a socket already exists.

- **State**: `connecting | open | closed`, derived from `readyState`; `onStateChange` subscribers. `features/trade/useConnectionState.ts` reads it through `useSyncExternalStore` so every component agrees on the state in the same render (the ticket's banner and its disabled buttons cannot disagree).
- **Heartbeat**: on open, `{type:'ping'}` every **25 s**.
- **Reconnect/backoff**: `onclose` (which always follows `onerror`) tears down and, unless `closedByUser`, schedules `delay = min(1000 * 2^attempts, 15_000)` ms (2 s, 4 s, 8 s, 15 s, ...); `attempts` resets to 0 on a successful open.
- **Resubscribe**: the set of channels survives a drop; on open the whole set is re-sent as one `{type:'subscribe', channels:[...]}`. `subscribe()` only sends channels not already in the set; `unsubscribe()` always sends. The set is **not reference-counted** (see 2.4).
- `disconnect()` sets `closedByUser`, cancels the timer, clears the channel set; a socket still `CONNECTING` is closed in its `onopen` (avoids a StrictMode console warning on web).
- Frames are `{event, payload}`; handlers are per event plus a `'*'` wildcard.
- Channels: `market:<assetId>`, `user:<userId>`.

2.2 What each event updates

`useRealtime()` is mounted once in the signed-in shell (web `components/Layout.tsx`, mobile `app/(app)/_layout.tsx`). While `status === 'authenticated' && userId`, it connects, subscribes `user:<id>`, loads favourites from the server, and pre-loads the support thread (best effort).

Event	Effect
`market.tick`	`useMarketStore.applyTick(tick)` (latest tick + previous price per asset)

Event	Effect
`wallet.updated`	invalidate `USDT_KEY`; if `currency === 'USDT'` **stop** (never file a USDT balance in an INR slot); else default missing `accountType` to `LIVE`, `useAccountStore.upsertWallet`, and patch `ACCOUNTS_KEY` in place (`mergeWallet` replaces the entry of the same type)
`usdt.updated`	invalidate `USDT_KEY` (record, lists and balances refetch together)
`trade.created`	`useTradeStore.upsertTrade`, invalidate `['trades']`
`trade.updated`	`useTradeStore.upsertTrade`
`trade.settled`	`useTradeStore.settleTrade`, `announceResult(trade)`, `setQueryData(['trade', id], trade)`, invalidate `['trades']` and `['wallet','transactions']`
`support.message`	`useSupportStore.addMessage`
`support.conversation`	`useSupportStore.setConversation`
`support.typing`	only when `side === 'AGENT'`: `setAgentTyping(name ?? 'Support')`
`notification.created`	invalidate `['notifications']`

On cleanup (sign-out or user change): unsubscribe handlers, `socket.disconnect()`, reset the support, account-balance and legacy wallet stores. The **active account choice survives** sign-out (it is a device preference).

A second effect keeps the legacy single-wallet store (`stores/walletStore.ts`) pointed at the **active** account's wallet, so older readers follow a DEMO/LIVE switch without knowing about accounts.

2.3 Account loading fallback (`useAccounts`)

Query `ACCOUNTS_KEY`, `staleTime 10_000`, enabled when signed in:

1. `api.wallet.accounts()`, filtered to non-USDT wallets, `accountType ?? 'LIVE'`.
2. On error (older server without `/wallet/accounts`): `api.wallet.get('LIVE')` as a one-element list.

When data lands: `setWallets`, and if the persisted `active` account is not among the returned types (e.g. DEMO on an old server), `setActive(types[0])` so balances are never blank.

`features/trade/useTradingWallet.ts` resolves the wallet the ticket validates against: account store -> `['wallet','accounts']` list entry of the active type -> legacy wallet **only if its type matches**. A live balance must never stand in for a demo one.

`features/trade/useActiveTrades.ts` loads `['trades','open']` from any screen (`staleTime 0`, `refetchInterval 30_000` as a safety net) and writes it to the trade store only when `isFetchedAfterMount`, so a previous session's cached list never reaches the nav count. `countOpenFor(trades, account)` counts with `accountOf(trade)` (`accountType ?? 'LIVE'`).

2.4 `useLiveFeeds` subscription rules

`features/trade/useLiveFeeds.ts` subscribes `market:<id>` for every asset with a running trade while a list of running trades is on screen, so a live result is never computed from a frozen price. Because socket subscriptions are a plain set (the chart's unmount drops a channel whoever else wanted it):

- With `keepWorkspace: true` (the trade-screen panel), channels for the workspace's `activeAssetId` and `openAssetIds` are **never released** from here.
- Wanted channels are re-asserted every **5 s** (`REASSERT_MS`) and immediately after the chart changes `activeAssetId` (that effect runs after the chart's unsubscribe in the same commit). Re-asserting is free because `subscribe` only sends missing channels.
- The key is the sorted, de-duplicated asset id list, so re-renders do not churn subscriptions.

`useMarketSubscription(assetId)` (in `useRealtime.ts`) is the chart's own subscribe/unsubscribe.

---

3. Stores (zustand)

All persisted stores use `zustand/middleware` `persist` . Web storage is `localStorage` ; mobile passes `storage: persistStorage` (`mobile/src/stores/storage.ts` , `createJSONStorage(() => AsyncStorage)`). AsyncStorage is asynchronous, so a mobile persisted store renders its defaults for a frame and then rehydrates; nothing persisted gates a server decision.

Store	Persist key	What persists
`accountStore`	`bt.account`	`active` only (balances are session data)
`settingsStore`	`bt.settings` (version 5)	everything
`workspaceStore`	`bt.workspace`	`openAssetIds` , `activeAssetId`
`drawingsStore`	`bt.drawings` (version 1)	`drawings` only
`favouritesStore`	`bt.favourites`	`favourites` , `recent`
`onboardingStore`	`bt.onboarding`	`completed`
`themeStore`	`bt.theme`	web: raw string in localStorage; mobile: `{preference}` via persist
`authStore` , `tradeStore` , `tradeResultStore` , `orderPreviewStore` , `marketStore` , `walletStore` , `supportStore` , `ticketStore`	-	memory only

3.1 `accountStore` - active DEMO/LIVE

`active: AccountType` defaults to `'DEMO'` : "nobody should learn the product with real money". `wallets: Partial<Record<AccountType, Wallet>>` keyed by `accountType ?? 'LIVE'` . Every trade is placed on `active` (read at send time, see 4.6). `features/account/funds.ts` : `openFunds(kind)` switches to LIVE first (with an info toast) because money only moves in and out of LIVE; `useResetDemo()` calls `/wallet/demo/reset` , upserts the wallet and shows the server's 409 message verbatim when demo trades are still open.

3.2 `tradeStore`

`openTrades` holds only statuses `OPEN | SETTLING`, sorted by `expiryTimestamp` ascending; `upsertTrade` removes a trade that left those statuses. `settleTrade` removes it from open and prepends it to `recentlySettled` (de-duplicated by id, capped at **20**). A module-level `useAuthStore.subscribe` resets the store whenever `user.id` changes.

3.3 `tradeResultStore` - dedupe

`results` newest first, capped at `MAX_RESULTS = 10`; `seen: Set<tradeId>`. `announce(trade)` is a no-op for an id already seen, because a settlement can arrive twice (socket `trade.settled` and an early close resolving from its own response). `announceResult()` (`features/trade/announceResult.ts`) is the only entry point. Results do not time out; the newest stays until dismissed, older ones fold as "+N more" (`TradeResultToasts.tsx`).

3.4 `orderPreviewStore` - INTENT / PLACING / REJECTED

The only order state the client owns outright; it has `id: 'preview'`, never a trade id.



Rules: `showIntent` is ignored while a non-INTENT preview exists (a hover cannot unfreeze an in-flight order) and when the draft is unchanged (same asset, direction, stake, expiry). `clearIntent(assetId?)` only clears an INTENT. The REJECTED timer is module-scoped and only clears the preview if it is **still** REJECTED, so a new order placed meanwhile survives.

3.5 `settingsStore` and migrations

Defaults: `defaultStake 500_00` (minor units), `defaultExpirySeconds 60`, `soundEnabled true`, `confirmBeforeTrade true`, `tradeMode 'beginner'`, `chartType 'candles'`, `timeframe 60`, plus `APPEARANCE_DEFAULTS` (`gridOpacity 7` on a 0-10 dial, `timezone SYSTEM`, `performanceMode false`, `autoScroll true`, `showOrderLabels true`, `hideTradeResults false`, colour overrides `null`, `indicators []`, `showExpiryGuide true`). `setGridOpacity` clamps to `[0, GRID_OPACITY_MAX=10]`.

`TIMEFRAMES = [5, 15, 30, 60, 300, 900, 1800, 3600, 10800, 86400]` seconds - "must match the server's set". `timeframeLabel -> 5s/1m/3h/1d`; `timeframeName -> "15 minutes"`, singular for 1.

`migrate(persisted, from)` (current `version: 5`):

1. Always merge `APPEARANCE_DEFAULTS` under the stored state (v1 had no appearance keys).
2. `from < 3` and `gridOpacity > 10`: rescale a stored percentage, `round(g / 100 * 10)`.
3. Any stored `timeframe` not in `TIMEFRAMES` (e.g. the removed 10 s) becomes **15**.

4. `from < 5`: force `confirmBeforeTrade = true` and `tradeMode = 'advanced'` (existing users had every tool on screen).

3.6 `workspaceStore`

`openAssetIds` (tabs, `MAX_TABS = 8`, oldest drops when full), `activeAssetId`, and transient UI (`settingsOpen`, `transferModal`, `assetPickerOpen`, `chartLayout 1|2`). `closeAsset` on the active tab moves to the last remaining tab. `ensureOpen(assetIds)` prunes ids the server no longer lists, seeds the first two assets when nothing valid remains, and fixes an invalid active id.

3.7 `drawingsStore`

`drawings: Record<assetId, Drawing[]>`; `Drawing = { id, kind, points: ChartPoint[] }` with `ChartPoint = { time: epoch ms, price }` - **chart terms, never pixels**, so a drawing stays on its candles through pan, zoom, resize and timeframe changes. Kinds and points needed: `trend 2, ray 2, hline 1, vline 1, rect 2, fib 2` (`hline` ignores `time`, `vline` ignores `price`). `MAX_DRAWINGS_PER_ASSET = 50` (oldest fall off). `armed: { kind, assetId }` scopes a picked tool to one chart (two charts can be on screen on web). `add` validates completeness and finite numbers, stores, selects the new drawing and disarms. `selectDrawings(state, assetId)` returns a frozen shared `NO_DRAWINGS` array for empty assets - inline `[]` in a selector would loop renders under `Object.is`. Ids come from `crypto.randomUUID()` (web) / `expo-crypto` (mobile).

Migration v0 -> v1 (`migrateDrawings`): v0 stored `levels: { [assetId]: [{id, price}] }`. Each finite price becomes `{ id (kept or new), kind: 'hline', points: [{ time: 0, price }] }`, capped at 50 per asset; unreadable entries are dropped.

3.8 `favouritesStore`

Optimistic: `toggle` updates locally, then `PUT /users/me/favourites` fire-and-forget. `load()` replaces the local list with the server's (called by `useRealtime` on sign-in); a failed load keeps the local list. `recent` is device-only, most recent first, `MAX_RECENT = 8`.

3.9 `themeStore`

See section 8.

3.10 `supportStore`

Held in a store because messages arrive from three directions (own send, server echo, agent reply). `merge(existing, incoming)` keys by `id` and sorts by `createdAt` then `id`, so an echo never duplicates. `pending: { tempId, body, createdAt, failed }[]` for unsent messages; `resolvePending(tempId, message?)` removes the pending entry and merges the confirmed message; `failPending` flags it. Typing: `setAgentTyping(name)` sets `agentTypingUntil = now + 5000` (`TYPING_TTL_MS`, the server repeats the signal while typing continues); an `AGENT` message clears it immediately. `reset()` on sign-out.

3.11 `onboardingStore`

`completed` per **device**, not per account (it is an explanation, not a consent record). Replaying from Learn does not reset it.

3.12 Others

`marketStore`: `ticks[assetId]`, `previous[assetId]` (for up/down flash), `feeds`. `walletStore`: legacy single wallet + `lastChangeAt`. `ticketStore`: section 4.

---

4. Trade ticket - `features/trade/ticketModel.ts`, `ticketStore.ts`, `TradeTicket.tsx`

`ticketModel.ts` is pure and identical on both platforms; the component only renders its answers. All money is integer **minor units** (paise).

4.1 Durations

- `durationChips(asset)`: one chip per `asset.supportedExpiries`, sorted ascending; `payoutPercent` from `asset.payouts.find(p => p.expirySeconds === s)`; a duration without a payout stays visible but disabled with `unavailableReason = 'No payout for <label>'`.
- `durationLabel(s)`: `formatExpiry` below an hour (`30s`, `1m`, `1.5m`), `"1h"` / `"1h 30m"` above.
- `pickDuration(chips, wanted)`: `wanted` if usable, else the first usable, else the first chip. The ticket replaces an unusable duration in `ticketStore` **without** touching the saved default (another asset may offer it).
- `durationRow(chips, selected, max = 4)`: up to 4 chips; when more exist, the rest go behind "More" (`hidden` count), and a selected long duration takes the last slot so the selection is always visible.

4.2 Payout and stake limits

`payout = asset.payouts.find(p => p.expirySeconds === expirySeconds)`. `stakeLimits(asset, payout) = { min: max(asset.minStake, payout.minStake), max: min(asset.maxStake, payout.maxStake) }` (missing values default to 0 / `MAX_SAFE_INTEGER`).

`contractTerms(stake, payoutPercent)`:

```
profit      = round(stake * payoutPercent / 100)
totalReturn = stake + profit
maxLoss     = stake
```

The ticket's preview draft uses the same formula for `potentialReturn`. These are displays of the server's configured rate; the accepted trade's own `profit` / `potentialReturn` are what count.

4.3 Amount: presets, Max, parsing, validation

- `PRESET_AMOUNTS_MAJOR = [100, 500, 1000, 5000]` -> minor, **filtered** to `[min, max]` (a preset the server would refuse is hidden, not disabled).
- **Max** = `min(available, limits.max)`, added only when `>= limits.min`, `> 0` and not equal to an existing preset.
- `parseAmount(text)`: strips grouping commas; accepts `^\d*(\.\d{0,2})?$` (not a lone `.`); returns `toMinor(value)` or `null`.
- `amountState(text, limits, available)` - checked in the order a trader can fix them: `empty -> invalid (null or <= 0) -> below-min -> above-max -> insufficient -> valid`. Insufficient balance is reported last on purpose: telling someone to deposit for an amount the platform would refuse anyway sends them to the deposit screen for nothing.
- `amountMessage(state, currency)`: the copy, e.g. `"Not enough balance. Available: ₹1,234.00"`.
- Stepper: `amountStep(v)` = 1000 major when `v >= 10_000`, 500 when `>= 1000`, else 100; `stepAmount` clamps to `[min, max]`. `amountText(minor)` writes back without trailing `.00`. Writing an amount via preset/stepper also saves it as `settings.defaultStake`.

4.4 `ticketStore`

Holds `amountText` (the raw field text, so empty/half-typed is a nameable state), `expirySeconds` and `focusRequest`. Initialised from `settings.defaultStake` / `defaultExpirySeconds`. `setExpiry` also saves the default; `prefill(stake, expiry)` ("Trade Again" from a result card in the shell) saves both defaults and bumps `focusRequest`, which makes the ticket scroll into view and focus the amount.

4.5 Blocking

```
feedBlocked   = !asset || feed.status in {OFFLINE, STALE}
blockedReason = !asset             -> 'Choose an asset'
              : !asset.marketOpen  -> 'Market closed'
              : connection != open -> 'Live prices disconnected — reconnecting…' (also a banner)
              : feedBlocked        -> 'Price feed <status> - trading paused'
              : !payout            -> 'This duration is not available right now'
cannotTrade   = !!blockedReason || stake === null
```

`components/assetList.ts#assetAvailability` applies the same rule to list rows (`closed | paused | open`).

4.6 Placing: confirm vs one-tap, idempotency, quotedPrice, cooldown

```
press(dir):   if pending || cannotTrade: return
              confirmBeforeTrade ? open ConfirmTradeSheet(dir) : send(dir, livePrice)
confirm():    "don't ask again" -> toggleConfirm();  send(dir, reQuote?.serverPrice ?? livePrice)
send(dir,p):  setPending(dir); orderPreview.commit(draft, p); mutate({dir, quotedPrice: p ?? undefined})
```

- `livePrice = marketStore.ticks[asset.id].price ?? asset.feed.lastPrice ?? null` - the price on screen. With no tick at all, `quotedPrice` is omitted and the server applies no guard.
- **Idempotency**: a fresh `crypto.randomUUID()` (mobile: `expo-crypto`) per attempt. The server collapses retries of the same attempt; a deliberate second trade gets a new key.

- `accountType` is read from `useAccountStore.getState().active` at send time, not from the render closure.
- **Success**: clear re-quote/sheet error, `orderPreview.resolve()`, `tradeStore.upsertTrade`, invalidate `['wallet']` and `['trades']`, toast with the **server's** `entryPrice` and `potentialReturn`. Nothing is shown as accepted before this.
- **Failure**: `orderPreview.reject()` (cancelled marker). `PRICE_MOVED` with `details.serverPrice` sets a re-quote offered back to the trader at the new price; any other error goes into the sheet (confirm mode) or a toast (one-tap). The re-quote is cleared when asset, expiry or amount changes.
- **Cooldown**: `onSettled` keeps `pending` set for **350 ms** more; the buttons stay disabled from the press until then, so a double tap is never two trades.
- **Preview hooks**: web shows an INTENT preview on mouse hover or keyboard `:focus-visible` only (touch emits no matching leave). Mobile shows it on `onPressIn` and clears on release/slide-off unless the confirmation opened. While the confirmation is open the preview is re-shown. Changing asset or account clears the intent and closes the confirmation; unmount clears it.
- Mobile adds a light haptic on placement (`Haptics.impactAsync(Light)`, errors swallowed).

4.7 Early close ("Sell now")

`PositionsPanel.tsx#OpenPosition`: query `['trade', id, 'quote']` -> `api.trades.quote(id)`, `enabled: running && status === 'OPEN'`, `refetchInterval: 2000`, `staleTime: 1000`. The quote is display-only; `POST /binary/trades/:id/close` re-prices server-side, so a stale on-screen quote cannot be traded against. Button text `Sell now <quote.value>` when `quote.available`, otherwise `quote.unavailableReason ?? 'Pricing...'`, or `'Settling…'` when the countdown is over. On success: `settleTrade`, `announceResult`, invalidate wallet and trades, toast `"<value> returned (<signed customerResult>)"`. A row whose countdown hit zero while still `OPEN` is displayed as `SETTLING`.

---

5. Results: projection, settlement, draw rule, status words

5.1 Live projection - `features/trade/liveResult.ts`

What a running trade **would** pay if it ended at the price on screen. Worded as a projection ("If it ends now"); never a result.

```
liveResult(trade, price, drawPolicy = 'REFUND'):
  price not finite             -> null
  price == entryPrice          -> REFUND: {draw, 0} | LOSS: {losing, -stake} | WIN: {winning, profit}
  winning = UP ? price > entry : price < entry
  winning ? {winning, +trade.profit} : {losing, -trade.stake}
```

The amounts are the ones the server wrote on the trade (`profit`, `stake`). `liveTotal(trades, priceOf, now, drawPolicy)` sums over trades with `expiryTimestamp > now` and reports `running` and `priced` counts separately, so a total never silently covers fewer trades than the line claims. Expired-but-unsettled trades are excluded (their outcome depends on a price the screen may not have seen). `amountTone(a)` -> `up | down | neutral`. `STANDING_WORDS`: `winning`, `losing`, `draw, refunded`.

5.2 Settlement display - `features/trade/settlement.ts`

- `settlementOf(trade)`: `trade.settlementAmount` when present - already **net and signed** (`payout - stake` on a win, `-stake` on a loss, `0` on a draw, the customer's result on an early close). Fallbacks only for a terminal row missing the field: WON -> `profit`, LOST -> `-stake`, DRAW/CANCELLED -> `0`, else `null`.
- `payoutOf(trade) = stake + (settlementOf(trade) ?? -stake)`: what came back to the wallet (full payout on a win, `0` on a loss, the stake on a draw). The result popup shows this, so a loss reads as zero, not negative.

Both the chart overlay and the result popup use these, so they cannot disagree.

5.3 Draw rule wording - `features/trade/drawRule.ts`

`drawPolicyOf(asset) = asset.drawPolicy ?? 'REFUND'` (sent by the server with each asset). `TIE_SENTENCE`, `TIE_CLAUSE`, `TIE_LABEL` give the copy per policy (`REFUND/LOSS/WIN`). `useDrawPolicy(assetId?)` reads the shared `['assets']` query (no extra request) and falls back to the first asset's policy. The `DRAW` status only exists under REFUND; under LOSS/WIN a tie settles as LOST/WON.

5.4 Status words and tones - `utils/tradeStatus.ts`

One terminology everywhere: `OPEN 'Active'`, `SETTLING 'Settling'`, `WON 'Won'`, `LOST 'Lost'`, `DRAW 'Draw · refunded'`, `CANCELLED 'Cancelled'`, `CLOSED_EARLY 'Sold early'`. `statusTone`: OPEN/SETTLING `live`, WON `up`, LOST `down`, CLOSED_EARLY by the sign of `settlementAmount`, else `neutral`. `isActive(status)` = OPEN or SETTLING. `durationOf(trade) = round((expiry - entry) / 1000)`.

---

6. Chart logic

6.1 History, timeframes and live tick folding

`PriceChart.tsx` (both platforms) loads `['candles', assetId, timeframe]` = `api.market.candles(id, 300, timeframe)` with `refetchInterval: max(15_000, timeframe * 1000)`. Bars for every timeframe, including the sub-minute ones (5/15/30 s), come from the server; the client does not aggregate history. `bucketMs = timeframe * 1000`. In beginner mode the chart type is restricted to `BEGINNER_CHART_TYPES = ['candles', 'line', 'area']` (`ChartToolbar.tsx`) and indicators/drawings are hidden (but kept).

When a candle response lands: the bars are copied into `barsRef`, painted, `lastBucketRef` = last bar, and `seriesFrameRef = bucketMs` ("the series now holds this frame"). The view jumps to the live edge only when the dataset changed (asset or timeframe) or the view was already following (`isFollowingRealTime`: `|scrollPosition - rightOffset| < 1.5` bars), and only with `autoScroll`. A background refetch never drags a trader out of history.

Live tick folding into the open bucket:

```
if seriesFrameRef != bucketMs: skip          // timeframe changed, refetch in flight
openTime = floor(tick.timestamp / bucketMs) * bucketMs
if previous && openTime < previous.time: skip   // never fold backwards
bucket = previous.time == openTime
  ? { open: prev.open, high: max(prev.high, p), low: min(prev.low, p), close: p }
  : { open: p, high: p, low: p, close: p }      // a new bar
replace last bar or push
```

The guard exists because a tick bucketed with a new, wider frame (`floor(now / 3h)`) before the refetch lands would be older than the newest bar, which makes `lightweight-charts` throw "Cannot update oldest data" and takes the chart down. Skipping is safe: the in-flight refetch contains that bucket. Times are in ms in the client and divided by 1000 for `lightweight-charts` ' `UTCTimestamp` .

Other display rules: a DEMO watermark under the series when the active account is DEMO (font size `clamp(round(width * 0.17), 44, 150)` , per-theme alpha 0.06/0.075/0.055); axis and crosshair labels use the chosen time zone (formatting only); grid colour alpha = `gridOpacity / 10` .

6.2 Indicators - `utils/indicators.ts` , `features/trade/chart/indicatorCatalog.ts`

Display aids from the loaded bars only; never sent to the server. Every function returns points only where its window is fully populated (no padded heads).

Indicator	Params (fixed)	Formula
SMA	20	`SMA_i = (1/n) * sum(close_{i-n+1..i})` , first point at bar `n-1` (running sum)
EMA	50	`k = 2/(n+1)` ; seed = SMA of first `n` ; `EMA_i = (close_i - EMA_{i-1})*k + EMA_{i-1}`
Bollinger	20, 2	middle = SMA20; `sigma` = **population** std-dev (divide by n) of the window; bands = `middle +/- 2*sigma`
RSI	14	Wilder: seed avg gain/loss over changes 1..n (first value on bar `n`); then `avg = (avg*(n-1) + x)/n` ; `RSI = 100 - 100/(1 + G/L)` ; `L=0` : 100, both 0: **50**
MACD	12, 26, 9	`macd = EMA12 - EMA26` from bar 25 (offset `slow-fast` aligns the arrays); `signal = EMA9(macd)` ; `histogram = macd - signal`
Stochastic (slow)	14, 3, 3	raw %K = `(close - LL14)/(HH14 - LL14) * 100` (range 0 -> **50**); `%K = SMA3(raw)` ; `%D = SMA3(%K)`

`INDICATORS = ['sma','ema','bollinger','rsi','macd','stochastic']` (`settingsStore`); stored lists are filtered with `isIndicator` (a list from another build may name an unknown one). SMA, EMA and Bollinger are `overlay` placement (on price; colours: warn, 60% strong text, crosshair grey); RSI/MACD/Stochastic are `pane` oscillators drawn in bands under the price in `INDICATOR_ORDER` (guide lines RSI 70/30, Stochastic 80/20). `searchIndicators(query)` requires every typed word to appear in name, short name, params, category label or keywords. Recomputation is throttled by `shouldRun` (6.8) with key `indicators | theme | candles.dataUpdatedAt` .

Oscillator bands are separate charts that follow the main chart's visible logical range; every price axis is widened to the widest one so bar `i` sits at the same x in every band (web: `syncAxisWidths` ; mobile: shared `engine.axisW`).

6.3 Logical-index time projection - `features/trade/chart/timeProjection.ts`

Both chart engines lay the time axis out **by bar index**, not by time. Anything drawn at an instant (entries inside a bar, future expiries, drawing points) goes through a fractional logical index:

```
timeToLogical(ms, bars, bucketMs):
  ms >= last.time  -> (count-1) + (ms - last)/bucketMs        // extrapolate into the future
  ms <  first.time -> (ms - first)/bucketMs                   // extrapolate into the past
  else i = lastAtOrBefore(bars, ms)                           // binary search
       -> i + min(1, (ms - bars[i].time)/bucketMs)            // gaps (closed market) held at next slot
logicalToTime = inverse (linear extrapolation outside the data)
x = x0 + logical * spacing
```

On web, `readLogicalAxis(chart)` recovers `x0` and `spacing` from `timeScale().logicalToCoordinate(0)` and `(1)`, because the library answers 0 for non-integer indices. On mobile the engine's `View` already carries `x0` and `s`. Mobile duplicates every function over flat time arrays (`*Times`) and marks them `'worklet'`, so the UI thread and the JS tap handlers run the same code. The orders, expiry guide and drawings all project through this, so a trend line drawn to an order's expiry sits exactly on it.

6.4 Drawings geometry - `features/trade/chart/drawingGeometry.ts`

Pure pixel geometry after projection:

- `FIB_LEVELS = [0, 0.236, 0.382, 0.5, 0.618, 1]` ; `fibPrice(start, end, l) = end - (end - start) * l` (0 at the end of the move, 1 at its start; first click is the start).
- `distanceToSegment` (clamped projection), `rayEnd(a, b, plot)` (a point beyond the plot along a->b: max corner distance + plot diagonal), `boxOf` .
- `hitsDrawing(shape, p, plot, tolerance)` : hline by `|dy|` , vline by `|dx|` , trend/ray by segment distance, rect/fib anywhere inside the box +/- tolerance. `topmostHit` scans from the last (top-most) drawing. Tolerances (`DrawingLayer.tsx`): web `MOUSE_REACH 6` , `TOUCH_REACH 14` , slop 4/10 px, click <= 700 ms; mobile `TOUCH_REACH 14` .
- `actionAnchor` : where the delete button hangs (right end of hline, top of vline, the higher end of a line, top-right of a box).

6.5 Expiry guide - `features/trade/chart/ExpiryGuide.tsx`

A vertical, grey, dotted line at `Date.now() + expirySeconds * 1000` , labelled with the clock time on the time axis. Shown only when `showExpiryGuide` , the market is open, the chart is the one the ticket trades (`activeAssetId`), and the asset supports the ticket's duration; hidden while an order preview for that asset is on the chart. Visually distinct from orders (no price, no amount). Web moves DOM nodes from a rAF loop (`useFrameLoop` , 30 fps in performance mode); mobile records a Skia picture driven by `useFrameClock` .

6.6 Order layer - `features/trade/orders/orderModel.ts` , `ChartOrderLayer.tsx` , `useOrderProjection.ts`

`buildOverlayOrders({assetId, preview, openTrades, settled, seenAt, now})` merges, in paint order: recently settled trades (each lingers `SETTLED_LINGER_MS = 6000` from when **this client first saw** it settled - not from `settledAt` , which a lagging clock could make already expired), open trades, then the preview last (on top).

Visual states: `PREVIEW | ACTIVE | EXPIRED | WON | LOST | CANCELLED`. Mapping: DRAW and SETTLING -> EXPIRED; OPEN past expiry -> EXPIRED with `awaitingSettlement` ("Settling"); CLOSED_EARLY -> WON if `settlementAmount >= 0` else LOST; REJECTED preview -> CANCELLED (lingers `CANCELLED_LINGER_MS = 1800`). An INTENT preview `tracksClock` (entry = now, expiry = now + duration, price = live, all recomputed per frame so it glides); a PLACING preview is pinned. `entryPrice`, once non-null, is never overwritten. Colours (`orderTone`): preview green/red dashed; placed orders **brand yellow** regardless of direction; outcome colour once settled; expired/cancelled grey and faded. `needsHeartbeat` keeps the 1 s heartbeat only while something is ACTIVE, PREVIEW or fading. `isInTheMoney` is indicative only.

Right-offset room-making (web `ChartOrderLayer.tsx`), only when the view is at the live edge and `autoScroll` is on:

```
needed = 6                                                    // BASE_RIGHT_OFFSET
guide:   needed = max(needed, ceil(guideSeconds*1000 / bucketMs) + 2)
pending orders: needed = max(needed, applied, 18)             // ORDER_RIGHT_OFFSET, hold what was made
                for each: needed = max(needed, ceil((expiry - lastBar.time)/bucketMs) + 2)
needed = min(needed, 42, max(6, floor(plotWidth * 0.5 / barSpacing)))
```

Projection: web `useOrderProjection` runs a rAF loop (60 fps, 30 in performance mode) writing DOM styles only on a change > 0.5 px, because the price scale autoscales on every tick without an event. Mobile computes the same geometry as a Reanimated derived value on the UI thread.

6.7 Trade-window chart data - `features/trade/chart/tradeWindow.ts`

The trade detail chart (`TradeWindowChart.tsx`, query `['trade-window', id]`, `staleTime Infinity`, refetch every 15 s only for a running trade drawn from candles, `retry 1`):

1. `tickWindow(trade)`: `pad = clamp(0.25 * duration, 10 s, 120 s)`; `stepMs` = 500 (<= 1 min), 1000 (<= 5 min), else 2000; `from = entry - pad`, `to = expiry + pad`.
2. If `ticksRetained(from, now)` (`now - from < 15 min - 45 s`), fetch ticks (`windowMs = clamp(now - from + 5 s, 1 s, 15 min)` for older servers); use them only if `ticksReachEntry`.
3. `resampleTicks`: a regular grid anchored at the entry (`time = entry + k*step`), each point takes the last tick at or before it (the server's expiry-price rule), future points and points before the first tick are whitespace, and on-grid `pins` force the recorded entry/exit prices.
4. Otherwise one-minute candles: `candleWindow` = 20 min before entry to `min(now, expiry + 5 min)`, `limit = clamp(minutes + 2, 10, 1000)`; `candlesForWindow` returns null unless a candle contains the entry (covers a server that ignored `to`). Fallback retry with `MAX_CANDLES = 1000` latest.
5. `tradeMoments`: exit = sold-at time for CLOSED_EARLY, else expiry; null while running or for CANCELLED.

6.8 Redraw throttling - `utils/redraw.ts`

`shouldRun(ref, key, performanceMode, now)`: always runs when `key` changed (a structural change the trader caused); a repeat under the same key is skipped only when performance mode is on and less than `PERFORMANCE_REDRAW_MS = 1000` passed. Performance mode never delays input.

6.9 The mobile Skia engine - `mobile/src/features/trade/chart/engine/`

The engine replaces `lightweight-charts` but keeps its model, so everything projected on web projects identically.

- **Threads** (`types.ts`): the JS thread owns data (history, ticks, indicators, orders) and writes Reanimated shared values; the UI thread owns the viewport and repaints in worklets. A pan or pinch never costs a React render.
- **Series as flat arrays** (`SeriesData { t, o, h, l, c, bucket }`), so a tick updates the last element in place (`series.modify` worklet) instead of copying a thousand objects across threads. `NaN` in `c` is whitespace. The tick fold on mobile also checks `data.bucket === bucketMs` , and a new bar bumps `rightOffset` by 1 when the newest bar is not visible, so a view parked in history stays put.
- **Viewport** (`useChartEngine.ts`): shared values `size` , `axisW` (56 initially), `spacing` (initial 10, min 0.5, max 50), `rightOffset` , `restingOffset` (6), `series` , `extents` (overlay indicator values the price scale fits to), `closeOnly` (line/area), `minMove` , `crosshair` . The derived `view` = `fitPrices(horizontal(...))` .
- **Horizontal** (`scale.ts`): `x0 = pw - (count - 1 + rightOffset + 0.5) * s` - the lightweight-charts placement; visible `first/last` from the left/right logical edges.
- **Price autoscale**: min/max of visible highs/lows (closes for line/area) and visible overlay values; empty view holds the newest bar; flat range padded by `max(|hi|*0.0005, 5*minMove, 1e-9)` ; margins applied as `lo - span*bottom/inner` , `hi + span*top/inner` , `inner = max(0.05, 1 - top - bottom)` . Oscillators can pass a `FixedRange` .
- **Gestures**: `Gesture.Race(longPress-crosshair (260 ms), Simultaneous(pan, pinch), tap (<= 700 ms, <= 10 px))` . Pan is horizontal only (`activeOffsetX +/-6` , `failOffsetY +/-12` , so the page can still scroll vertically); dragging the time axis scales spacing by `exp(dx / 220)` ; a fling uses `withDecay` (deceleration 0.997) clamped by `clampOffset` (at least 2 bars stay on screen). Pinch zooms about the fingers (`offsetKeeping`) or keeps the newest bar fixed when following. `isFollowing` = `|rightOffset - restingOffset| < 1.5` ; `scrollToRealTime` animates 260 ms; `fitContent` waits until there are bars and a size.
- **Tick marks**: `priceMarks` picks a nice step from `[1, 2, 2.5, 5, 10] * 10^k` about 34 px apart, never finer than the instrument precision; `timeMarks` picks from a step table (1 s ... 30 d) so labels are about 64 px apart, labels a day change with the date, seconds below a minute.
- **Time zones on the UI thread**: `Intl` is too slow per label per frame, so `buildZoneInfo` computes piecewise UTC offsets on the JS thread (sampled every 3 days, each switch narrowed by bisection to 1 s) and the painter formats from that table.
- **Painting**: `mainPainter.ts#paintMain` records grid, series (`paintCandles` draws wick-only below 3 px spacing, body `floor(0.8 * s)` forced odd), overlays, last-price line/tag, axes, crosshair (magnet to close on the price chart) and the DEMO watermark into one `SkPicture` via `useDerivedValue(createPicture(...))` . Layers (`host.ts` : `drawings` , `guide` , `orders`) record their own pictures and are drawn into the same `Canvas` in that order. Taps are offered to layers by priority (`orders 30` , `drawings 10`) before falling through to the crosshair.
- **Clock** (`useFrameLoop.ts#useFrameClock`): a `useFrameCallback` clock that writes `now` only when it would move something by half a pixel or change a second label (half rate in performance mode).

---

7. Formatting, time, export, search, asset list

7.1 Money - `utils/format.ts`

`MINOR = 100`; `toMinor(major) = round(major * 100)`, `fromMinor(minor) = minor / 100` - the only conversion points, so a display value cannot leak into a request. `formatMoney` uses `en-IN` currency formatting with exactly 2 decimals (default INR); `formatSignedMoney` prefixes `+` for positive; `formatMoneyCompact` drops `.00` only for whole amounts (chart tags); `formatPrice(p, precision)` uses `en-US` with fixed precision; `formatExpiry` (`30s`, `1m`, `1.5m`, `1.0h`); `formatCountdown` `mm:ss` with `ceil` and clamped at `00:00`. `currencySymbolFor` asks `Intl.NumberFormat#formatToParts`; mobile adds a `KNOWN_SYMBOLS` fallback (`INR ₹, USD $, EUR €, GBP £`) for engines without `formatToParts`.

7.2 Time - `utils/time.ts`

The server speaks epoch ms; a zone only changes how an instant is written. `SYSTEM_ZONE = 'SYSTEM'` resolves via `Intl...resolvedOptions().timeZone || 'UTC'`. `TIME_ZONES` lists 11 zones plus system. `zoneOffsetMinutes` formats the instant in the zone and rebuilds it as UTC (hour 24 normalised), per instant so DST is correct on both sides. Labels: `zoneOffsetLabel` `UTC+05:30`, `zoneChip` `UTC+5:30`, `formatClock` `HH:mm:ss`, `formatHourMinute`, `formatDayTime` `5 Sep 12:30`. `hooks/useCountdown.ts`: `useCountdown(target, 250ms)` recomputes from the wall clock each tick (correct after backgrounding); `useNow(1000)` aligns ticks to whole seconds so every clock on screen turns over together.

7.3 USDT - `utils/usdt.ts`

`MICRO_PER_USDT = 1_000_000`, `USDT_DECIMALS = 6`. No float arithmetic: `parseUsdt` matches `^(?:(\d+)(?:\.(\d{0,6}))?|\.(\d{1,6}))$`, pads the fraction to 6 digits and converts the digit string (max 16 digits, must be a safe integer); commas, signs, exponents are refused. `formatUsdt(micro, {trim, unit, signed})` prints 6 decimals by default (`trim` keeps at least 2), groups thousands, `--` for non-finite. `toUsdtInput` writes an amount back into a field so it parses to the same value.

7.4 CSV - `utils/csv.ts`

`escapeCsvValue`: values starting with `= + - @ \t \r` get a leading `'` (formula injection); fields with `" , \n \r` are quoted with doubled quotes. `toCsv(rows, columns)` uses CRLF. Web `downloadCsv` prepends a UTF-8 BOM and clicks a Blob link (revoked after 1 s). Mobile `shareCsv(filename, csv)` writes the BOM-prefixed file to `Paths.cache` with `expo-file-system` and opens the share sheet (`expo-sharing`, `text/csv`).

7.5 Day grouping - `utils/dayGroups.ts`

Days are counted in the **chosen display zone**, on calendar keys (`"2026-09-30"`) so DST never produces a 23/25-hour day. `dayKey`, `shiftDay`, `daysBetween`, `weekdayOf`, `dayLabel` (`Today`, `Yesterday`, `Mon 28 Sep`, year added when different). `groupByDay(items, timeOf, now, zone)` keeps input order; an out-of-order item joins its existing group.

7.6 Text search - `utils/textSearch.ts`

`nodeText` extracts the words from JSX (Learn guides are searched from the rendered tree, not a copy). `words` lower-cases, strips accents (NFKD), normalises curly quotes, splits on non letters/digits. `matchesQuery`: every query word must be a **prefix** of some word in the text ("draw" does not match "withdrawal"; "refun" matches "refunded"); empty query matches all.

7.7 Asset list rules - `components/assetList.ts`

- `assetAvailability`: `!marketOpen -> closed`, feed STALE/OFFLINE -> `paused`, else `open`. `openCount` counts only `open`.
- `bestPayout` = max `payoutPercent`. `categoriesOf` in first-seen order. `catalogueOrder` by `displayOrder`.
- `assetsInFilter`: `all | favourites | recent | category:<name>`; Recent keeps recency order.
- `matchesSearch`: case-insensitive substring of symbol or name.
- `sortAssets`: `default` (chip order), `payout` (desc, tie -> `displayOrder`), `name` (numeric, base-sensitivity compare, tie -> `displayOrder`), so rows do not reshuffle between refreshes.
- `formatNextOpen(nextOpenAt, now, zone)`: `Opens 14:30` (today), `Opens Mon 09:00` (< 7 days), `Opens 12 Oct 09:00`; `null` when unknown or already past.

---

8. Theme system

Tokens. The palette is a set of named tokens stored as space-separated RGB channels (`"246 216 74"`) so Tailwind/NativeWind can apply alpha (`bg-accent/60`). Groups: `surface-950..500`, `content-100..400`, `accent(-soft|-solid)`, `up/down(-soft|-solid|-bright)`, `warn`, `critical`, `brand`, `scrim`, `chart-grid|axis|crosshair|bg|bull|bear|price-line`.

Three themes + system. `ThemePreference = 'light' | 'twilight' | 'dark' | 'system'`; `THEMES` is that list, and `cycle()` steps through it in that order (`light -> twilight -> dark -> system`; the web doc comment still says "light -> dark -> system"). "System" follows the OS live (`watchSystemTheme`) and resolves only to `light` or `dark`.

- **Web** (`client/src/stores/themeStore.ts`): tokens are CSS custom properties in `client/src/index.css` (`:root` = dark, `:root[data-theme='twilight']`, `:root[data-theme='light']`). The resolved theme is written to `<html data-theme>` and `color-scheme`; the preference to `localStorage['bt.theme']`. An inline script in `client/index.html` applies the same key before first paint (change both together). `utils/theme-colors.ts` reads tokens with `getComputedStyle` and converts to hex/ `rgba(a, b, c, x)` because chart libraries cannot parse `rgb(1 2 3)` (lightweight-charts throws "Cannot parse color").
- **Mobile** (`mobile/src/theme/tokens.ts`): `THEME_TOKENS: Record<'dark'|'twilight'|'light', TokenTable>` with the same values; `ThemeRoot.tsx` sets them as NativeWind `vars()` on the root view (they cascade through context, so sheets/modals get them), sets the status-bar style and the system window background. `tokenColor(theme, name, alpha)` serves Skia and other colour props; `readChartPalette(theme)` builds the chart palette (e.g. grid = `chart-grid` at 0.55 alpha). `SCRIM_ALPHA = { dark 0.72, twilight 0.7, light 0.45 }`. The store persists only `preference` and recomputes `resolved` on rehydrate; system theme comes from `Appearance`.

Parity and contrast tests (`mobile/tests/theme-contrast.test.ts` , web counterpart `client/tests/theme-contrast.test.ts` reading `index.css`):

- Parity: parses `client/src/index.css` token blocks (`:root {` , `:root[data-theme='twilight'] {` , `:root[data-theme='light'] {`) and asserts every mobile token exists there with the same value.
- WCAG contrast per theme: text pairs >= 4.5:1 (content-100/200/300 on surface-950, content-400 on surface-900 and 850, accent/up/down/warn/critical on surface-900); primary button label (`surface-950` on `accent-solid`) >= 4.5; white on `down-solid` >= 4.5; white on `up-solid` >= 3 (large bold text).
- Surface hierarchy: page vs panel >= 1.15, panel border (`surface-700`) vs panel >= 1.4, input (`surface-850`) vs panel >= 1.03, border ramp `800 < 700 < 600 < 500` increasingly distinct from the panel, scrim luminance < 0.1.

---

9. Mobile-specific differences and test setup

Differences from web (README "Differences from the web client", `PORTING.md`):

- **Phone layout only**: the < 640 px variant of every responsive web layout; no split two-chart view, no icon rail, no wide tables. Tabs (`app/(app)/(tabs)`): Home, Markets, Trade, Trades, Account; other screens are pushed. Overlays mounted once in `app/(app)/_layout.tsx` : `SettingsDrawer` , `TransferModal` , `TradeResultToasts` , `Toaster` .
- **Sheets** (`components/ui/Sheet.tsx`) replace web popovers, dropdowns and modals.
- **Deposit proof is image-only**: `expo-image-picker` from the photo library, single image, `Compatible` representation (HEIC -> JPEG); types PNG/JPEG/WebP, max 5 MB (`features/transfer/Transfer.tsx`). Web also accepts PDF.
- **In-app date pickers**: `features/trades/DateRangeField.tsx` opens a month calendar in a sheet, values are calendar keys (`YYYY-MM-DD`), start <= end enforced.
- **CSV** opens the share sheet (`shareCsv`) instead of downloading.
- **Haptics** (`expo-haptics` , always `.catch` -guarded): light impact on trade placement, a success/error/warning notification on a result (`TradeResultToasts.tsx`).
- Tab screens stay mounted, so polling screens use `hooks/useScreenFocused.ts` to pause when another tab is in front; `hooks/usePullRefresh.ts` shows a spinner only for a pull, not for polls.
- Chart preview on press-in instead of hover; drawings and order markers are hit by touch only.
- Platform substitutions: `expo-crypto` `randomUUID` , `setTimeout` without `window.` , SecureStore for tokens, AsyncStorage for persisted stores.

Test setup. Both apps run vitest over pure-logic suites (`client/tests/*.test.ts` , `mobile/tests/*.test.ts` , same suite names: active-trades, asset-list, auth-schemas, chart-colors, chart-geometry, csv, day-groups, duration-row, indicators, live-result, redraw, store-selectors, text-search, theme-contrast, time, trade-ticket, trade-window, usdt-model, usdt-money). `mobile/vitest.config.ts` runs in `node` and aliases native packages to in-memory stand-ins under `mobile/tests/mocks/` : `@react-native-async-storage/async-storage` , `expo-secure-store` , `expo-crypto` (Node `randomUUID`), `expo-constants` , `expo-file-system` , `expo-sharing` (`isAvailableAsync -> false`) and `react-native` (`Appearance` fixed to dark, `Platform`). Commands: `npm test` and `npm run typecheck` in `mobile/` (which is not part of the root npm workspace), `npm test` in `client/` .

---

10. Where to look

Topic	Web	Mobile	Identical?
HTTP client, tokens, refresh	`client/src/services/http.ts`	`mobile/src/services/http.ts`	no (storage, multipart)
Endpoints	`client/src/services/api.ts`	`mobile/src/services/api.ts`	no (`UploadFile`)
API URL / WS URL	`VITE_API_URL` , `VITE_WS_URL`	`mobile/src/config.ts`	-
App boot, QueryClient	`client/src/main.tsx` , `client/src/App.tsx`	`mobile/src/app/_layout.tsx`	-
Auth store	`client/src/stores/authStore.ts`	`mobile/src/stores/authStore.ts`	no (hydrate retry)
Websocket	`client/src/websocket/client.ts`	`mobile/src/websocket/client.ts`	no (URL, timers)
Realtime bridge	`client/src/hooks/useRealtime.ts`	`mobile/src/hooks/useRealtime.ts`	yes
Live feeds, connection, open trades, trading wallet	`client/src/features/trade/use{LiveFeeds,ConnectionState,ActiveTrades,TradingWallet}.ts`	same paths	yes
Stores	`client/src/stores/*.ts`	`mobile/src/stores/*.ts` (+ `storage.ts`)	logic yes, storage/theme no
Ticket rules	`client/src/features/trade/ticketModel.ts` , `ticketStore.ts`	same	yes
Ticket component	`client/src/features/trade/TradeTicket.tsx`	`mobile/src/features/trade/TradeTicket.tsx`	no
Early close	`client/src/features/trade/PositionsPanel.tsx`	same path	no (UI)
Projection / settlement / draw rule	`client/src/features/trade/{liveResult,settlement,drawRule,announceResult}.ts`	same	yes
Status words	`client/src/utils/tradeStatus.ts`	same	yes
Main chart	`client/src/features/trade/PriceChart.tsx`	`mobile/src/features/trade/PriceChart.tsx`	no
Indicators	`client/src/utils/indicators.ts` , `features/trade/chart/indicatorCatalog.ts`	same	yes
Time projection	`client/src/features/trade/chart/timeProjection.ts`	same path (worklets + `*Times`)	no
Drawings	`chart/drawingGeometry.ts` , `chart/DrawingLayer.tsx` , `stores/drawingsStore.ts`	same	geometry yes
Expiry guide	`chart/ExpiryGuide.tsx` , `chart/useFrameLoop.ts`	same paths (`useFrameClock`)	no
Orders overlay	`orders/orderModel.ts` , `orders/ChartOrderLayer.tsx` , `orders/useOrderProjection.ts`	same	model yes

Topic	Web	Mobile	Identical?
Trade detail chart	`chart/tradeWindow.ts`, `chart/TradeWindowChart.tsx`	same	model yes
Redraw throttle	`client/src/utils/redraw.ts`	same	yes
Skia engine	-	`mobile/src/features/trade/chart/engine/*`	mobile only
Formatting	`client/src/utils/{format,time,usdt}.ts`	same	format differs (symbol fallback)
CSV, days, search	`client/src/utils/{csv,dayGroups,textSearch}.ts`	same	csv differs (share sheet)
Asset list rules	`client/src/components/assetList.ts`	same	yes
USDT queries/keys	`client/src/features/usdt/queries.ts`	same	yes
Theme	`client/src/stores/themeStore.ts`, `client/src/index.css`, `client/index.html`, `client/src/utils/theme-colors.ts`	`mobile/src/stores/themeStore.ts`, `mobile/src/theme/{tokens.ts,ThemeRoot.tsx}`, `mobile/src/utils/theme-colors.ts`	no
Tests	`client/tests/*.test.ts`	`mobile/tests/*.test.ts`, `mobile/tests/mocks/*`, `mobile/vitest.config.ts`	suites mirrored
Port contract	-	`mobile/PORTING.md`, `mobile/README.md`	-